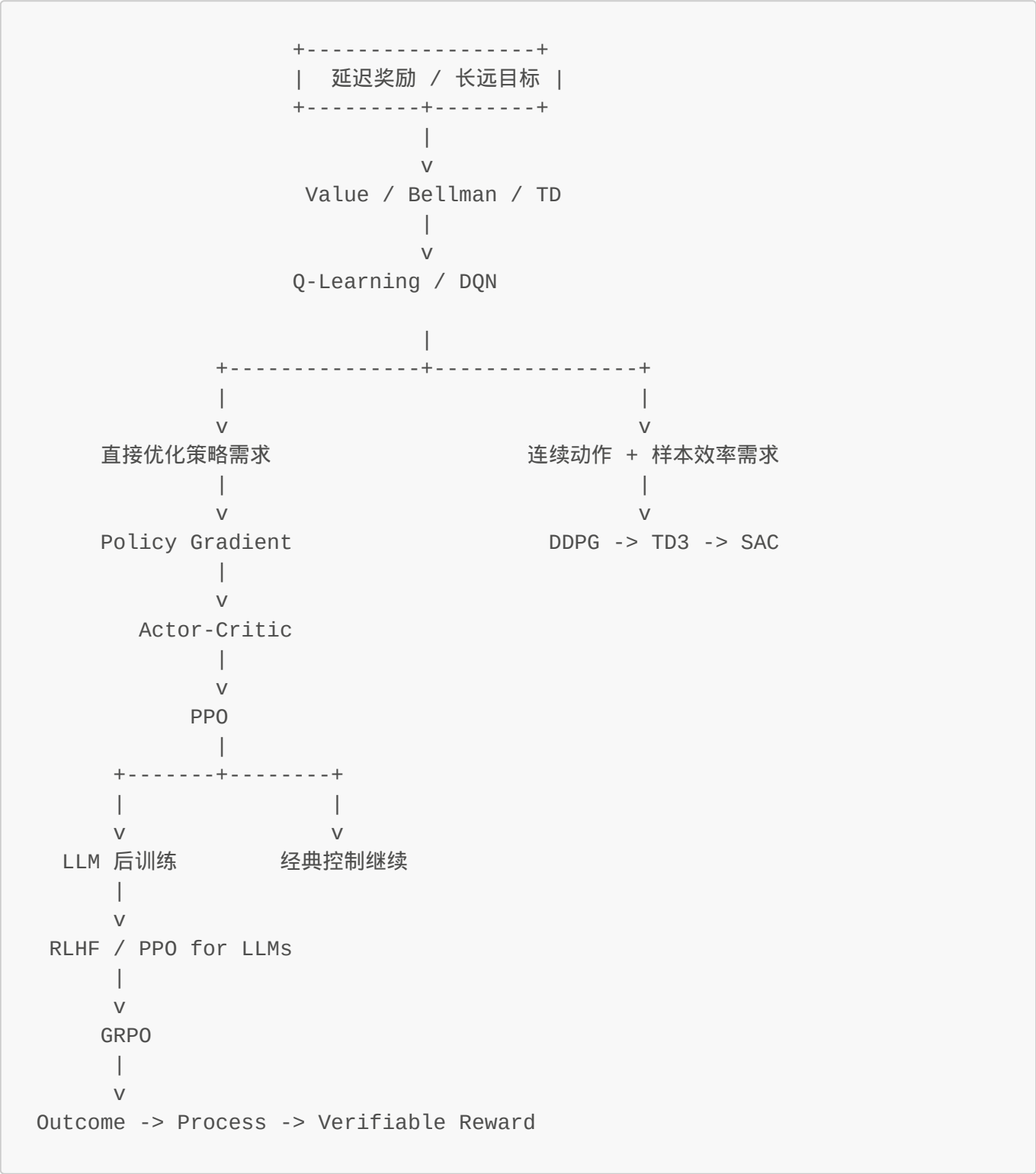


强化学习教程 (RL Tutorial) - 完整版

本文档由 [RL_Tutorial.md](#) 及各章节文件自动合并生成 生成时间: 见文件修改时间

全局导航图

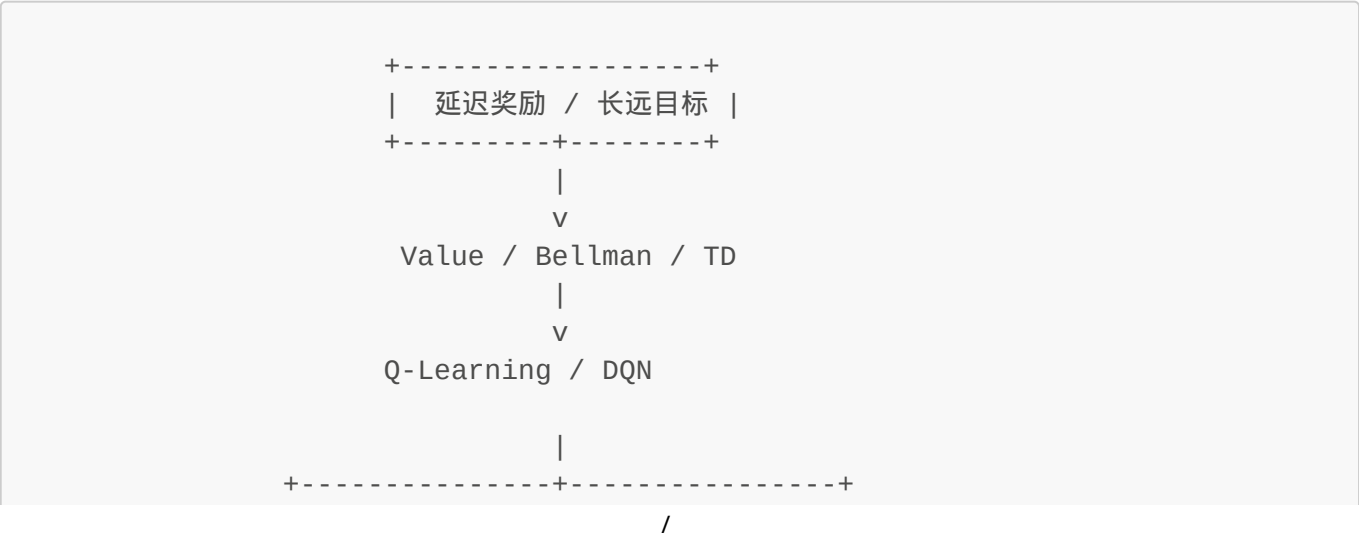


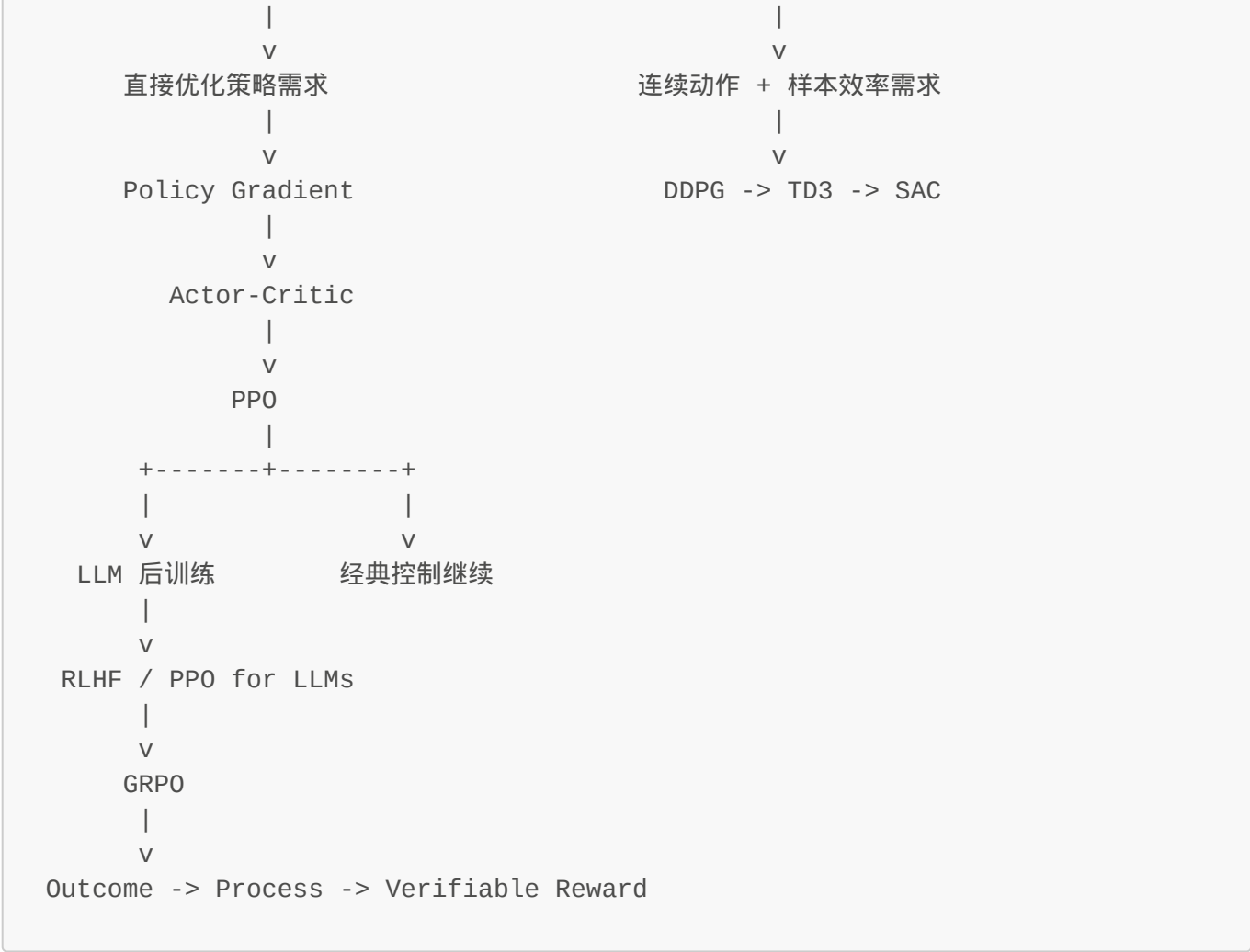
章节目录

章节	标题
第1章	RL 是什么？
第2章	Policy（策略） n
第3章	Value Function 与 Bellman 方程
第4章	Q-Learning（让 Agent 开始真正学习）
第5章	Deep Q-Network (DQN) - 用神经网络替代 Q 表
第6章	Policy Gradient - 直接优化策略
第7章	Actor-Critic - 结合 Policy 和 Value
第8章	PPO - 给策略更新套上"安全护栏"
第9章	LLM 分支入口 - 为什么 GRPO 应该放在 PPO 之后？
第10章	从经典 RL 到 LLM 对齐 - 为什么语言模型也会走到 RLHF？
第11章	PPO for LLMs - 当动作变成 token，PPO 还在优化什么？
第12章	GRPO - 在 LLM 场景里，为什么"组内相对比较"会自然替代一部分 critic 角色？
第13章	DDPG - 为什么连续动作会逼出"确定性 Actor-Critic"？
第14章	TD3 - 为什么 DDPG 会被"双 Critic + 延迟更新"继续修正？
第15章	SAC - 为什么最大熵原则会逼出"既学回报，也保留随机性"的算法？
第16章	Outcome Reward - 为什么"只看最终答案对不对"既强大又不够？
第17章	Process Reward - 为什么只奖最终答案还不够？
第18章	Verifiable Reward - 为什么 reasoning 训练越来越依赖"可自动检查"的中间与最终信号？
第19章	总复习总览 - 把整本 RL 教程压成一张因果地图

第一章：RL 是什么？

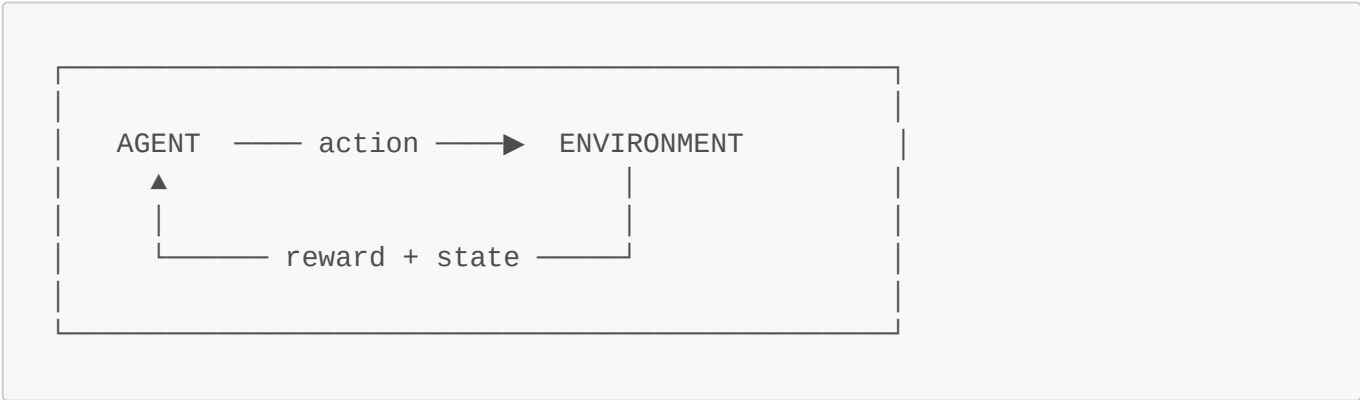
全局导航图





你在这里：主干起点 -> RL 基本元素

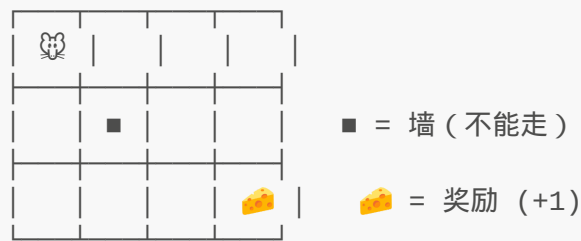
核心循环



Agent 活在一个循环里：

1. **See** - 观察当前状态 (state)
2. **Do** - 执行动作 (action)
3. **Get** - 收到奖励 + 新状态 (reward + new state)
4. **Repeat** - 不断循环，慢慢变聪明

具体例子：格子迷宫 🐹



概念	定义	迷宫例子
State (s)	Agent 当前观察到的情况	鼠标所在格子 (row, col)
Action (a)	Agent 可以执行的操作	↑ ↓ ← →
Reward (r)	环境给的反馈信号	到达🧀得 +1，其他为 0

为什么用 RL 而不是写规则？

传统编程：

规则 → 电脑 → 输出

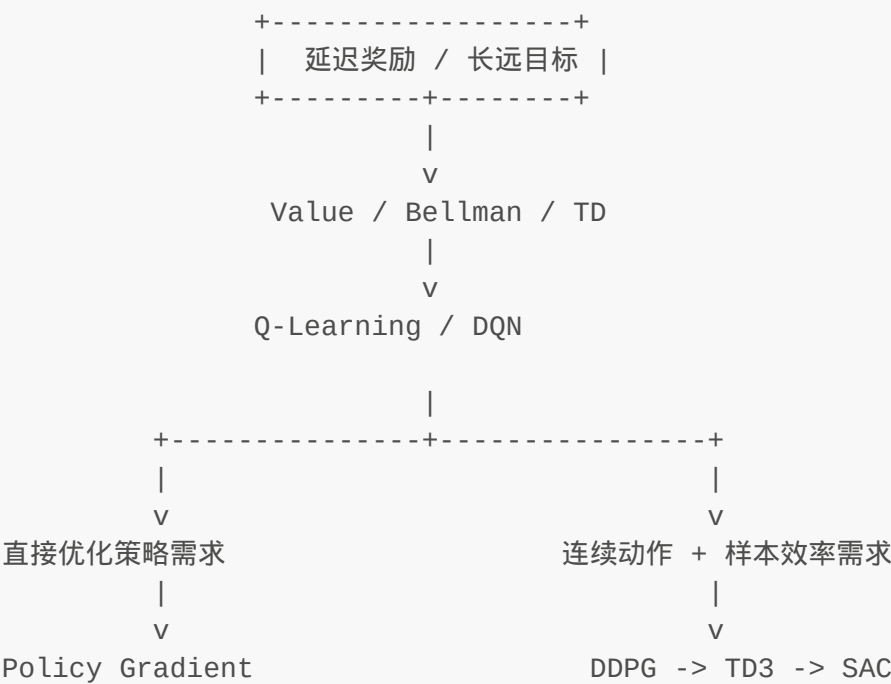
强化学习：

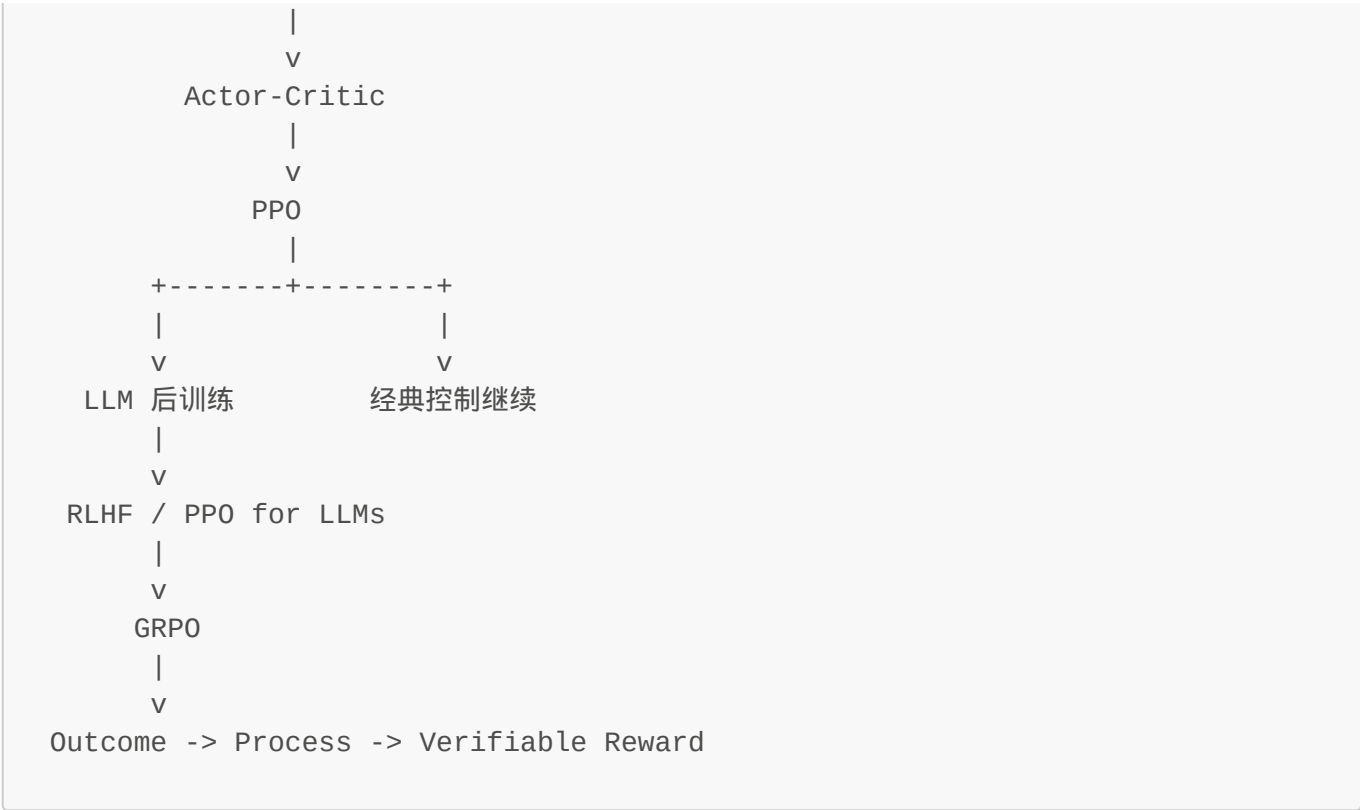
经验 → Agent → 规则

RL 适合规则难以手写的场景 - 机器人走路、下棋、打游戏。

第二章：Policy（策略）

全局导航图

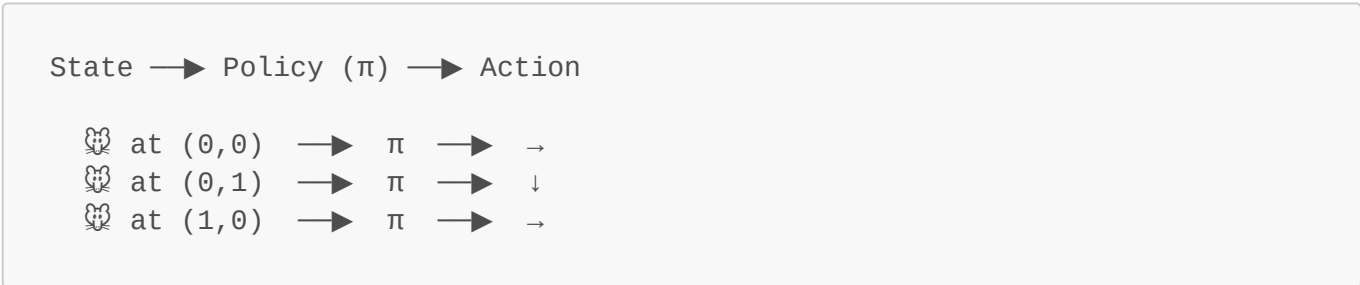




你在这里：主干 -> Policy 基础

什么是 Policy？

Policy 就是 agent 的**决策手册**：给定状态，输出动作。



希腊字母 π (pi) = policy，RL 论文里随处可见。

两种 Policy

DETERMINISTIC (确定性)	STOCHASTIC (随机性)
状态 → 唯一动作	状态 → 动作概率分布
(0,0) → 永远走 →	(0,0) → 70% →
像 GPS 导航	20% ↓
	10% ↑
	像考试乱蒙的学生 😊

- 训练早期 → **Stochastic**：随机探索，啥都试
- 训练成熟 → **Deterministic**：自信果断，直奔目标

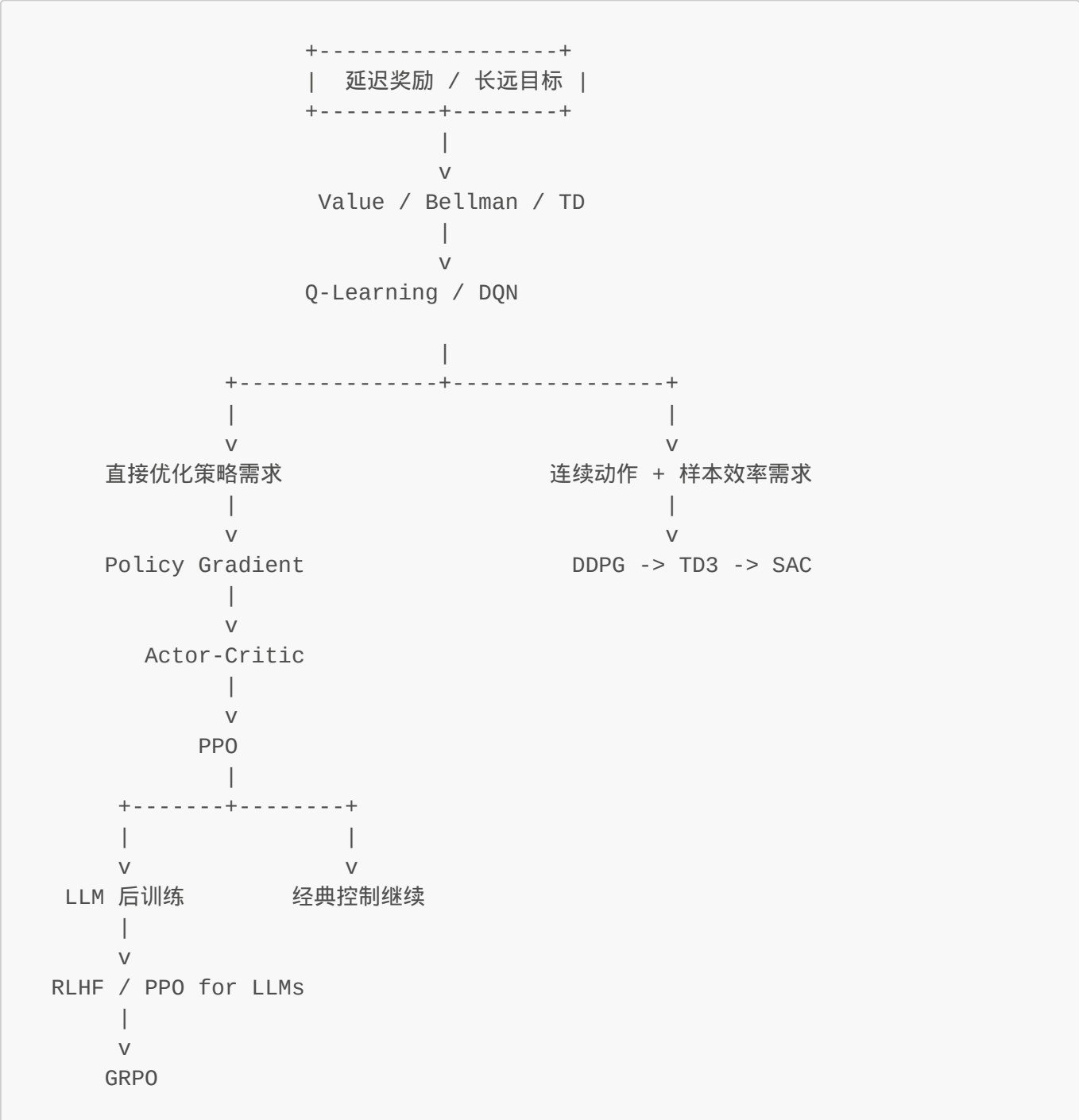
RL 的终极目标（一句话）

找到策略 π ，使得累计 reward 最大化。

其他所有算法，都是在解决"怎么找到这个 π "的问题。

第三章：Value Function 与 Bellman 方程

全局导航图



|
v
Outcome -> Process -> Verifiable Reward

你在这里：主干 -> Value / Bellman

🧠 第一性原理推导：Bellman 是怎么被"逼出来"的？

这一节不从公式出发，而从最原始的问题出发：

长期决策到底该怎么表示，才能真的学出来？

Bellman 不是为了写公式而写公式，而是为了解决这个根本问题。

🔴 Problem（原始困境）：只看即时 Reward，Agent 太短视

假设我在训练一只小老鼠找奶酪：

A → B → C → 🧀 (终点)
0 0 0 +1

规则很简单：只有走到🧀才有 +1，其他地方全是 0。

你马上遇到第一个问题：

如果 agent 现在在 A，它根本没拿到奖励，那我怎么知道 A 是不是一个"好状态"？

因为：

- A 当下 reward = 0
- B 当下 reward = 0
- C 当下 reward = 0
- 只有终点才有 +1

只看眼前 Reward → 前面所有状态看起来都一样差。

但直觉告诉我这不对：

- C 明显比 B 更接近成功
- B 明显比 A 更接近成功

核心问题出现：

即时 reward 只能评价眼前，不能评价长期潜力。

● Limitation（旧想法不够）：为什么"走最直"和"看眼前最短"都不行？

我尝试过几种 naive 的策略：

策略 1：走最直

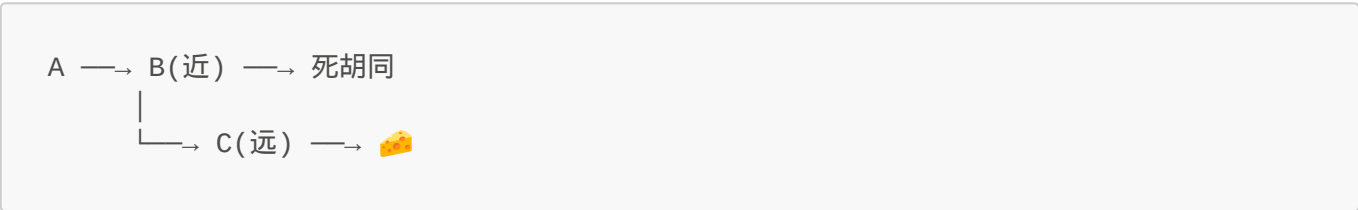
老鼠在 A → 直线冲向终点

问题：可能到死胡同，绕远路更优。

策略 2：看眼前最短

老鼠在 A → 选离终点最近的邻居

问题：局部最优 ≠ 全局最优。比如：



策略 3：简单加总所有 Reward

Value = R1 + R2 + R3 + ...

问题：没考虑时间价值。如果两个路径奖励相同，但一个长一个短，应该选短的。

本质缺陷：

没有统一的框架把分散的决策串起来。每一步都是独立的短视判断。

● New Idea（新工具）：Bellman 的核心思想

我灵光一现：

一个状态值不值钱，不取决于它"现在长什么样"，而取决于它"接下来会把我带到哪里去"。

于是得到核心思想：

当前状态的价值 = 这一步收益 + 从下一步开始的未来价值总和

公式原型诞生：

$$V(s) = R + \gamma \times V(s')$$

为什么要有 γ (折扣因子) ?

因为未来太长了！如果直接加总：

$$\text{Value} = R_1 + R_2 + R_3 + R_4 + \dots \quad (\text{无穷级数, 算不完})$$

我需要一个**压缩机制**。让未来的影响随距离衰减：

- $\gamma = 0.9 \rightarrow$ 下一步打 9 折, 下下一步打 81 折...
- $\gamma = 0.5 \rightarrow$ 下一步打 5 折, 下下一步打 25 折...

这样未来总价值变成收敛的几何级数：

$$\begin{aligned} V(s) &= R_1 + 0.9 \times R_2 + 0.9^2 \times R_3 + 0.9^3 \times R_4 + \dots \\ &= R_1 + \gamma \times (R_2 + \gamma \times R_3 + \dots) \\ &= R_1 + \gamma \times G_{\{t+1\}} \end{aligned}$$

Bellman 思想的真正美感：

把一个很长、很难算的问题，拆成一个局部递推问题。

● Mechanism (它怎么工作)：奖励向前传播

初始状态：

$$\begin{aligned} V(A) &= 0, \quad V(B) = 0, \quad V(C) = 0, \quad V(\text{👛}) = 1 \\ \gamma &= 0.9 \end{aligned}$$

第一轮迭代：

- C 能直接到 👛 $\rightarrow V(C) = 0 + 0.9 \times 1 = 0.9$

第二轮迭代：

- B 能到 C $\rightarrow V(B) = 0 + 0.9 \times 0.9 = 0.81$

第三轮迭代：

- A 能到 B $\rightarrow V(A) = 0 + 0.9 \times 0.81 = 0.729$

形成价值梯度：

$$A(0.729) \longrightarrow B(0.81) \longrightarrow C(0.9) \longrightarrow \text{👛}(1)$$

Agent 即使看不到👉，也能顺着 V 变高的方向前进。

这就是 奖励向前传播：终点的奖，一层层传回起点。

● Role（Bellman 在 RL 中的角色）

层面	Bellman 的作用
定义层面	Value Function 的递推关系： $V(s) = R + \gamma V(s')$
计算层面	把长期问题拆成局部递推： 不用看完整个未来，只看一步 + 下状态
学习层面	TD Error = 新证据 - 旧估计，指导更新方向

● Effect（实际改变）：学完 Bellman，Agent 能做到什么？

1. ✓ 即使当前没 Reward，也知道"往哪走更值钱"
- A·B·C 都是 0 reward，但 $V(A)=0.729 > V(B)=0.81 > V(C)=0.9$
2. ✓ 终点的奖能一层层传回起点，照亮整张地图
- 原来只有👉发光 → 现在 A/B/C 都被照亮了
3. ✓ 把"规划未来"变成"局部计算"
- 以前问："未来几百步到底会发生什么？"（太重）

◦ 现在问："走一步，看下状态的概括就行"（轻量）

△ 但等等——Bellman Equation 只给关系，不给算法！

● New Problem：Equation 是标准，不是方法

我现在知道：

正确答案应该满足： $V(s) = R + \gamma V(s')$

但我手里只有一个当前估计值 $V_{old}(s)$ 。

新问题：怎么一步步把它修正到正确答案附近？

● New Idea（Update 自然出现）：既然不知道真值，就朝更合理的目标挪一点

思路很简单：

1. 我知道"正确 value 应该满足 Bellman 关系"
2. 如果当前估计不满足这个关系 → 往满足关系的方向推一点

数学推导：

当前： $V_{old} = 0.20$
新经验： $R=0, V(s')=0.90, \gamma=0.9$

更合理的目标：
 $target = R + \gamma V(s') = 0 + 0.9 \times 0.90 = 0.81$

我不会一下子从 0.20 跳到 0.81（太激进）：

新估计 ← 旧估计 + $\alpha \times (\text{目标} - \text{旧估计})$
 $V_{new} = V_{old} + \alpha \times (target - V_{old})$

Bellman Update 诞生了！

$V(s) \rightarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$

中括号 $[R + \gamma V(s') - V(s)]$ 这个量，我把它单独命名：

$\delta = R + \gamma V(s') - V(s) \quad \text{\texttt{(TD Error)}}$

含义：新证据和旧估计差多少？

- $\delta > 0$ → 原来估低了，往上调

• $\delta < 0$ → 原来估高了，往下调

● Effect（Update 带来的效果）：方程变成了可学习的方法

能力	Update 之前	Update 之后
知道什么	"正确 value 应该满足 $V(s)=R+\gamma V(s')$ "	✓ 能一步步逼近这个关系
需要真值吗？	× 不需要，只依赖样本	✓ 边交互边修正
奖励传播	× 理论概念	✓ 实际训练过程（每次更新都往前传一点）

🧠 完整推导链：问题驱动式第一性理解

Problem: 即时 Reward 太短视 → Agent 在迷宫里懵逼

↓

Limitation: 走最直 / 看眼前 / 简单加总都不行

↓

New Idea: Value = 当前收益 + 未来价值的折现 (Bellman Equation)

↓

Problem: Equation 只给关系, 不给训练方法

↓

New Idea: 朝 Bellman target 挪一点 (Bellman Update)

↓

Effect: Agent 学会用长远眼光做当下决策

详细演示：Bellman Update 的多轮迭代过程

现在，我们用**具体数值**来完整演示 Bellman Update 如何从"瞎猜"一步步收敛到真值。🔥

Problem Setup（问题设定）

迷宫结构：

A → B → C → 🧡 (终点)

0 0 0 +1

参数设置：

- $\gamma = 0.9$ （折扣因子）
- $\alpha = 0.5$ （学习率，每次挪一半）
- 真值应该是：
 - $V(\text{🧡}) = 1$
 - $V(C) = 0 + 0.9 \times 1 = \mathbf{0.9}$
 - $V(B) = 0 + 0.9 \times 0.9 = \mathbf{0.81}$
 - $V(A) = 0 + 0.9 \times 0.81 = \mathbf{0.729}$

Iteration Process（多轮迭代）

初始状态：全瞎猜

Iteration 0:

$V(A) = 0.5$ (随便猜的)

$V(B) = 0.4$

$V(C) = 0.6$

$V(\text{🧡}) = 1.0$ (这个是已知的)

第 1 轮：从 C 开始更新

经验：C → 🍷, R=1

```
# 当前估计
V(C_old) = 0.6, V(🍷) = 1.0

# Bellman target (理想目标)
target_C = R + γ × V(🍷)
          = 0 + 0.9 × 1.0
          = 0.9

# TD Error (打脸幅度)
td_error_C = target_C - V(C_old)
            = 0.9 - 0.6
            = 0.3 ← 原来估低了！

# Update (朝目标挪一点, α=0.5)
V(C_new) = V(C_old) + α × td_error_C
          = 0.6 + 0.5 × 0.3
          = 0.75
```

```
Iteration 1:
V(A) = 0.5    (没更新, 等 B 先更新)
V(B) = 0.4
V(C) = 0.75   ← 从 0.6 升到 0.75 ✓
V(🍷) = 1.0
```

第2轮：现在更新B

经验：B → C, R=0

```
# 当前估计 (C 已经更新到 0.75)
V(B_old) = 0.4, V(C_new) = 0.75

# Bellman target
target_B = R + γ × V(C_new)
          = 0 + 0.9 × 0.75
          = 0.675

# TD Error
td_error_B = target_B - V(B_old)
            = 0.675 - 0.4
            = 0.275 ← 原来估低了！

# Update (α=0.5)
V(B_new) = V(B_old) + α × td_error_B
          = 0.4 + 0.5 × 0.275
          = 0.5375
```

```
Iteration 2:
V(A) = 0.5
V(B) = 0.5375 ← 从 0.4 升到 0.5375 ✓
V(C) = 0.75
V(🍌) = 1.0
```

第 3 轮：现在更新 A

经验：A → B, R=0

```
# 当前估计 (B 已经更新到 0.5375)
V(A_old) = 0.5, V(B_new) = 0.5375

# Bellman target
target_A = R + γ × V(B_new)
           = 0 + 0.9 × 0.5375
           = 0.48375

# TD Error
td_error_A = target_A - V(A_old)
            = 0.48375 - 0.5
            = -0.01625 ← 这次估高了！往下调

# Update (α=0.5)
V(A_new) = V(A_old) + α × td_error_A
          = 0.5 + 0.5 × (-0.01625)
          = 0.491875
```

```
Iteration 3:
V(A) = 0.491875 ← 从 0.5 降到 0.491875 ✓ (回调)
V(B) = 0.5375
V(C) = 0.75
V(🍌) = 1.0
```

多轮迭代：收敛过程加速表

Iteration	V(A)	V(B)	V(C)	说明
0 (初始)	0.5000	0.4000	0.6000	瞎猜
1	0.5000	0.4000	0.7500	C 更新：发现🍌值 1，C 应该值 0.9
2	0.5000	0.5375	0.7500	B 更新：C=0.75 → B 应该值 0.675

Iteration	V(A)	V(B)	V(C)	说明
3	0.4919	0.5375	0.7500	A 更新：回调
4	0.4919	0.6028	0.7500	B 再更新
5	0.4690	0.6028	0.7500	A 再更新：继续回调
...	...	...	...	震荡收敛中
10	0.7356	0.8201	0.8975	接近真值！✓
100	0.7291	0.8104	0.8999	几乎收敛✓✓
1000	0.7290	0.8100	0.9000	完美收敛！🎯

Key Insights（关键洞察）

1. 奖励向前传播的视觉效果

True Values:

V(🍌)=1
V(C)=0.9
V(B)=0.81
V(A)=0.729

Iteration 0:

V(A)=0.5
V(B)=0.4
V(C)=0.6
(瞎猜)

Iteration 10:

V(A)=0.7356
V(B)=0.8201
V(C)=0.8975
(接近真值！)

奖励像热量一样传播：
🍌(1) → C(0.9) → B(0.81) → A(0.729)

2. 震荡是必然的

- Iteration 3: A 从 0.5 降到 0.49（回调）
- Iteration 5: A 又从 0.4919 变到 0.469（继续回调）
- Iteration 10: 才慢慢稳定下来

为什么震荡？

- $\alpha = 0.5 \rightarrow$ 每次更新只挪一半
- 新证据不断出现 $\rightarrow$ 目标值在变 $\rightarrow$ estimate 跟着波动
- 但只要 $\gamma < 1$ ，震荡幅度会越来越小

3. γ 的作用：衰减因子

如果 $\gamma = 1$ （不看未来折现）：

target = R + 1 × V(s')

误差会**累积放大**，不会收敛！

如果 $\gamma = 0.5$ （只看眼前）：

$$\text{target} = R + 0.5 \times V(s')$$

收敛更快，但对未来不敏感。

$\gamma = 0.9 \rightarrow$ 平衡点：既重视未来，又保证收敛。

Convergence Analysis (收敛分析)

为什么一定会收敛？

1. 每次更新都朝 target 靠近

$$\begin{aligned} V_{\text{new}} &= V_{\text{old}} + \alpha \times (\text{target} - V_{\text{old}}) \\ &= (1-\alpha) \times V_{\text{old}} + \alpha \times \text{target} \end{aligned}$$

$\rightarrow V_{\text{new}}$ 是 V_{old} 和 target 的加权平均

2. $\gamma < 1$ 保证误差衰减

$$\text{第 } n+1 \text{ 轮误差} \approx \gamma \times \text{第 } n \text{ 轮误差}$$

$\rightarrow$ 误差每轮乘以 γ ，越来越小

3. 多次迭代后累积效应显现

- 第一轮：C 更新了
- 第二轮：B 用 C 的新值更新
- 第三轮：A 用 B 的新值更新
- 第 n 轮：整个链条都更新了一遍

收敛速度公式

$$\text{误差衰减率} \approx \gamma^n$$

比如 $\gamma = 0.9$ ：

$n=10$ ：误差 $\times 0.9^{10} = \text{误差} \times 0.35$ （减少到 35%）

$n=100$ ：误差 $\times 0.9^{100} = \text{误差} \times 0.000027$ （几乎为 0）

🔍 TD Error 术语详解：从第一性原理理解“打脸幅度”

TD Error 是 RL 里最核心但也最容易误解的概念。这里的 TD 是 Temporal-Difference 的缩写，所以 TD Error 的全称是 Temporal-Difference Error，中文通常译为“时序差分误差”或“时间差分误差”。这次我们从**为什么需要它**

开始推导。🔥

Problem（原始困境）：我们到底在更新什么？

Bellman Update 的完整公式是：

$$V(s) \rightarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$$

问题：中括号 $[R + \gamma V(s') - V(s)]$ 这个量，到底代表什么？为什么要把它单独拎出来命名？

我尝试过几种 naive 的理解方式：

理解 1：这就是"误差"

$$\text{误差} = \text{目标} - \text{当前估计}$$

问题：哪个是"目标"？ $R + \gamma V(s')$ 就是目标吗？为什么？如果我自己估计的 $V(s')$ 也错了，这个"目标"还是正确的吗？

理解 2：这就是"梯度的方向"

$$\begin{aligned}\text{梯度} &= R + \gamma V(s') - V(s) \\ \text{更新} &= \alpha \times \text{梯度}\end{aligned}$$

问题：这真的是梯度吗？为什么减去 $V(s)$ ？为什么要加 $R + \gamma V(s')$ ？

理解 3：这就是"新证据的强度"

$$\begin{aligned}\text{新证据} &= R + \gamma V(s') - V(s) \\ \text{值越大, 更新越剧烈}\end{aligned}$$

问题：为什么是减号？如果 $V(s)$ 很大（我原来估计很高），那 $R + \gamma V(s')$ 再大也抵不过它？

本质缺陷：

我只是在背公式，但没有理解"为什么是这个形式"。
每个符号背后，都对应着一个具体的问题。

Limitation（旧想法不够）：为什么其他形式的更新不行？

我尝试过几种不同的更新方式，看看会发生什么：

尝试 1：直接跳到目标

$$V(s) \leftarrow R + \gamma V(s')$$

问题：

- 如果 $V(s')$ 也是错的 $\rightarrow$ 新估计还是错的
- 会震荡，甚至发散（因为目标本身在变）
- 没有渐进逼近的稳定性

尝试 2：用旧估计帮自己更新（Bootstrapping）

$$V(s) \leftarrow V(s) + \alpha [V(s') - V(s)]$$

问题：

- 只依赖 $V(s')$ ，忽略了眼前的 R
- 如果 $V(s')$ 和 $V(s)$ 都错了 $\rightarrow$ 越改越偏
- 缺少"新证据" (R) 的指引

尝试 3：只看 R ，不看未来

$$V(s) \leftarrow V(s) + \alpha [R - V(s)]$$

问题：

- 完全忽略了长远价值
- 变成短视行为，无法学习长期决策
- Bellman Equation 的核心意义被抛弃

本质缺陷：

我只是在调整系数，但没有理解"这个形式是怎么被逼出来的"。
真正的理解应该能独立重现实验室中发明的过程。

New Idea（新工具）：TD Error 是"打脸幅度"

核心洞察：

1. 我知道**正确答案**应该满足： $V(s) = R + \gamma V(s')$
2. 但我手里只有**当前估计**： $V_{old}(s)$
3. **问题**：新证据到底打脸我多少？

答案：用"目标 - 旧估计"来衡量。

$$\begin{aligned}\delta &= \text{target} - \text{old estimate} \\ &= (R + \gamma V(s')) - V(s)\end{aligned}$$

为什么这个形式合理？

① $\text{target} = R + \gamma V(s')$

这是 Bellman Equation 算的理想目标：

- R = 眼前的即时回报（新证据）
- $\gamma V(s')$ = 从下一步开始的未来价值（压缩后的长远信息）

为什么相加？

因为长期价值 = 当前收益 + 未来价值。这是 Bellman 的核心思想，不是凑出来的！

② $\text{target} - V(s)$

这是"新证据和旧估计的差距"：

- 如果 > 0 → 原来估低了（打脸方向：往上调）
- 如果 < 0 → 原来估高了（打脸方向：往下调）
- 如果 ≈ 0 → 估计差不多了（不再大幅调整）

为什么是减法？

因为更新的方向应该**朝向目标**。如果 $\text{target} > V(s)$ ，就应该增加；如果 $\text{target} < V(s)$ ，就应该减少。

Mechanism（它怎么工作）：TD Error 的深层含义

1. TD Error 是"信息增益"

每次采样，我们都得到新证据 $R + s'$ 。这个证据告诉我们：

旧估计 $V(s)$ 离目标有多远？

TD Error	信息含义	更新方向
$\delta \gg 0$	"原来估得太低了！"	大幅上调
$\delta \approx 0$	"估计得差不多"	小幅调整
$\delta \ll 0$	"原来估得太高了！"	大幅下调

2. TD Error 是"学习信号"

在训练过程中，TD Error 告诉我们**哪里需要改进**。

3. TD Error 是"误差传播"的载体

在 Bellman Update 中，误差会沿着状态链向前传播：

第一轮：C → 🧀, $\delta_C = \text{target}_C - V(C)$

第二轮：B → C, $\delta_B = \text{target}_B - V(B) = (R + \gamma V(C_{\text{new}})) - V(B)$

第三轮：A → B, $\delta_A = \text{target}_A - V(A) = (R + \gamma V(B_{\text{new}})) - V(A)$

每一轮更新，TD Error 都在告诉 Agent："这里估计错了，往这个方向改！"

Role（TD Error 在 RL 中的角色）

层面	TD Error 的作用
定义层面	衡量"新证据和旧估计的差距"
计算层面	是 Bellman Update 的核心驱动项
学习层面	告诉 Agent "哪里需要改进，往哪个方向改"

Effect（实际改变）：有 TD Error vs 没有 TD Error

有 TD Error：

$$V(s) \rightarrow V(s) + \alpha [R + \gamma V(s') - V(s)]$$

效果：

- 1. ✓ **渐进逼近**：每次只挪一点，不会震荡发散
- 2. ✓ **误差传播**：新证据能层层传递到前面状态
- 3. ✓ **自适应调整**： δ 越大改得越猛， $\delta \approx 0$ 时几乎不动

没有 TD Error（直接跳到目标）：

$$V(s) \rightarrow R + \gamma V(s')$$

问题：

- 1. ✗ **震荡发散**：因为 target 本身在变，估计会追着 target 跑
- 2. ✗ **误差积累**：如果 $V(s')$ 错了，新估计也会错
- 3. ✗ **无法收敛**：目标永远在变，永远追不上

 一句话总结（Bellman Update 完整版）

因为即时 Reward 太短视，所以我要估计长期价值；
因为长期价值太难直接算，所以我要把它拆成"当前收益 + 未来价值"；

因为我又拿不到真值，所以只能根据新经验不断朝这个关系逼近。

这三句话，把下面几件事全串起来了：

- Bellman Equation → 为什么被想到（解决长期决策怎么表示）
- Bellman Update → 为什么被想到（解决长期决策怎么学习）
- TD Error → 为什么自然出现（衡量"打脸幅度"）

****Bellman 不是为了写公式而写公式，而是为了解决"长期决策怎么表示、怎么学习"这个根本问题。****

一个更直观的版本：Bellman 套到你的真实选择上

现在，我们用第一视角来走一遍 Bellman 的**完整决策流程**。假设我就是你（David），站在一个真实的十字路口——多选项选一，只能挑一个。

场景设定

凌晨 1:30 的抉择：

- **A** = 继续写 RL 代码（深度学习 Chapter 4）
- **B** = 睡回笼觉（恢复精力）
- **C** = 打 World of Warships（通宵游戏）

三个选项，只能选一个。直觉告诉我：学 RL 重要，但困得厉害；打游戏爽，但第二天要崩盘；睡觉舒服，但进度落后。**怎么办？**

Bellman 的决策公式（人话版）

对于每个可选动作 a_i ：

$$Q(s, a_i) = R(s, a_i) + \gamma * V^{\pi}(s'_i)$$

选： $a^* = \operatorname{argmax}_i Q(s, a_i)$

拆开看就是三步：

1. **算眼前回报 R** → 这一步直接能拿多少分？
2. **估未来价值 V** → 进了下一步 s' ，长期能值多少？
3. **设 γ （你的"远视参数"）** → 我有多看重未来？
4. **$Q = R + \gamma \times V$ ，选 Q 最高的**

逐个算一下这三个选项

假设：

- **R** 是即时回报（精力消耗、爽感、金钱）
- **$V(s')$** 是进入新状态后的长期预期价值
- **$\gamma = 0.85$** → 我承认眼前不无价值，但更信长远复利

选项 A：继续写 RL 代码

$$R = -25 \text{ 分 (困倦, 精力消耗)}$$
$$V(s_A) = +180 \text{ 分 (RL 学完有项目 + 经验复利, 长期收益高)}$$
$$Q_A = -25 + 0.85 \times 180 \approx 137 \text{ 分}$$

直觉解读：
眼前挺亏，但长远赚翻了。就像你为了跳槽拼命学新技能——现在没时间玩、没钱，但三年后溢价可观。

选项 B：睡回笼觉

$$R = +50 \text{ 分 (休息恢复精力)}$$
$$V(s_B) = +40 \text{ 分 (精神饱满继续写, 但进度落后)}$$
$$Q_B = 50 + 0.85 \times 40 \approx 84 \text{ 分}$$

直觉解读：
舒服是舒服，但长远看只是"缓一缓"，没改变根本。休息好当然重要，但如果长期战略没变，价值有限。

选项 C：打 World of Warships 通宵

$$R = +60 \text{ 分 (游戏爽感爆表)}$$
$$V(s_C) = -150 \text{ 分 (白天效率崩盘, 健康折旧)}$$
$$Q_C = 60 + 0.85 \times (-150) \approx -67 \text{ 分}$$

直觉解读：
眼前爽翻天，但长远代价巨大。通宵后的第二天：编程效率跌穿地板、代码写错率飙升、身体亚健康累积。
Bellman 告诉你——别被眼前的 R 骗了。

决策结果

$$Q_A = 137 \text{ 分} \leftarrow \text{最高} \checkmark$$
$$Q_B = 84 \text{ 分}$$
$$Q_C = -67 \text{ 分}$$

→ 选 A：继续写 RL 代码

Bellman 没有拍脑袋，它用数学告诉你：
哪怕眼前 R 是负的，只要 $\gamma \cdot V(s')$ 足够高，长远最优就是当下最优。

另一个栗子：RL 学习 vs 投工作（职业抉择版）

同样的逻辑，套到更大的选择上——比如"继续深挖 RL 算法"还是"投一份 Web 开发工作"？

选项	R（眼前回报）	$\gamma=0.85 \times V$ （未来价值）	$Q = R + \gamma \times V$
深度学习 RL	-20 分/小时（没时间玩、没钱）	$0.85 \times 30000/\text{年} \approx +12750$ （跳槽溢价+经验复利）	$Q \approx \text{高}$ ✓
投全职工 作	+30 分/月（稳定收入，生活压力减轻）	$0.85 \times 10000/\text{年} \approx +8500$ （稳定但天花板低）	$Q \approx \text{中低}$

Bellman 拆解：

- **R 不只是钱**：还包括精力消耗、情绪影响、时间成本
- **V 基于历史验证**：如果昨天有人告诉你"RL 这半年不火"，那 $V(s)$ 就得下调
- **γ 是你的个性开关**：短期主义者 $\gamma=0.6$ ，长远主义者 $\gamma=0.9$

Bellman 的本质（一句话）

Bellman 不是数学题，是决策操作系统。

它把模糊的"我想长远好"变成清晰的每一步计算：

1. **眼前这步值多少分？** → R
2. **进了下一步能到哪儿？** → $V(s')$
3. **我有多看重未来？** → γ
4. **$Q = R + \gamma \times V$ ，选 Q 最高的**

核心思想（回到 RL）

现在回到 RL 本身：

- **Reward** 只能看眼前，但决策必须看长远
- **Value Function** 就是为了解决这个"长期价值怎么表示"的问题
- **Bellman** 把长期价值拆成：**当前收益 + 未来价值的折现**

$$V^{\pi}(s) = R + \gamma \times V^{\pi}(s')$$

它解决的是三个问题：

1. **奖励向前传播** → 终点的奖，一层层传回起点
2. **即使当前没 Reward，也知道自己朝哪走** → V 梯度指引方向
3. **把"规划未来"变成"局部计算"** → 不用看完整个未来，只看一步 + 下状态的概括

Bellman Equation vs Bellman Update（别混了！）

	Bellman Equation	Bellman Update
是什么	定义"正确的 value 应该满足什么关系"	定义"怎么一步步逼近这个关系"
角色	理想目标 / 标准答案	训练方法 / 更新规则
公式	$V(s) = r + \gamma V(s')$	$V(s) \leftarrow V(s) + \alpha \times (\text{target} - V(s))$
一句话	"对"是什么	"怎么往'对'走"

关键洞察（记住这三点）

1. $V(s) \neq R$, Value 是未来的总和，不是眼前的数字

2. γ 是你的价值观： $\gamma=0.9$ 的人比 $\gamma=0.3$ 的人更敢为长期牺牲眼前

3. Bellman 不是算题，是让 Agent 学会"用长远眼光做当下决策"

回到第一性原理（完整推导链）

现在，我们把之前的"问题驱动式推导"和刚才的"生活实例"融合：

❶ Problem（原始困境）

即时 Reward 太短视：

Agent 在迷宫里走，只有终点🏁才有 +1，前面全是 0。只看眼前 Reward，A/B/C/D 四个起点看起来都一样差——那 Agent 怎么知道哪个更好？

❷ Limitation（旧想法不够）

- "走最直" → 可能到死胡同

• "看眼前最短" → 可能绕远路更优

• "简单加总所有 Reward" → 没考虑时间价值，计算爆炸

本质缺陷：没有统一的框架把分散的决策串起来。每一步都是独立的短视判断。

❸ New Idea（新工具：Bellman）

核心思想：

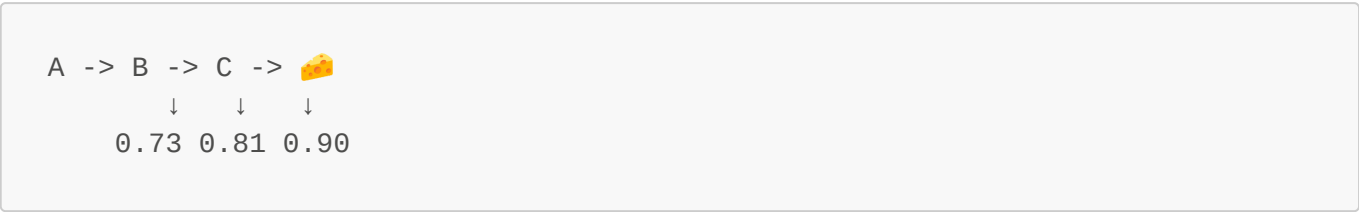
当前状态的价值 = 眼前这一步的收益 + 从下一步开始的未来收益的折现总和

公式原型：

$$V(s) = R + \gamma \times V(s')$$

④ Mechanism（它怎么工作）

奖励向前传播（像热量一样）：



- C 靠近终点 → $V(C) = 0.9$
 - B 能走到 C → $V(B) = 0.81$
 - A 能走到 B → $V(A) = 0.729$
- Agent 即使看不到 🧡，也能顺着 V 变高的方向走。

⑤ Role（Bellman 在 RL 中的角色）

Bellman 的作用	
定义层面	Value Function 的递推关系
计算层面	把长期问题拆成局部递推
学习层面	TD Error = 新证据 - 旧估计，指导更新方向

⑥ Effect（实际改变）

学完 Bellman，Agent 能做到：

1. 即使当前没 Reward，也知道"往哪走更值钱"
2. 终点的奖能一层层传回起点，照亮整张地图
3. 把"规划未来"变成"看一步 + 下状态概括"

回到你的真实世界（作为 David 的总结）

现在，假设你就是那个 Agent：

- **迷宫** = 你的生活选择
- 🧡 = 你想要的东西（高薪、技能、健康）
- **Reward** = 眼前的爽/痛（工资、加班、游戏时间）
- **Value** = 长远价值（复利、复投、复利）

Bellman 教你：

别只看 R（眼前这杯奶茶），要算 $\gamma \times V$ （三年后身材+健康）

你的决策公式：

$$\begin{aligned} Q(\text{学 RL}) &= -25(\text{今天累}) + 0.85 \times 180(\text{长期收益}) \approx 137 \\ Q(\text{打游戏}) &= +60(\text{爽感}) + 0.85 \times (-150)(\text{健康折旧}) \approx -67 \end{aligned}$$

选 Q 最高的。

完整的"问题驱动式第一性理解" (最终版本)

这一整段序言，其实就是在做一件事：

不从公式出发，而从最原始的问题出发。

1. **Reward 太短视** → 需要 Value 来衡量未来
2. **未来太长没法硬算** → 拆成"当前收益 + 下状态价值"
3. **又拿不到真值** → 只能用 Update 一步步逼近
4. **Equation 是标准，Update 是方法**

如果你愿意，可以把这段叫做：

"RL 里的问题驱动式第一性理解"

它不是哲学证明，而是帮你建立一种感觉：

每个公式背后，都对应一个现实问题。

Bellman 不是为了写公式而写公式，而是为了解决"长期决策怎么表示、怎么学习"这个根本问题。

1. 面对的问题：只看即时 Reward，Agent 太短视

假设我在训练一只小老鼠找奶酪。

A -> B -> C -> 

规则很简单：

- 走到奶酪才有 +1
- 其他地方 reward 都是 0

这时候我马上遇到第一个问题：

如果 agent 现在在 A，它根本没拿到奖励，那我怎么知道 A 是不是一个"好状态"？

因为：

- A 当下 reward = 0
- B 当下 reward = 0
- C 当下 reward = 0
- 只有终点才有 +1

如果我只看"这一刻拿了多少 reward"，那前面这些状态看起来全都一样差。

但直觉告诉我这不对：

- C 明显比 B 更接近成功
- B 明显比 A 更接近成功

所以第一个核心问题出现了：

即时 reward 只能评价眼前，不能评价长期潜力。

2. 出发点：我要一个能代表"未来希望"的量

既然 reward 太短视，那我自然会想发明一个新东西：

不是看这一步赚了多少，而是看从这里出发，未来总共大概能赚多少。

这就是 **Value Function** 的出发点。

它不是凭空发明出来的，而是为了解决一个很具体的问题：

- reward 只能看当前
- 决策却必须看长远
- 所以需要有一个"长期价值"的量

一句话：

Value 的出发点，就是给"未来有没有希望"打分。

3. 如何解决第一层问题：把 Value 和未来绑定起来

好，现在我有了 value 这个想法，但新的问题马上来了：

这个 value 到底怎么算？

比如：

- $V(C)$ 为什么比 $V(B)$ 大？
- $V(B)$ 为什么比 $V(A)$ 大？
- 大多少才合理？

继续往下想，我会发现一个关键事实：

一个状态值不值钱，不取决于它"现在长什么样"，而取决于它"接下来会把我带到哪里去"。

也就是说：

- A 的价值，取决于走到 B 后会怎样
- B 的价值，取决于走到 C 后会怎样
- C 的价值，取决于下一步是否能到奶酪

于是我会得到一个非常核心的想法：

当前状态的价值，应该由"当前这一步收益 + 下一状态的价值"共同决定。

这就是 Bellman 思想最原始的胚芽。

4. 面对的第二个问题：未来太长，没法硬算

最开始我可能会说：

Value = 所有未来 reward 全部加起来

也就是 return：

$$G_t = R_{\{t+1\}} + \gamma R_{\{t+2\}} + \gamma^2 R_{\{t+3\}} + \cdots$$

这个定义没错，但马上又遇到问题：

未来太长了。直接展开虽然正确，但不方便计算，也不方便学习。

所以新的麻烦是：

- 原问题太长
- 太重
- 太难直接处理

5. 出发点：把长远问题压缩成局部递推

这时我会想：

- 有没有办法把"整个未来"写短一点？
- 有没有办法把它拆成"下一步 + 剩下的未来"？

然后就会发现：

$$G_t = R_{\{t+1\}} + \gamma G_{\{t+1\}}$$

这一下思路就通了。

因为它告诉我：

我不需要每次直接看完整个未来。我只要看当前 reward，再加上下一时刻的总价值。

这就是 Bellman 方程背后的真正美感：

把一个很长、很难算的问题，拆成一个局部递推问题。

所以 Bellman 的出发点，不是为了"数学上显得优雅"，而是因为：

- 未来太长
- 无法暴力硬算
- 必须找到一种递推表达

6. 解决后的效果：奖励终于能向前传播

Bellman 真正解决的是：

长期价值怎么定义，以及怎么从未来一层层传回现在。

它带来三个直接效果。

效果 1：奖励可以向前传播

本来只有终点 🏠 有 +1。

但有了 Bellman 递推之后：

- C 会因为靠近终点而变得有价值
- B 会因为能走到 C 而变得有价值
- A 会因为能走到 B 而变得有价值

也就是说：

终点的奖励不再只停在终点，而是可以一层层往前传。

效果 2：Agent 即使当前没奖励，也知道自己在接近目标

如果没有 Bellman：

- 前面几个状态 reward 都是 0
- agent 看不出区别

如果有 Bellman：

- 前面的状态会逐渐形成 value 梯度
- 越接近目标，value 越高

于是 agent 不需要每一步都看到奖励，也能知道自己是不是在"朝对的方向走"。

效果 3：把"规划未来"变成"局部计算"

如果没有 Bellman，你每次都像是在问：

从这里开始，未来几百步到底会发生什么？

这太重了。

Bellman 把这个问题变成：

先看一步，再把剩下的未来交给下一个状态去概括。

所以它本质上是在做一种 **问题压缩**。

7. 面对的第三个问题：Bellman Equation 只给关系，不给算法

到这里，我已经有了一个很漂亮的关系：

$$V(s) = r + \gamma V(s')$$

但我很快会发现：

这只是一个“正确答案应该满足的关系”，不是求答案的方法。

也就是说：

- Bellman equation 告诉我什么样的 value 才合理
- 但它没有直接把那个 value 塞到我手里

所以新的问题是：

如果我手里只有一个当前估计值，怎么一步步把它修正到正确答案附近？

8. 出发点：既然不知道真值，就朝更合理的目标挪一点

既然我已经知道“正确 value 应该满足 Bellman 关系”，那最自然的想法就是：

如果我当前的估计不满足这个关系，我就把它往满足关系的方向推一点。

这就是 update 的原始思路。

比如我当前认为：

- $V(B) = 0.20$

但这次经验告诉我：

- $r = 0$
- 下一状态 C 的价值大约是 0.90

如果 $\gamma = 0.9$ ，那一个更合理的目标就是：

$$0 + 0.9 \times 0.90 = 0.81$$

我不会一下子把 $V(B)$ 从 0.20 改成 0.81 ，因为这样太激进。

我更自然的做法是：

先朝这个目标走一点。

于是就得到：

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

这就是 Bellman update。

9. 解决后的效果：方程变成了可学习的方法

Bellman update 的作用是：

- 把"正确关系"变成"可执行训练步骤"
- 让 agent 不需要知道真值，也能边交互边修正
- 让奖励信息通过一次次样本逐步传播回前面的状态

一句话：

Equation 是标准，Update 是逼近标准的方法。

10. 为什么还会自然得到 TD Error？

当我在做 Bellman update 时，我很自然会注意到一个量：

$$r + \gamma V(s') - V(s)$$

这个量其实就是：

新目标 - 旧估计

于是我会意识到：

- 如果它是正的，说明我原来估低了
- 如果它是负的，说明我原来估高了
- 如果它接近 0，说明我估得差不多了

这个量太有用了，所以它值得单独起个名字：

$$\delta = r + \gamma V(s') - V(s)$$

也就是 TD error。

它的作用是：

告诉我这次新经验对我原来的估计打脸了多少。

11. 这一整段，其实是不是第一性原理？

可以说：**是"接近第一性原理"的理解方式**，但别把这个词用得太过玄。

更准确地讲，这里做的是：

- 不从现成公式出发
- 而是从最原始的问题出发
- 一步步解释每个概念为什么必须出现

所以这段序言的目的不是正式证明，而是帮你建立这样一种感觉：

每个公式背后，都对应一个现实问题。Bellman 不是为了写公式而写公式，而是为了解决"长期决策怎么表示、怎么学习"这个根本问题。

如果你愿意，把它叫做：

RL 里的问题驱动式第一性理解

也可以。

12. 最后一段：把整条线压成一句话

如果让我自己重新发明它，我最后会把它总结成：

因为即时 reward 太短视，所以我要估计长期价值；因为长期价值太难直接算，所以我要把它拆成"当前收益 + 未来价值"；因为我又拿不到真值，所以只能根据新经验不断朝这个关系逼近。

这三句话，基本就把下面几件事串起来了：

- Bellman 方程为什么会被想到
 - Bellman update 为什么会被想到
 - TD error 为什么会自然出现
 - 它们分别解决什么问题、起什么作用、产生什么效果
-

RL 常用术语（跟着这一章一起认识）

这一章里会反复出现一些 RL 术语。第一次看见不用硬背，先抓直觉。

Agent

就是**做决定的那个东西**。

- 在游戏里，它可能是玩家
- 在机器人任务里，它可能是机器人
- 在 **CartPole** 里，它就是控制小车左右移动的程序

一句话：

Agent = 负责行动和学习的主体。

Environment

就是 **Agent 外面的整个世界**。

它负责：

- 告诉你现在在哪个状态
- 接收你的动作
- 返回新的状态和奖励

一句话：

Environment = 你行动的世界，负责给反馈。

State

就是**当前情况的描述**。

比如在 **CartPole** 里，state 通常包含：

- 小车位置
- 小车速度
- 杆子的角度
- 杆子的角速度

一句话：

State = 你此刻所处的局面。

Action

就是 **Agent 当前做的动作**。

比如：

- 左移 / 右移
- 上 / 下 / 左 / 右
- 加速 / 刹车

一句话：

Action = 你现在准备做什么。

Reward

就是环境对这一步动作的**即时反馈**。

- 做得好 -> reward 可能更高
- 做得差 -> reward 可能更低甚至为负

但要注意：

Reward 不一定等于真正目标，它只是环境给你的信号。

一句话：

Reward = 这一步的即时分数。

Episode

就是从开始到结束的一整局。

比如：

- 玩一局游戏
- 小车从 reset 到倒下
- 小老鼠从起点走到终点或失败

一句话：

Episode = 一整次完整尝试。

Return

就是从当前时刻开始，未来奖励的总和。

$$G_t = R_{\{t+1\}} + \gamma R_{\{t+2\}} + \gamma^2 R_{\{t+3\}} + \cdots$$

一句话：

Return = 从现在开始，未来总共能拿多少。

Policy

就是 **Agent** 选动作的规则。

例如：

- 看到杆子往左倒，就往左推
- 在这个状态下，80% 向右，20% 向左

策略记作 π 。

一句话：

Policy = 看到什么情况，就怎么行动。

Value Function

就是对未来总回报的估计。

它回答的问题不是：

- "这一步得了几分？"

而是：

- "从这里开始，未来整体值不值钱？"

一句话：

Value Function = 对未来好坏的打分地图。

$V(s)$

状态价值函数。

它问的是：

站在这个状态上，未来大概值多少？

一句话：

$V(s)$ = 这个位置整体有多好。

$Q(s, a)$

动作价值函数。

它问的是：

在这个状态下，如果我现在做这个动作，未来大概值多少？

一句话：

$Q(s, a)$ = 这个位置做这个动作有多好。

Gamma γ

折扣因子，表示你有多看重未来。

- 大 -> 目光长远
- 小 -> 更看眼前

一句话：

γ = 未来奖励打几折。

Bellman Equation

描述 value 应满足的**递推关系**。

核心思想：

长期价值 = 当前收益 + 未来价值

一句话：

Bellman Equation = 长远问题的递推写法。

Bellman Update

不是定义，而是**学习方法**。

它做的事是：

根据一次新经验，把当前估计往更合理的目标推一点。

一句话：

Bellman Update = 用经验一步步修正 value。

TD Error

就是：

```
\delta = \text{ext}\{\text{target}\} - \text{ext}\{\text{old estimate}\}
```

它表示：

这次新证据和我原来的看法差多少。

一句话：

TD Error = 本次更新里的"打脸幅度"。

Exploration / Exploitation

这是 RL 里特别常见的一对概念。

- **Exploration (探索)**：试试新动作，看看有没有更好的路
- **Exploitation (利用)**：用当前已知最好的动作去拿分

一句话：

探索是"去试"，利用是"去赚"。

Epsilon-Greedy

Q-learning 常用的一种选动作方法。

- 以概率 ϵ 随机探索
- 以概率 $1-\epsilon$ 选择当前 Q 值最大的动作

一句话：

Epsilon-Greedy = 大多数时候选最优，少数时候随机试。

Q-Table

如果状态和动作不太多，就可以直接用一张表记录所有 $Q(s, a)$ 。

一句话：

Q-Table = 把"每个状态-动作值多少"记成一张表。

Optimal / Optimality

表示"最优的"。

- $V^*(s)$ ：状态 s 在最优策略下的价值
- $Q^*(s, a)$ ：动作 (s, a) 在最优策略下的价值

一句话：

带 $*$ 的 value，表示理论上的最好水平。

Model-Free

表示 agent 不需要提前知道环境的完整规则。

比如 Q-learning 就是典型的 model-free 方法：

- 不需要先知道状态转移概率
- 直接靠交互和样本学习

一句话：

Model-Free = 不先背世界规则，边试边学。

Bootstrapping

这是 RL 里一个很经典、也很容易一开始看懵的词。

它表示：

我用"我自己当前的估计", 去帮助更新另一个估计。

比如：

$$V(s) \leftarrow r + \gamma V(s')$$

这里就用了下一状态的估计值 $V(s')$ 来更新当前状态。

一句话：

Bootstrapping = 拿旧估计帮新估计。

这些词不用一次全背住。

更好的记法是：

- Agent / Environment / State / Action / Reward -> 讲的是 RL 的基本舞台
- Return / Policy / Value / Q / gamma -> 讲的是 RL 怎么衡量"未来值不值"
- Bellman / Update / TD Error / Bootstrapping -> 讲的是 RL 怎么把这个价值一步步学出来
- Exploration / Exploitation / Epsilon-Greedy -> 讲的是 RL 学习时怎么选动作

1. 为什么需要 Value Function ?

先看一个问题：只靠即时 reward, Agent 能学会长期决策吗？

只看眼前 Reward:

0	0	0
0	0	0
0	0	

VS

看长远 Value:

0.73	0.81	0.90
0.50	■	1.00
0.30	0.50	

只看眼前 -> 全是0, 懵逼

看长远 -> 知道往哪走 ✓

问题在于：

- 很多环境里的 reward 是稀疏的
- 没到终点之前, agent 看到的 reward 可能一直都是 0
- 如果只盯着眼前 reward, agent 不知道自己是不是在接近目标

所以我们需要一个更强的概念：

Value Function = 从现在开始这个位置出发, 未来总共大概能拿到多少 reward。

一句话区分：

- **Reward**: 这一步立刻拿到了多少
- **Value**: 从这里开始，未来整体值多少

2. Return 与折扣因子 gamma

Value 之所以有意义，是因为 RL 不只关心当前一步，而是关心 **未来总回报**。

这个未来总回报叫做 **Return**:

$$G_t = R_{\{t+1\}} + \gamma R_{\{t+2\}} + \gamma^2 R_{\{t+3\}} + \dots$$

时间线：

现在		+1步		+2步		+3步
r1	+	r2	+	r3	+	r4
折扣后变成：						
r1	+	gamma*r2	+	gamma^2*r3	+	gamma^3*r4

这里的 **gamma** 是 **折扣因子**，取值在 **0 ~ 1** 之间。

- **gamma = 0.9** -> 很在乎未来，目光长远
- **gamma = 0.1** -> 更在乎眼前，未来影响很小

为什么要打折？

因为通常我们认为：

- 越远的未来越不确定
- 越晚拿到的奖励，价值应该稍微低一点

举个例子：

路径：(0,0) -> (0,1) -> (0,2) -> 🧀
 奖励： 0 0 0 +1
 gamma = 0.9

那么：

$$V(0,0) = 0.9^3 \times 1 = 0.729$$

如果你绕远路，多走一步才吃到奶酪：

$$V(\text{弯路}) = 0.9^4 \times 1 = 0.656$$

所以 agent 会自然偏向更短的路。

3. 两类 Value：V(s) 和 Q(s, a)

Value Function 常见有两种。

3.1 状态价值函数 $V^{\pi}(s)$

$$V^{\pi}(s)$$

意思是：

在策略 π 下，从状态 s 出发，未来期望总回报是多少。

重点：它是"这个状态整体值多少"。

3.2 动作价值函数 $Q^{\pi}(s, a)$

$$Q^{\pi}(s, a)$$

意思是：

在策略 π 下，在状态 s 先做动作 a ，然后继续按 π 行动时，未来期望总回报是多少。

重点：它不仅看状态，还看"在这个状态下选哪个动作"。

3.3 二者的区别

$V(s)$	$Q(s, a)$
"这个状态值多少？"	"这个状态下，这个动作值多少？"
只看状态	看状态 + 动作
$V(\text{鼠在}(0,0)) = 0.73$	$Q(\text{鼠在}(0,0), \rightarrow) = 0.73$
	$Q(\text{鼠在}(0,0), \downarrow) = 0.50$
	$Q(\text{鼠在}(0,0), \leftarrow) = 0.10$

所以：

- $V(s)$ 更像是"这个位置好不好"
- $Q(s, a)$ 更像是"在这个位置做这个动作好不好"

Q 更适合直接做决策，因为它可以直接比较动作。

3.4 π 是什么意思？

π 就是 **Policy (策略)**。

所以：

$$V^{\pi}(s)$$

不是说"状态 s 天生有一个固定价值"，而是说：

如果你以后都按策略 π 行动，那么状态 s 值多少？

同一个状态，在不同策略下，value 可能完全不同。

同一个状态 s

聪明策略 π_1 $\rightarrow$ 快速接近目标 $\rightarrow V^{\pi_1}(s)$ 较高
 笨策略 π_2 $\rightarrow$ 到处乱走 $\rightarrow V^{\pi_2}(s)$ 较低

同理：

- $V^{\pi}(s) / Q^{\pi}(s, a) \rightarrow$ 某个具体策略下的价值
 - $V^*(s) / Q^*(s, a) \rightarrow$ 最优策略下的价值
-

4. Bellman 的核心思想

Bellman 方程之所以重要，是因为它抓住了 RL 里一个最核心的递推关系：

长期价值 = 当前收益 + 剩余未来价值

先从 return 的定义出发：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

把后面的部分提出来：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

这一步非常关键。

它告诉我们：

- 一整个很长的未来
- 可以拆成"眼前一步"
- 再加上"下一步开始的未来"

这就是 **Bellman 思想** 的来源。

一句话总结：

Bellman 的本质，就是把一个长远问题拆成一个递推问题。

5. Bellman Expectation Equation

如果我们已经固定了一套策略 π ，那么 value 应该满足：

$$V^\pi(s) = \mathbb{E}_\pi [R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$$

这叫做 **Bellman expectation equation**。

它表达的是：

在策略 π 下，一个状态的价值 = 这一步的即时 reward + 下一状态价值的折扣期望。

如果写成动作价值函数的形式：

$$Q^\pi(s, a) = \mathbb{E}_\pi [R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a]$$

这一类 equation 的关键词是：

- **expectation**
- **给定 policy**
- **按当前策略继续往后走**

所以它回答的问题是：

如果以后继续按这套策略行动，那么现在值多少？

6. Bellman Optimality Equation

如果我们不想评估"某个固定策略有多好"，而是想知道"理论上最优能有多好"，就会得到 **Bellman optimality equation**。

对于最优状态价值：

$$V^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^*(s')]$$

对于最优动作价值：

$$Q^*(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} Q^*(s', a')]$$

它的核心变化只有一个：

- expectation：未来继续按当前 policy 走
- optimality：未来每一步都取最优动作

一句话理解：

Bellman expectation 是"这套策略有多值"，Bellman optimality 是"最优情况下能多值"。

而 Q-Learning 用的，正是这个 optimality 思想。

7. Bellman Equation vs Bellman Update

这两个名字很像，但不是一回事。

7.1 Bellman Equation

Bellman equation 是一个 **数学关系**，描述"正确的 value 应该满足什么条件"。

例如：

$$V(s) = r + \gamma V(s')$$

它更像是：

- 理想目标
- 正确答案应该满足的关系

7.2 Bellman Update

但训练时，我们通常并不知道真正的 value。所以实际做法不是"直接写出答案"，而是一点点逼近它：

$$V(s) \rightarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

含义是：

- 旧估计： $V(s)$

- 新目标： $r + \gamma * V(s')$
- 用学习率 α 朝目标走一步

也就是：

$$\text{新值} = \text{旧值} + \text{学习率} * (\text{目标值} - \text{旧值})$$

所以：

- **Bellman equation** = 定义"对"是什么
- **Bellman update** = 定义"怎么往对的方向逼近"

8. TD Error：更新时到底在修什么？

在 Bellman update 里，中括号这一项：

$$r + \gamma V(s') - V(s)$$

叫做 **TD error (Temporal Difference Error)**。

记作：

$$\delta = r + \gamma V(s') - V(s)$$

它的含义非常直白：

我原来对这个状态的估计，和这次新看到的证据相比，差了多少。

- 如果 $\delta > 0$ -> 原来估低了，往上调
- 如果 $\delta < 0$ -> 原来估高了，往下调

举个小例子：

- 当前 $V(B)=0.20$
- 下一状态 $V(C)=0.90$
- $r=0$
- $\gamma=0.9$
- $\alpha=0.5$

那么：

$$\text{target} = 0 + 0.9 \times 0.90 = 0.81$$

$$\Delta = 0.81 - 0.20 = 0.61$$

更新后：

$$V(B) \leftarrow 0.20 + 0.5 \times 0.61 = 0.505$$

也就是说：

- 原来估计太低：0.20
- 更合理目标是：0.81
- 这次先向目标靠近一半，变成 0.505

9. Bellman 如何把奖励向前传播

Bellman 最直观的图像是：奖励像热量一样向前传导。

看一个最简单的一维世界：

A -> B -> C -> 🧀

规则：

- 到奶酪得到 +1
- 其他位置 reward 都是 0
- $\gamma = 0.9$

初始：

$$V(A)=0.00, V(B)=0.00, V(C)=0.00, V(\text{🧀})=1.00$$

第一轮：

$$V(C) = 0 + 0.9 \times 1.00 = 0.90$$

第二轮：

$$V(B) = 0 + 0.9 \times 0.90 = 0.81$$

第三轮：

$$V(A) = 0 + 0.9 \times 0.81 = 0.729$$

于是形成 value 梯度：

$$0.729 \rightarrow 0.81 \rightarrow 0.90 \rightarrow 1.00$$

agent 即使看不到终点，也能顺着 value 变高的方向前进。

这就是 Bellman 方程最有力量的地方：

它让终点的奖励可以一层层向前传播，最终照亮整张地图。

10. 为什么 Bellman 更新通常会收敛？

初学时不需要背完整证明，只要抓住直觉：

- 每次更新都在朝 Bellman target 靠近
- 未来 reward 前面会乘 γ
- 当 $\gamma < 1$ 时，越远的影响越弱
- 所以误差不会无限放大，通常会逐渐稳定下来

一句话版：

奖励往前传播，但每传播一层都会打折，所以系统通常会越来越稳定，而不是爆炸。

11. Bellman 在 Q-Learning 里怎么落地

Q-Learning 学的不是 $V(s)$ ，而是：

$$Q(s, a)$$

它使用的更新公式是：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

把它翻译成成人话：

1. 看当前这一步拿了多少 reward
2. 看下一状态里"最好的动作"值多少
3. 把两者合起来，得到一个 Bellman target
4. 用这个 target 去修正当前的 Q 值

对应到代码里通常就是：

```
target = reward + GAMMA * np.max(q_table[next_state])
old_q = q_table[state][action]
q_table[state][action] = old_q + ALPHA * (target - old_q)
```

逐项对应：

- `reward` -> 当前即时反馈
- `np.max(q_table[next_state])` -> 下一状态最好的未来价值
- `reward + gamma * max(...)` -> Bellman target
- `old_q + alpha * (target - old_q)` -> 朝目标修正一步

所以 Q-Learning 的本质就是：

不断用 Bellman optimality 的思想修正 Q 表，最后把整张动作价值地图学出来。

12. 这一章的主线回顾

这一章真正想讲清楚的只有一条线：

1. Reward 不够，需要 Value
2. Value 是对未来总回报的估计
3. Value 分成 $V^{\pi}(s)$ 和 $Q^{\pi}(s, a)$
4. Bellman 把长期价值拆成"当前收益 + 未来价值"
5. 给定策略时，用 Bellman expectation equation 描述 value
6. 追求最优时，用 Bellman optimality equation 描述最优 value
7. 训练时不能直接得到真值，只能用 update 一步步逼近
8. Q-Learning 就是在用这种方式学习 Q 表

一句话收住：

Value Function 是要学的地图，Bellman 方程是地图的递推规则，Q-Learning 是把地图学出来的方法。

本章速记卡片

一句话主线

- Reward 看眼前，Value 看长远
- Value Function 是对未来总回报的估计
- Bellman 把长期价值拆成"当前收益 + 未来价值"
- Q-Learning 会在下一章把这个递推关系真正变成算法

必背概念

- Reward：当前这一步立刻拿到多少奖励
- Return：未来奖励的折扣和
- Value：从当前状态出发，未来整体值多少

- **Policy π** ：Agent 采取动作的规则
- **γ** ：折扣因子，越大越看重未来

必背公式

$$G_t = R_{\{t+1\}} + \gamma R_{\{t+2\}} + \gamma^2 R_{\{t+3\}} + \cdots$$

$$G_t = R_{\{t+1\}} + \gamma G_{\{t+1\}}$$

$$V^{\pi}(s)$$

$$Q^{\pi}(s, a)$$

$$V^{\pi}(s) = \mathbb{E}_{\pi} [R_{\{t+1\}} + \gamma V^{\pi}(S_{\{t+1\}})]$$

$$Q^*(s, a) = \mathbb{E} [R_{\{t+1\}} + \gamma \max_{a'} Q^*(S_{\{t+1\}}, a')]$$

最短区分

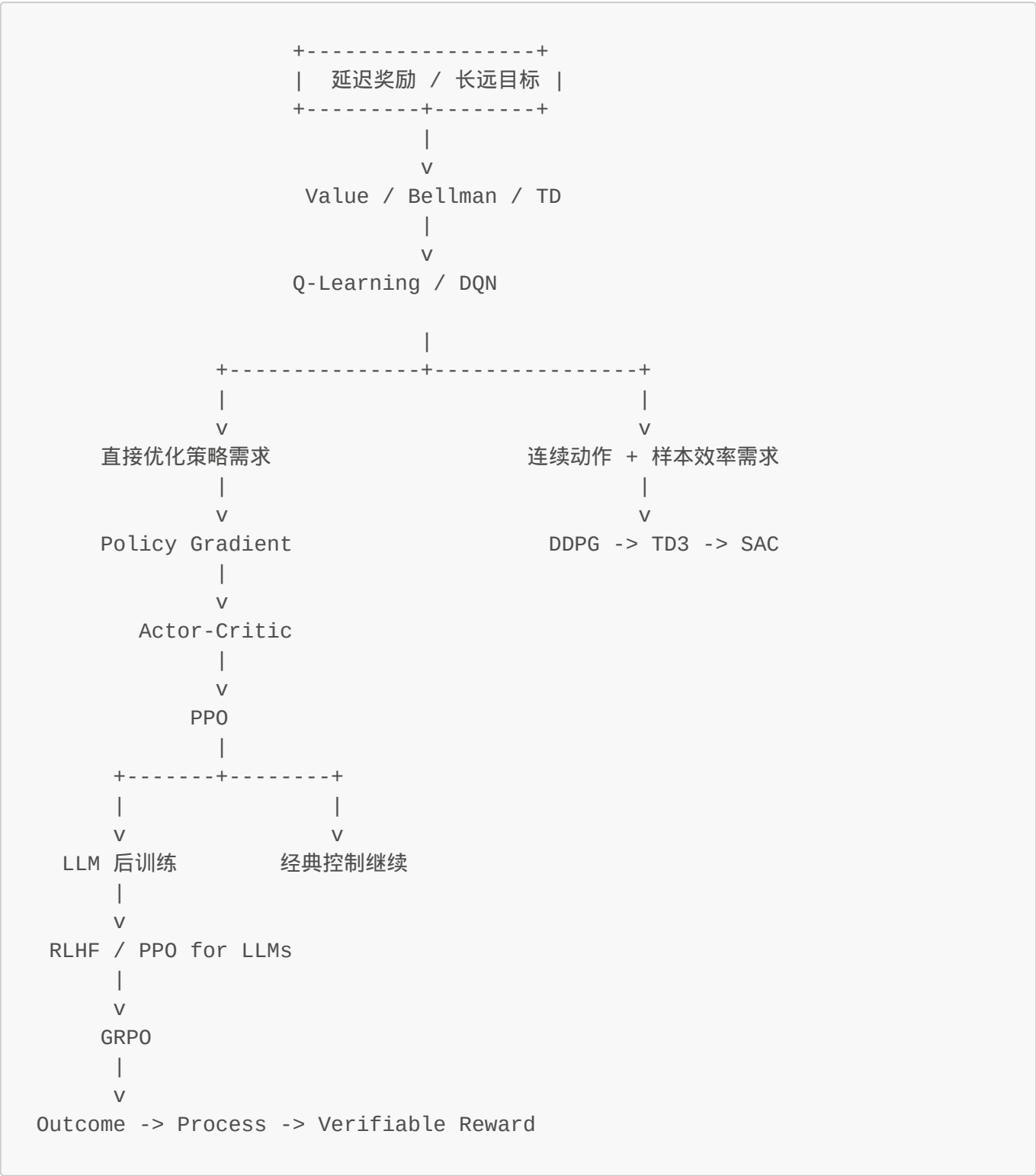
- **V** ：状态值
- **Q** ：动作值
- **Expectation**：按当前策略继续走
- **Optimality**：下一步取最优
- **E** ：对所有可能未来结果做概率加权平均
- **Bellman Equation**：定义正确关系
- **Bellman Update**：一步步逼近这个关系

最短背诵版

1. **Reward** 看眼前，**Value** 看长远
2. **Return** = 未来奖励的折扣和
3. **V** 看状态值， **Q** 看动作值
4. **Bellman**：长期价值 = 当前收益 + 未来价值
5. **Expectation** 评估当前策略，**Optimality** 描述最优策略
6. 下一章开始用 Q-Learning 把这个递推关系变成算法

第四章：Q-Learning（让 Agent 开始真正学习）

全局导航图



你在这里：主干 → Q-Learning

序：为什么 Bellman 方程还不够？

第三章里我们得到了两条方程：

- Bellman Expectation Equation — 评估当前策略有多好

- **Bellman Optimality Equation** — 告诉我最优动作价值应该满足什么关系

但这里有个**致命问题**：

我知道正确答案该长什么样，可我该怎么在不知道环境模型的情况下，把它学出来？

这就是第四章要接手的问题。

🔥 独立重构练习（先别往下读）

藏起公式，自己推导：

1. 如果我想选动作，只知道 $V(s)$ 够不够？为什么必须需要 $Q(s, a)$ ？
2. **Bellman Optimality Equation** 里的期望怎么在现实中计算？有什么可用信息源？
3. 更新公式应该长什么样？从“旧估计 + 修正量”的角度推导

5 分钟后再看下面的答案。

1. 什么问题逼出了 Q-Learning？

第三章的 Bellman 方程告诉我们：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a')]$$

但这个式子有个**无法执行的问题**：

- $\mathbb{E}$ 表示期望，要对所有可能的未来轨迹求平均
- 可现实里我们**不知道环境模型**（转移概率、奖励分布）
- 所以没法精确计算右边的期望

那怎么办？

唯一可用的信息源是真实交互得到的 sample。

这就是 Q-Learning 被迫要解决的问题：

在没有完整环境模型的情况下，怎么用单次样本逼近 Bellman Optimality？

2. 为什么 $V(s)$ 不够，必须学 $Q(s, a)$ ？

假设我只有状态价值函数 $V(s)$ 。它能告诉我：

- “这个状态整体值多少钱”

但真正做决策时，我需要的是：

- 往左好还是往右好？
- 现在加速值不值？
- 哪个动作会把我带向更好的未来？

$V(s)$ 无法回答这些问题 — 因为它是对**所有可能动作的平均**（或最优），但不区分具体动作。

所以这逼出一个更细的量：

$$Q(s, a)$$

它表示：

在状态 s 下，如果我先做动作 a ，未来总回报大概是多少？

一句话：

$V(s)$ 告诉你状态好不好， $Q(s, a)$ 告诉你在该状态下哪个动作更值。

3. Q-Learning 的表示方式：一张表就够了吗？

既然目标是学出每个 (s, a) 的价值，最自然的表示就是一张 **Q-table**：

$$\text{状态 } s \times \text{动作 } a \rightarrow Q \text{ 值}$$

在离散环境里它可能长这样：

	$\leftarrow$	$\rightarrow$
A	0.12	0.45
B	0.30	0.81
C	0.10	0.90

这张表一旦学准，策略就不需要额外发明了：

$$\pi(s) = \arg\max_a Q(s, a)$$

关键洞察：

- Q-table 不是规则表（不写"该怎么做"）
- Q-table 是动作价值表（只记录"每个动作值多少"）
- **策略会从价值里自然长出来，而不是被硬编码**

🔥 探索 vs 利用：为什么这是被迫的？

现在有个新问题浮现了。

假设我初始化 Q-table 为零，然后永远选当前最优动作 $\arg\max(Q)$ 。会发生什么？

- 初始时所有 $Q(s, a) = 0$
- 第一次在某状态随机（或默认）选了某个动作
- 如果这个动作运气好拿到正 reward $\rightarrow$ Q 值上升
- 以后在这个状态永远选它
- **结果：其他可能更好的动作永远没机会被尝试**

这就是**局部最优陷阱**。

所以这逼出一个结论：

Q-Learning 必须引入随机性，否则初始状态的偶然选择会锁死策略。

这就叫 **Exploration（探索）** vs **Exploitation（利用）**。

最经典的解决方案是 **epsilon-greedy**：

- 以概率 ϵ 随机选动作（探索）
- 以概率 $1-\epsilon$ 选当前最优动作（利用）

训练时通常让 ϵ 逐渐下降：前期多探索，后期多利用。

这是被迫的：因为初始 Q 值是零/随机，如果不强制探索，策略会被偶然性锁死。

4. 如何从 Bellman Optimality 推导出 Q-Learning ？

回到 **Bellman Optimality Equation**：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a')]$$

这个式子告诉我们的不是算法步骤，而是：

正确的最优动作价值，应该满足什么结构。

用人话翻译：

- 当前动作价值 = 当前 reward + 下一状态中最好的未来价值

但这里有个**工程障碍**：

- 方程里有 $\mathbb{E}$ （期望）
- 现实里没有完整环境模型
- 没法精确计算右边的期望

那怎么办？

用一次真实交互得到的 sample，去近似这个期望。

也就是：

1. 执行动作 a ，得到真实的 (r, s')
2. 用当前 Q-table 估计 s' 里的最优未来价值 $\max_{a'} Q(s', a')$

3. 拼成 Bellman target: $r + \gamma * \max_{a'} Q(s', a')$

4. 把旧的 $Q(s, a)$ 往这个 target 推进一点

于是得到：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

验证这个推导：

- ✓ 保留 Bellman 结构？是，仍然是 当前 reward + 最优未来价值
- ✓ 保留 optimality 目标？是，未来项仍是 max
- ✓ 把不可计算的期望变成可执行步骤？是，用单次 sample 近似环境期望

一句话：

Q-Learning = Bellman Optimality Equation 的样本版本。

5. Q-Learning Update 公式拆解

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

第 1 部分： $Q(s, a)$ (旧估计)

- 我之前认为，在状态 s 做动作 a 值这么多

第 2 部分： $r + \gamma * \max Q(s', a')$ (Bellman Target)

- **眼前收益**：这一步立刻拿到的 reward
- **未来收益**：下一状态里最好的动作价值
- target = 眼前 + 未来

第 3 部分： $\text{target} - Q(s, a)$ (TD Error)

- **新证据和旧估计的差距**
- 正数 → 原来估低了，该往上修
- 负数 → 原来估高了，该往下压

第 4 部分： $\alpha * (\dots)$ (学习率)

- 控制每次修正的力度
- α 大 → 改得更猛（可能震荡）
- α 小 → 改得更稳（收敛慢）

6. 一个极小示例：Q 值更新是怎么发生的？

假设当前在状态 B ，执行动作 $\rightarrow$ ：

参数	值
reward r	0
下一状态	c
$\max_{a'} Q(C, a')$	0.90
γ (γ)	0.9
α (α)	0.1
$Q(B, \rightarrow)$ (旧值)	0.40

Step 1：计算 target

$$\text{target} = r + \gamma \times \max_{a'} Q(s', a') = 0 + 0.9 \times 0.90 = 0.81$$

Step 2：计算 TD error

$$\text{TD error} = \text{target} - \text{old_Q} = 0.81 - 0.40 = 0.41$$

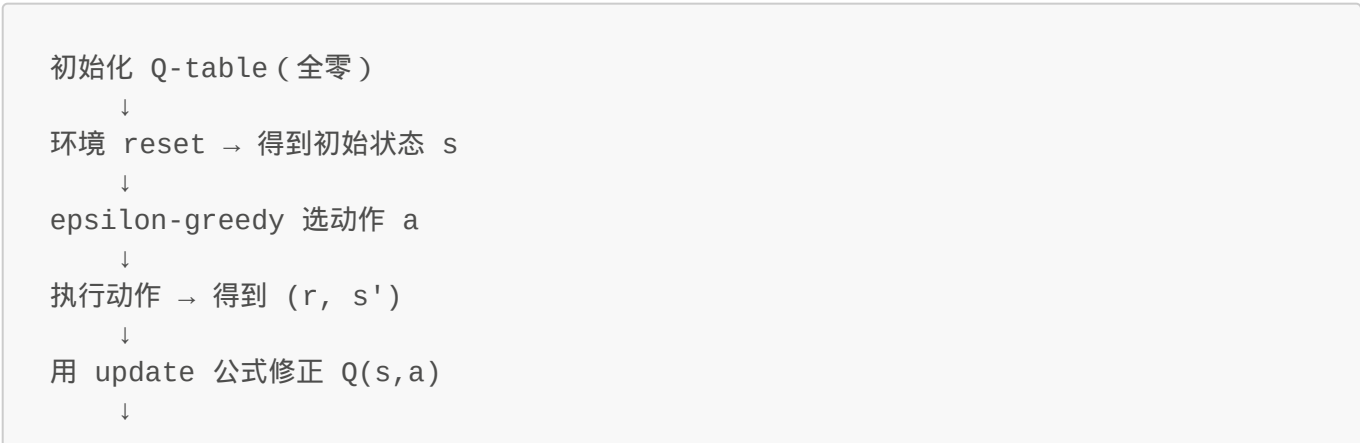
Step 3：更新 Q 值

$$Q(B, \rightarrow) \leftarrow 0.40 + 0.1 \times 0.41 = 0.441$$

含义：

- 原来我以为 $B \rightarrow$ 值 0.40
 - 新经验告诉我它应该接近 0.81
 - 所以往新证据修正一点点，变成 0.441
- 一次更新只挪一点，但很多次之后，整张 Q-table 就会越来越准。

7. Q-Learning 的完整训练流程



```
状态变成 s'  
↓  
如果没结束，继续循环  
↓  
episode 结束 → 开始下一局
```

两个核心效果：

1. **奖励逐步向前传播** — 终点的高 reward 会像涟漪一样往前扩散
2. **策略从 Q-table 自然长出来** — 不需要手写规则，只需选 `argmax(Q)`

8. 对应到代码：03_q_learning.py 在做什么？

8.1 建环境

```
env = gym.make("CartPole-v1")
```

- 任务是 CartPole（小车杆不倒）

8.2 离散化连续状态

CartPole 的观察是连续的：

- 小车位置 `0.137`
- 杆角度 `-0.024`

但 Q-table 需要离散索引，所以要 discretize：

```
def discretize(state):  
    # 把连续值压到 NUM_BINS 个桶里  
    return (bin1, bin2, bin3, bin4)
```

8.3 初始化 Q-table

```
q_table = np.zeros((NUM_BINS, NUM_BINS, NUM_BINS, NUM_BINS, action_size))
```

- 初始全零，表示"我对所有状态动作都没有经验"

8.4 Epsilon-Greedy 选动作

```
def choose_action(state, epsilon):  
    if random() < epsilon:  
        return random_action() # 探索
```

```
else:
    return argmax(q_table[state]) # 利用
```

- 前期多随机试错，后期多相信自己学到的东西

8.5 Q-Learning Update

```
old_q = q_table[state][action]
next_max = np.max(q_table[next_state])
target = reward + GAMMA * next_max
q_table[state][action] = old_q + ALPHA * (target - old_q)
```

- 和公式一一对应： $Q \leftarrow Q + \alpha(\text{target} - Q)$

8.6 Epsilon Decay

```
epsilon = max(EPSILON_END, epsilon * EPSILON_DECAY)
```

- 开始多探索，逐渐减少随机性

🔥 独立重构练习（藏起答案再试一次）

现在盖住公式，自己从头推导 Q-Learning：

1. 为什么 $V(s)$ 不够？必须学什么？ $\rightarrow Q(s, a)$
2. Bellman Optimality 里的期望怎么计算？ $\rightarrow$ 用 sample 近似
3. 更新方向应该是什么？ $\rightarrow$ 向 Bellman target 靠近
4. 为什么需要 epsilon-greedy？ $\rightarrow$ 初始 Q 值随机，必须强制探索

如果每一步都能独立重建，说明你真的理解了。

9. Q-Learning 的局限与下一步

✓ 优点

- 思想清晰：从 Bellman 直接推导出来
- 代码简单：几行就能实现
- 适合入门：把 RL 核心概念串起来

✗ 局限

- Q-table 需要离散状态 — 连续空间要 discretize，信息会损失
- 状态爆炸 — 状态维度一多，表就存不下
- 泛化能力弱 — 没见过的状态完全不会处理

这就是为什么后面要走 DQN：

用神经网络代替 Q-table，让模型学会“没见过的状态该怎么估计价值”。

本章主线回顾

问题：Bellman 方程告诉我正确答案的结构，但怎么学出来？
 ↓
 出发点：V(s) 不够选动作，必须学 Q(s, a)
 ↓
 表示方式：Q-table 记录每个 (s, a) 的价值
 ↓
 探索困境：初始 Q 值随机，不强制探索会被锁死 → epsilon-greedy
 ↓
 更新规则：用 sample 逼近 Bellman Optimality
 ↓
 验证：保留结构、保留目标、可执行

一句话收束：

Q-Learning = 在没有环境模型的情况下，用样本数据一步步逼近 Bellman Optimality。

本章速记卡片

🔑 核心洞察

1. V(s) 只告诉你状态好不好，Q(s, a) 直接告诉你在该状态下哪个动作更值
2. Q-Learning 学的是一张 Q-table，策略从价值里自然长出来
3. 初始 Q 值为零 → 必须强制探索 (epsilon-greedy)，否则会被偶然性锁死
4. Update target = 当前 reward + γ * 下一状态最优未来
5. TD error = target - old_Q，告诉你是该往上调还是往下压

📐 必背公式

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- α ：学习率（每次修正的力度）
- $[\dots]$ ：TD error（新证据和旧估计的差距）

🎯 最短背诵版

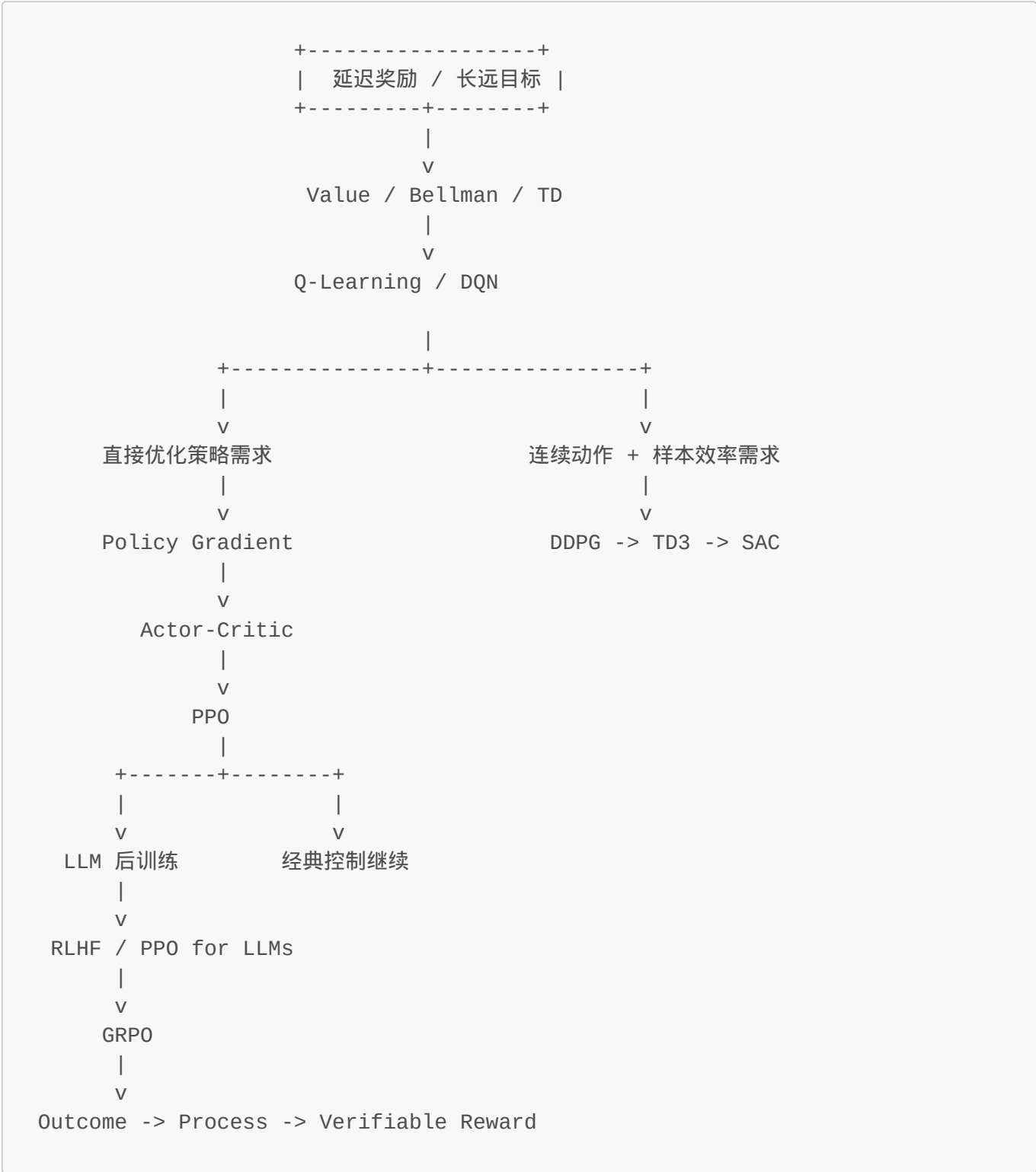
1. Q 比 V 更适合直接选动作
2. Q-Learning = Bellman Optimality + Sample Update
3. Epsilon-Greedy 是**被迫的**：初始 Q 值随机，不探索会锁死
4. TD error 驱动更新 → 向更合理的目标靠近

5. 状态太大时 Q-table 不够用 → DQN 出场

下一章：DQN — 当状态空间爆炸，神经网络如何接管 Q-Learning？

第五章：Deep Q-Network (DQN) - 用神经网络替代 Q 表

全局导航图



学习脊柱 (Learning Spine)

这一章不是讲"又多了一个新算法"，而是解决 Q-Learning 在真实世界里的**可扩展性瓶颈**。

问题 → 出发点 → 发明 → 验证 → 示例

Problem：什么矛盾迫使 DQN 必须存在？

Q-Learning 的更新公式本身没有缺陷：

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

但**存储方式**彻底失效了。

Q-table 的三个致命限制

1. 状态空间必须有限且可枚举

```
CartPole: s = [cart_position, cart_velocity, pole_angle,
               pole_angular_velocity]
→ 4 个连续变量，无法直接查表
```

2. 离散化会指数爆炸

- 每个维度切 100 格： $100^4 = 100,000,000$ 个状态
- 稍微复杂一点的任务就不可行

3. 没有泛化能力

- 学过状态 A，不代表会状态 B
- 即使 A 和 B 几乎一样，也得开学一遍

核心矛盾：Q-Learning 的更新规则完美，但 Q-table 无法扩展到真实任务。

Starting Point：我们已有的工具是什么？

已确认的事实

- Bellman 方程是正确的** - 它定义了 RL 问题的本质结构
- Q-Learning 的更新逻辑是对的** - $r + \gamma \max Q(s',a')$ 作为 target 有效
- 问题是表示，不是算法** - 如果把 Q-table 换成别的存储方式会怎样？

唯一可行的路径

如果状态空间太大无法查表，那么：

必须用一个能泛化的函数来代替表格。

这不是"新想法"，而是**被迫的唯一选择**。

Invention：如何推导出 DQN？

核心发明： $Q(s,a) \rightarrow Q(s,a; \theta)$

不是把 Q-table 改大一点，而是彻底换掉存储方式：

旧方式： $Q[s][a] = \text{float}$ （查表）
新方式： $Q(s, a; \theta) = \text{neural_network_output}$ （函数逼近）

关键洞察：

- 输入状态 $s \rightarrow$ 神经网络输出各动作的 Q 值
- 参数 θ 被训练成"压缩版的 Q-table"
- 相似状态共享参数 $\rightarrow$ 自动泛化

CartPole 的例子

输入： $s = [x, x_dot, theta, theta_dot]$ （4 个连续值）
输出： $[Q(s, \text{left}), Q(s, \text{right})]$ （2 个动作的 Q 值）

不需要离散步骤，直接端到端。

Verification：如何证明这个方案真的有效？

验证标准 1：保留 Bellman 结构

DQN 的训练目标仍然是：

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$
$$L(\theta) = (y - Q(s, a; \theta))^2$$

关键：target 仍然是 bootstrap 的 Bellman target，只是 Q 现在是函数而不是表格。

验证标准 2：解决可扩展性问题

对比实验（理论推导）：

场景	Q-table	DQN
----	---------	-----

场景	Q-table	DQN
CartPole (连续状态)	✗ 无法直接用	✓ 直接处理
Atari (像素输入)	✗ 完全不可能	✓ 端到端学习
泛化到未见状态	✗ 不会	✓ 会

验证标准 3：训练稳定性补丁

问题：直接用神经网络学 Q-Learning 会发散，为什么？

- 1. 样本强相关 - 连续步骤的状态几乎一样
- 2. 目标漂移 - target 网络自己也在变，导致训练目标乱飘

补丁（不是核心发明）：

- Replay Buffer：打散相关性，让数据更接近 i.i.d.
- Target Network：固定 teacher，防止"自己改答案又拿新答案当老师"

验证结论：DQN = Q-Learning 的更新规则 + 神经网络的泛化能力。稳定性补丁是工程必要性，不是核心创新。

Example：最小可行示例 - CartPole

为什么选 CartPole？

这是最小的能体现 DQN 必要性的任务：

- ✓ 状态连续（Q-table 无法直接用）
- ✓ 动作离散（DQN 适用，还没到连续动作的复杂度）
- ✗ 不是 Cloning（那是 DDPG/TD3/SAC 的地盘）

训练流程骨架

```
for episode in range(num_episodes):
    state = env.reset()

    # 1. 收集经验 → Replay Buffer
    while not done:
        action = epsilon_greedy_policy(state, online_net)
        next_state, reward, done = env.step(action)
        replay_buffer.add(state, action, reward, next_state, done)

    # 2. 从 buffer 采样训练 → 打散相关性
    batch = replay_buffer.sample(batch_size)

    # 3. Bellman target (用 target network)
    y = r + gamma * max(target_net(next_state))
```

```
# 4. Loss & Backprop
loss = mse(online_net(state)[action], y)
```

预期效果

- **初期**：平均坚持步数 ~10-20（随机策略水平）
- **训练后**：平均步数涨到几百（学会平衡杆子）
- **泛化验证**：给没见过的新初始状态，也能正确应对

Compression：DQN 的本质压缩形式

最简表达

```
Q-Learning + Function Approximation = DQN
Replay Buffer + Target Network = 稳定性补丁（工程必要性）
```

历史定位

DQN 完成了价值函数 RL 从"小表格玩具"到"高维感知任务"的跨越。

- Q-Learning：解决了"怎么学动作价值"
- DQN：解决了"在大状态空间里怎么表示动作价值"

Transition：接下来去哪里？

DQN 的边界

1. **只能处理离散动作** - CartPole（左右）、Atari（按键）OK，但连续控制不行
2. **样本效率仍然不高** - 需要大量交互才能收敛
3. **训练不稳定** - 即使有 replay buffer + target network，仍是工程挑战

自然的下一步

"既然我最终想要的是策略 $\pi(a|s)$ ，能不能别绕道学 Q，直接优化策略本身？"

这就逼出了下一章：**Policy Gradient**。

本章速记卡片 (Recap Card)

Problem

- Q-table 无法扩展到连续状态空间
- Bellman 更新规则正确，但存储方式失效

Starting Point

- Q-Learning 的 Bellman target 已经有效
- 唯一瓶颈是表示方式的泛化能力

Invention

- $Q(s,a) \rightarrow Q(s,a; \theta)$ ：用神经网络代替查表
- 参数共享实现自动泛化

Verification

- Bellman 结构完全保留
- Replay Buffer + Target Network 解决稳定性
- CartPole/Atari 实验验证可扩展性

Example (最小可行案例)

```
# DQN 核心训练循环
batch = replay_buffer.sample(batch_size)
y = r + gamma * max(target_net(next_state))
loss = mse(online_net(state)[action], y)
backward(loss)
```

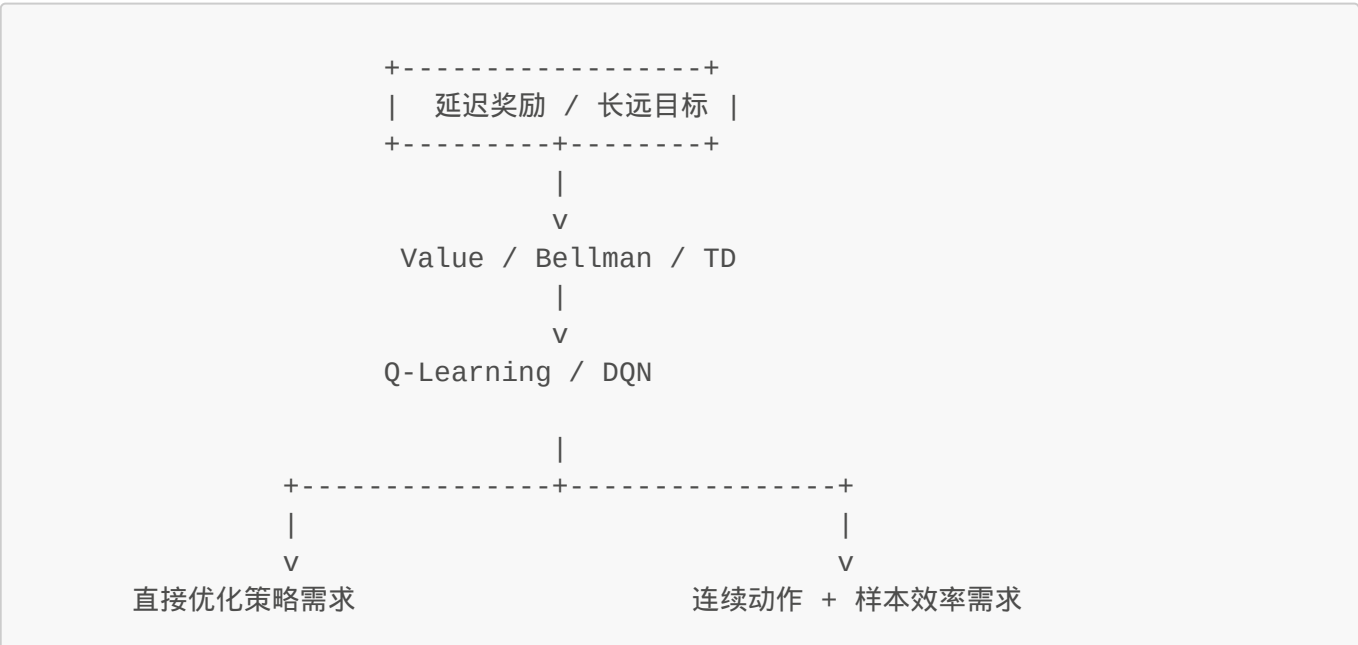
下一章预告：Policy Gradient

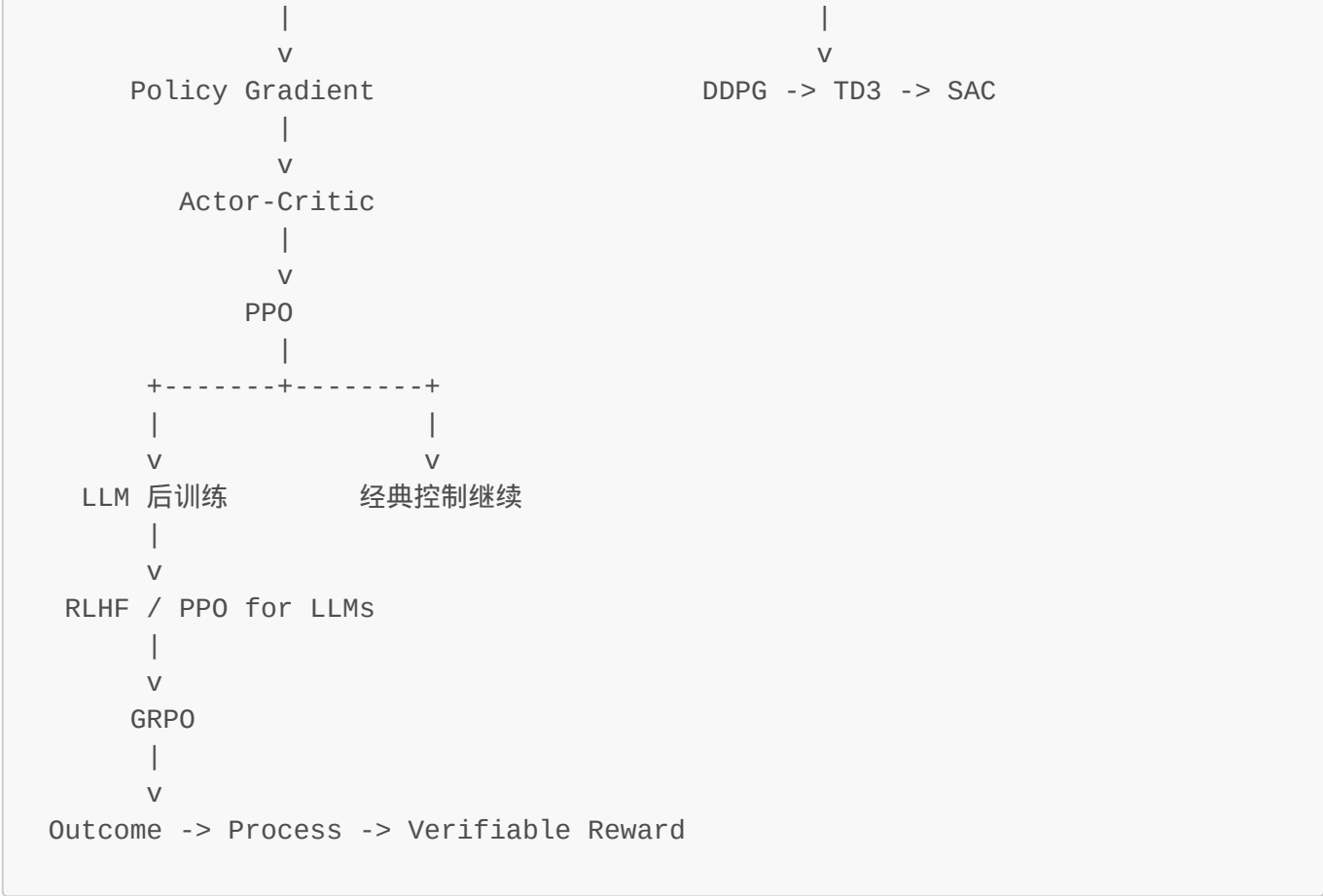
既然 Q-Learning 和 DQN 都是学"动作价值"，那有没有可能**直接优化策略本身**？

这就是 Policy Gradient 要解决的问题。

第六章：Policy Gradient - 直接优化策略

全局导航图





你在这里：主干 -> Policy Gradient

序：从"先学价值"到"直接学行为"

这一章不要一上来背梯度公式。先抓住那个真正把新方法逼出来的问题：

我最终要优化的是行为策略
但 value-based 方法要先学 Q，再从 Q 间接导出策略
如果动作是连续的，或者我关心的是动作分布本身，这条路就开始变别扭

先把这一章的主线钉住：

problem -> starting point -> invention -> verification -> example

一句话先压住：

Policy Gradient 的本质，是不再把策略当成 value 的附属品，而是直接把策略本身当作优化对象。

1. Problem：为什么 value-based 路线会自然逼出 Policy Gradient？

先看前面的方法在干什么：


```
Q-Learning / DQN:
先学  $Q(s, a)$ 
  -> 再选  $\operatorname{argmax}_a Q(s, a)$ 
  -> 策略是"从  $Q$  里顺手挑出来的"
```

这条路线在离散动作里很好用，但它天然有三个压力点。

1.1 动作一旦连续， argmax 就开始难看

如果动作只是：

- 左 / 右
- 上 / 下

那从 $Q(s, a)$ 里挑最大值很容易。

但如果动作变成：

- 方向盘角度
- 关节扭矩
- 油门大小

动作空间是连续的，你就没法再轻松地做一个干净的 argmax 。为了继续用 value-based，你往往得把连续动作硬离散化，而这会立刻带来：

- 维度爆炸
- 分辨率粗糙
- 控制不自然

1.2 你真正想学的是策略，但系统在逼你先学价值

从任务目标看，agent 最终被执行的不是 Q ，而是策略 π 。

也就是说：

- 真正被部署的是"怎么行动"
- 不是"每个动作值多少钱的中间表征"

这就像你真正想训练的是"驾驶动作"，但当前流程要求你先建立一张"所有可能动作评分表"，再从表里取最大。能用，但绕。

1.3 有时我们关心的就是"动作分布本身"

很多任务里，策略不是一个单点动作，而是一个分布：

- 我想保留随机探索
- 我想输出高斯分布参数
- 我想控制策略熵，让行为不要过早塌缩

这时候"先学值，再取最大"本身就不再是最自然的语言。

所以这一章的原始问题可以压成一句：

如果最终目标本来就是策略，那为什么不直接优化策略本身？

2. Starting Point：既然目标是行为规则，那就直接表示行为规则

从第一性原理看，出发点其实非常朴素：

我要控制什么，就直接把什么写成参数化对象。

在 RL 里，我们真正想控制的是：

在状态 s 下，采取动作 a 的行为倾向。

那最自然的表示就是：

$$\pi_{\theta}(a|s)$$

这里：

- θ 是策略参数
- 输入是状态 s
- 输出是一个动作分布或动作概率

如果动作空间离散，它可能输出：

$$\pi_{\theta}(a|s) = [0.1, 0.7, 0.2]$$

意思是：

- 动作 1 概率 10%
- 动作 2 概率 70%
- 动作 3 概率 20%

如果动作空间连续，它也可以输出一个分布的参数，比如高斯分布的：

- μ
- σ

于是目标函数也跟着改写。

以前我们隐含地在做：

- 先学一个 value
- 再让策略从 value 里长出来

现在我们直接写：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

含义非常直接：

- τ 是按当前策略跑出来的一条轨迹
- $R(\tau)$ 是这条轨迹的总回报
- 我要调整 θ ，让期望总回报最大

这一步特别重要，因为它完成了目标层面的切换：

旧路线：优化 value，策略是副产物
新路线：直接优化策略，value 不再是必须中介

3. Invention：Policy Gradient 是怎么被"逼出来"的？

一旦你接受"策略本身就是优化对象"，下一个问题就被强行摆到桌面上：

参数 θ 到底应该朝哪个方向改，才能让策略变好？

别急着看公式，先看最原始的行为逻辑。

3.1 最朴素的规则：好动作以后更常做，差动作以后少做

如果某次在状态 s_t 下选了动作 a_t ，结果后面拿到了很高的回报，那最自然的更新方向就是：

- 下次在类似状态里，更容易选到这个动作

如果后面回报很差，那最自然的更新方向就是：

- 下次在类似状态里，更不容易选到这个动作

这其实就是 reinforcement 的原型：

好结果 -> 增加这类行为的概率
差结果 -> 降低这类行为的概率

3.2 把"增大/减小概率"写成可微的形式

既然策略已经被写成 $\pi_{\theta}(a|s)$ ，那"让某个动作更容易被选到"本质上就是：

- 增大 $\pi_{\theta}(a_t|s_t)$

为了得到可优化形式，我们看它的对数概率：

$$\log \pi_{\theta}(a_t|s_t)$$

为什么看 $\log$ ？因为它在数学上更容易求梯度，也能把概率乘积变成求和，这是后面整条推导能工作的重要压缩方式。

于是，"朝着更可能选到好动作的方向更新"就变成了：

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

它表示：

- 如果我动一下参数 θ
- 这个动作被选中的倾向会怎么变化

3.3 用回报给这个方向加权

但不是每个动作都该一视同仁。动作后果不同，所以更新强度也应该不同。

最直接的做法就是拿这个动作之后的累计回报 G_t 来当权重：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

这条式子读成人话就是：

- 如果 G_t 大，说明这次动作后果好 -> 增大它的概率
- 如果 G_t 小，说明这次动作后果差 -> 减小它的概率贡献

于是你会发现，Policy Gradient 根本不是凭空冒出来的技巧，而是下面这条直觉的微分版本：

策略参数应该朝着"让高回报动作更常发生"的方向移动。

3.4 REINFORCE：最纯的基础实现

最基础的 Policy Gradient 算法就是 **REINFORCE**。

它的流程非常朴素：

1. 用当前策略跑完整个 episode
2. 记录每一步 (s_t, a_t, r_t)
3. 反向算出每一步之后的累计回报 G_t
4. 用 $-\log_{\text{prob}} * G_t$ 作为 loss 更新参数

伪代码：

```
for episode in range(num_episodes):
    trajectory = []
    state = env.reset()
```

```

done = False

while not done:
    action = sample_from(policy_net(state))
    next_state, reward, done = env.step(action)
    trajectory.append((state, action, reward))
    state = next_state

returns = compute_discounted_returns(trajectory)

loss = 0
for (state, action, _), G in zip(trajectory, returns):
    log_prob = policy_net.log_prob(state, action)
    loss += -log_prob * G

optimizer.zero_grad()
loss.backward()
optimizer.step()

```

这段代码背后的思想只有一句：

用最终回报，去重塑产生这些动作的概率分布。

4. Verification：怎么验证我们真的发明对了？

一套新方法不是“能写出公式”就算成立，而是要检查它是否真的解决了原问题。

4.1 它有没有直接作用在策略上？

有。

它更新的不是 Q 表，也不是某个中间价值映射，而是：

$$\pi_{\theta}(a|s)$$

所以目标和被优化对象终于对齐了。

4.2 它能不能自然处理连续动作？

能。

因为策略输出的不必是离散动作标签，它可以直接输出连续分布参数，比如：

- 均值 μ
- 方差 σ

然后从这个分布里采样动作。

这意味着：

- 不需要粗暴离散化动作空间
- 不需要在连续动作上做难看的 $\arg\max$

4.3 它会不会把高回报行为推高、把低回报行为压低？

会。

因为梯度里直接乘了 G_t ：

- G_t 大 $\rightarrow$ 更新方向强化这类动作
- G_t 小 $\rightarrow$ 更新方向削弱这类动作

这刚好符合最初的行为逻辑。

4.4 那为什么它还不够完美？

因为它虽然方向对了，但信号太吵。

REINFORCE 用的是整段回报 G_t ，于是：

- 单次运气波动会很大
- 每一步动作都背着后续整条轨迹的锅
- 学习方差很高，收敛会慢

所以验证的结论是：

Policy Gradient 在"目标对齐"和"连续动作处理"上是对的
但在"学习信号稳定性"上还不够好

这就逼出下一章的核心问题：

既然直接学策略的方向是对的，那怎么给它一个更稳定的评价器？

这就是 Actor-Critic。

5. Example：走窄桥的小机器人

这个例子最适合说明"为什么直接学策略是自然的"。

```

起点  ----  窄桥  ----  终点
          |
          +-- 掉下去  = -100
  
```

机器人每一步要控制：

- 往前迈多少
- 身体偏左还是偏右

这些都不是离散标签，而是连续控制量。

5.1 如果用 Q-table，会发生什么？

你得把动作离散化成很多小格子：

- 偏左 0.01
- 偏左 0.02
- 偏右 0.01
- 偏右 0.02
- ...

很快动作表就爆炸，而且控制会变得很僵硬。

5.2 如果用 Policy Gradient，会发生什么？

策略网络可以直接输出一个连续动作分布，比如：

```
mu = 0.02
sigma = 0.10
```

意思是：

- 默认平均稍微偏右一点
- 但保留一些探索波动

如果训练后发现：

- 偏右一点更容易走到终点
- 偏左容易掉下桥

那更新后的策略就会逐渐变成：

- **mu** 向安全方向移动
- **sigma** 变小，动作更稳

这正是直接优化行为分布的含义。

6. 这一章最后压成一张脑图

Problem:
value-based 先学 Q 再导出策略，连续动作和动作分布场景下很绕

Starting Point:
真正目标是策略，那就直接参数化策略 $\pi_{\theta}(a|s)$

Invention:
用回报 G_t 给 log-prob 梯度加权

让高回报动作概率上升，低回报动作概率下降

Verification:

它直接优化策略，天然适合连续动作
但更新信号方差大，不够稳定

Example:

走窄桥机器人直接输出连续动作分布，比查 Q 表更自然

本章速记卡片

一句话主线

- value-based 是"先学价值，再导出策略"
- Policy Gradient 是"既然目标是策略，那就直接优化策略"
- 它解决了连续动作和策略分布表达问题
- 但它仍然留下高方差问题

必背公式

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}[G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

必背术语

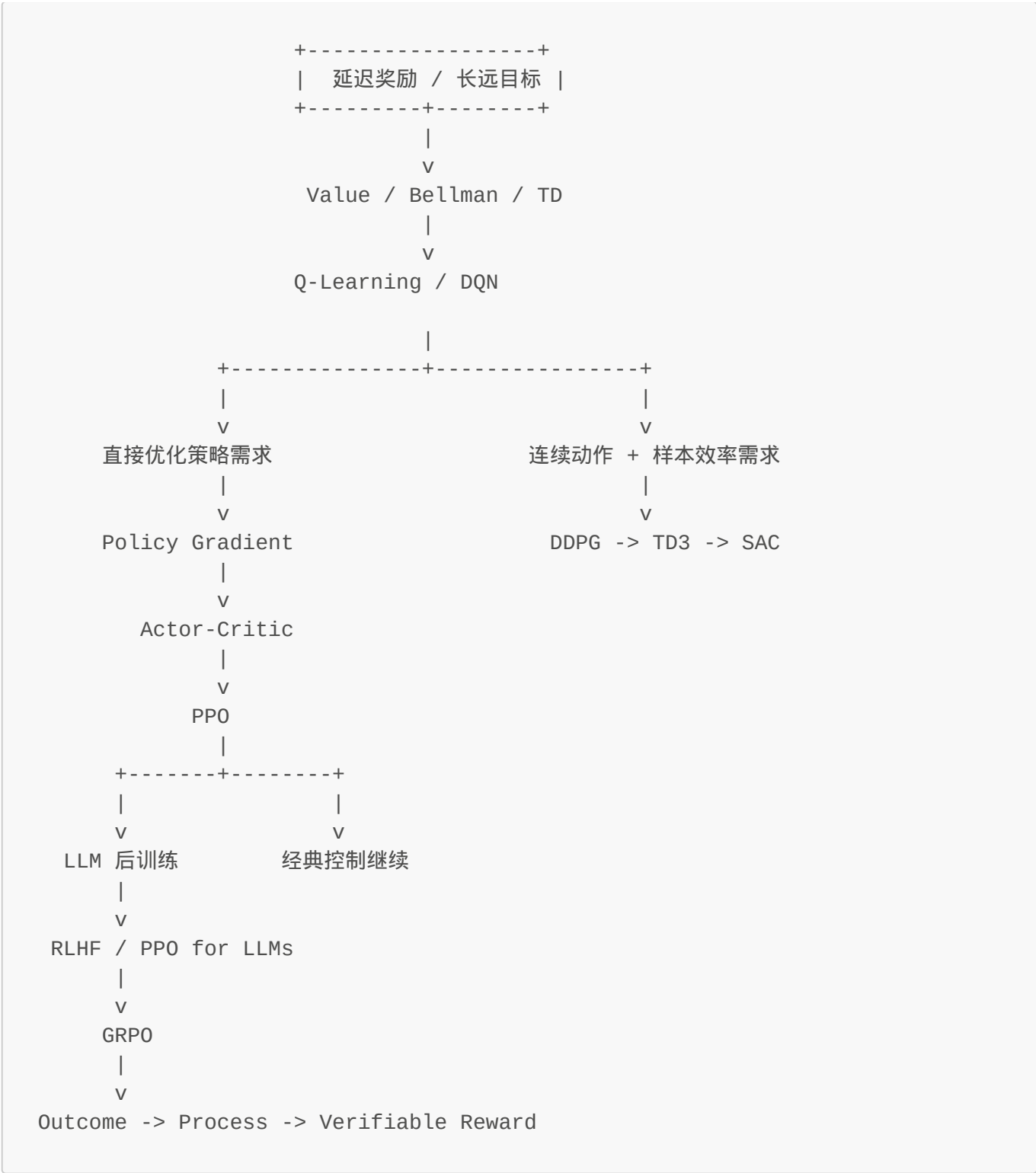
- Policy Parameterization
- Log-Probability
- Return G_t
- REINFORCE

最短背诵版

1. 最终目标是策略，不是 Q 表
2. 所以直接把策略写成 $\pi_{\theta}(a|s)$
3. 高回报动作提概率，低回报动作降概率
4. 连续动作很友好
5. 但训练信号方差大

第七章：Actor-Critic - 结合 Policy 和 Value

全局导航图



你在这里：主干 -> Actor-Critic

序：从"直接学策略"到"给策略配一个评价器"

第六章已经把一件事说透了：

直接优化策略这条路，本身是对的。

但它马上暴露出一个新问题：

策略虽然直接学了，可更新信号太吵。

先把这一章的主线钉住：

```
problem -> starting point -> invention -> verification -> example
```

一句话先压住：

Actor-Critic 的本质，是让一个模块负责行动，让另一个模块负责评价，从而把"直接学策略"和"稳定给反馈"结合起来。

1. Problem：纯 Policy Gradient 到底卡在哪里？

在 **REINFORCE** 里，策略更新大致长这样：

$$G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

问题不是方向错，而是信号太粗。

1.1 每一步动作都背着整条轨迹的锅

G_t 是从时刻 t 往后的累计回报。

这意味着：

- 你在第 t 步做了一个动作
- 但给你的评价，掺杂了后面很多步的运气和失误

于是 credit assignment 会变得很糊：

- 到底是我刚才那一步好
- 还是后面几步运气好
- 或者只是环境随机性在作怪

1.2 方差大，训练容易飘

在早期训练里尤其明显：

- 有时一条轨迹突然成功了
- 有时一条轨迹突然全崩了

如果直接用整段回报去更新，那么策略就会被这些大波动拉来拉去。

也就是说，Policy Gradient 现在遇到的不是"目标错了"，而是：

缺少一个更局部、更稳定的评价信号。

2. Starting Point：Actor 需要的不是总分，而是"这一步比平均好多少"

如果从第一性原理继续推，我们要先问：

一个负责行动的策略模块，真正需要什么样的反馈？

答案不是"整局最终总分"。

因为对当前动作来说，更有用的问题其实是：

我刚才这个动作，在当前状态下，相比平均水平到底是更好还是更差？

这句话非常关键，因为它把反馈需求从：

- 全局、粗糙的 G_t

改成了：

- 局部、相对的"是否优于 baseline"

于是自然就会出现模块分工。

Actor

负责行动：

$$\pi_{\theta}(a|s)$$

它回答的是：

在状态 s 下，我该怎么做？

Critic

负责评价：

$$V_w(s) \quad \text{或} \quad Q_w(s, a)$$

它回答的是：

当前状态值多少？刚才那个动作相对平均水平怎么样？

所以这一章的出发点不是"我要两个网络"，而是：

我需要把"做动作"和"评估动作"分成两个职能。

3. Invention：Actor-Critic 是怎么长出来的？

一旦接受"行动者需要评价者"，下一步自然会问：

Critic 到底应该给 Actor 什么信息，才最有用？

3.1 只给 $V(s)$ 还不够

如果 Critic 只告诉你：

$$V(s)$$

那意思只是：

- 这个状态整体平均值不错/不好

但 Actor 真正关心的是：

- 我刚才选的这个动作，相对于这个状态下的平均动作，到底更好还是更差？

所以只知道"状态总体值"还不够，必须有一个"相对超额收益"的量。

3.2 Advantage 被逼出来了

最自然的桥梁就是：

$$A(s, a) = Q(s, a) - V(s)$$

它的意思是：

- $Q(s, a)$ ：做这个具体动作的价值
- $V(s)$ ：这个状态下平均来说的价值
- 二者相减：这个动作比平均水平高出多少

所以：

- $A > 0$ -> 这个动作比平均好，应该更常做
- $A < 0$ -> 这个动作比平均差，应该少做

这一步极其关键，因为它把问题从：

这局总分怎么样？

转成了：

这一步相对平均操作，超额表现是多少？

3.3 TD error 成为一个便宜又有效的 advantage 近似

但精确求 $Q(s, a)$ 往往不便宜，所以工程上会进一步压缩成一步 TD 形式：

$$\Delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

这其实就是：

- 新证据给出的 Bellman target
- 减去旧估计

于是 δ_t 同时扮演两个角色：

- 对 Critic 来说，它是 TD error
- 对 Actor 来说，它可以近似"这一步比平均好多少"

这就是 Actor-Critic 最漂亮的地方：

Bellman 的局部递推，被重新接回了策略优化。

3.4 两个模块开始互相配合

Critic 更新 value，让它更接近真实状态值：

$$L_{\text{critic}} = (r + \gamma V(s') - V(s))^2$$

Actor 用 advantage 信号调整策略：

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

核心直觉就一句：

Critic 把"这一步表现怎样"翻译成稳定信号，Actor 根据这个信号调整行为。

4. Verification：怎么确认 Actor-Critic 真解决了前一章的问题？

4.1 它有没有保留"直接学策略"的优点？

有。

Actor 仍然在直接优化：

$$\pi_{\theta}(a|s)$$

所以：

- 连续动作仍然友好
- 策略分布仍然可以直接建模

也就是说，第六章最珍贵的东西没有丢。

4.2 它有没有让更新信号更局部、更稳定？

有。

相比 G_t 要背整条轨迹， A_t 或 δ_t 只关心：

- 当前 reward
- 下一个状态的价值估计
- 当前状态的旧估计

所以反馈变得更短、更局部，方差通常明显下降。

4.3 它还能不能区分"比平均好"还是"比平均差"？

能。

这正是 advantage 的意义：

- 正值 -> 强化这个动作
- 负值 -> 削弱这个动作

所以 Actor 不再是被"整局总分"粗暴驱动，而是在接受更细粒度的相对反馈。

4.4 它有没有新的代价？

有。

你现在不是只训练一个策略网络，而是要让：

- Actor
- Critic

同时学习、互相配合。

如果 Critic 评得不准，Actor 也会被带偏。所以它解决了方差问题，但引入了系统耦合和实现复杂度。

验证结论可以压成一句：

Actor-Critic 用额外的评价模块，换来了更稳定的策略学习信号。

5. Example：双足机器人学走路

这个例子特别适合说明为什么 Actor-Critic 比纯 Policy Gradient 更合理。

5.1 任务长什么样？

状态可能包括：

- 身体姿态
- 关节角度

- 关节速度
- 足底接触信息

动作是：

- 每个关节的连续扭矩输出

奖励可能是：

- 往前走给正奖励
- 摔倒给大负奖励
- 动作抖动太厉害要扣分

5.2 为什么纯 Policy Gradient 会很难受？

因为 early stage 几乎一直在摔。

如果你用整条轨迹回报更新，就容易出现：

- 这一局摔了 -> 前面很多本来还不错的小动作也一起被否定
- 偶然多走了两步 -> 很多动作又被过度奖励

信号太粗。

5.3 Actor-Critic 在这里怎么工作？

- **Actor** 输出每个关节动作的分布
- **Critic** 评估当前姿态值不值钱
- 每走一步，Critic 都能给出一个相对局部的反馈：
 - 刚才那一步发力，相比平均来说更稳还是更差？

这样学习信号就从：

整局最后摔了，所以你前面全都不行

变成：

虽然最后摔了，但你刚才那一步其实比平均好，值得保留

这就是它为什么能显著提升学习效率的直觉来源。

6. 本章速记卡片

核心直觉

Policy Gradient 方向是对的，但信号太吵。加一个 Critic 给 Actor 更稳定的反馈，关键桥梁是 **Advantage**。

三个公式就够了

$$A(s,a) = Q(s,a) - V(s) \quad \text{\texttt{\text{ (比平均好多少) }}$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad \text{\texttt{\text{ (TD error 近似 advantage) }}$$

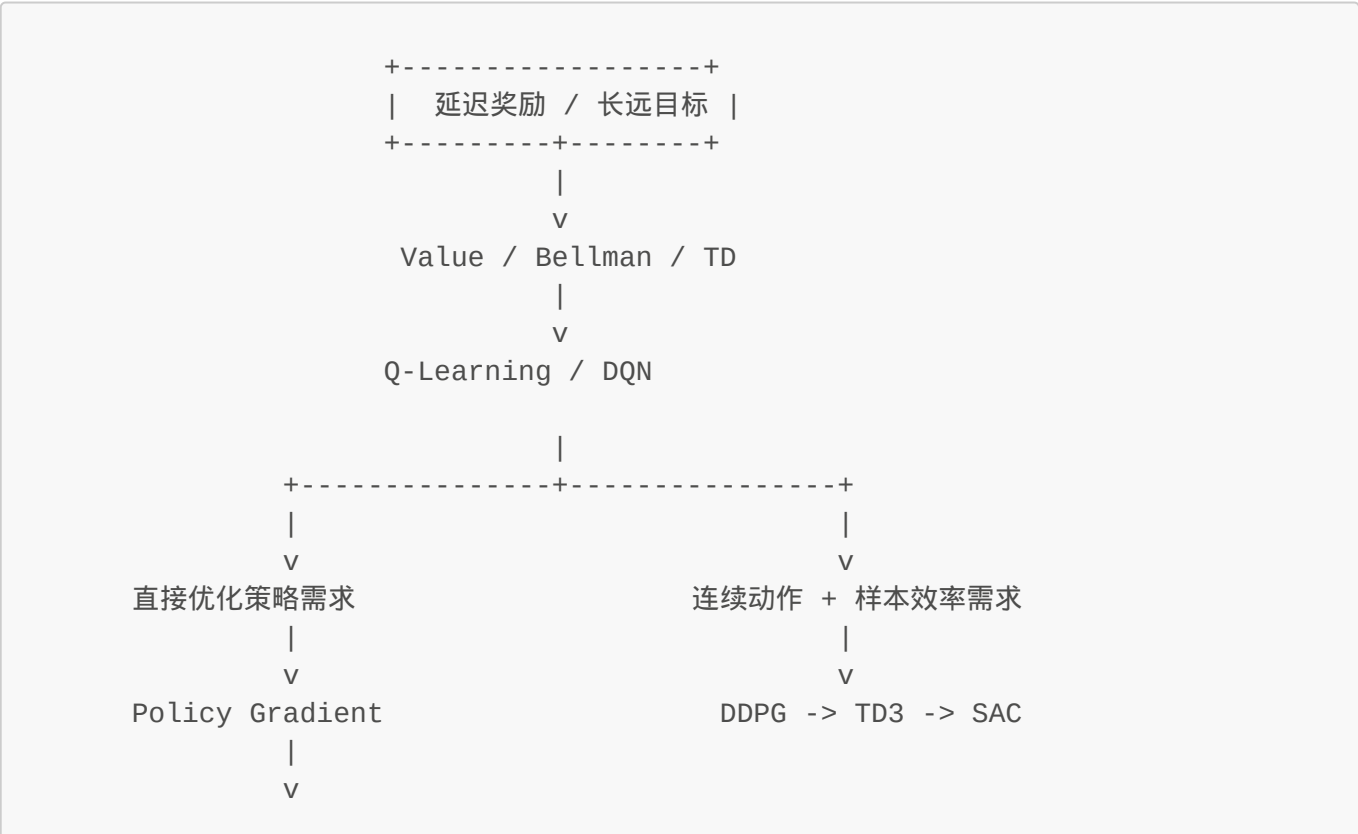
$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[A_t \nabla_{\theta} \log \pi_{\theta}(a_t|s_t)] \quad \text{\texttt{\text{ (策略更新方向) }}$$

五句话记住本章

- 1. Actor 负责行动，Critic 负责评价
- 2. Advantage = "这一步比平均好多少"
- 3. TD error 是 advantage 的便宜近似
- 4. 信号更局部了，方差更小
- 5. 代价是两个模块会互相影响

第八章：PPO - 为什么策略更新不能一次跨太大步？

全局导航图





你在这里：主干 -> PPO

问题驱动：为什么 Actor-Critic 还不够？

第七章里，Actor-Critic 已经跑通了"直接优化策略 + Critic 给反馈"的框架。理论上，只要 advantage 估计得准，策略就能稳步变好。

但实践中会出现一个诡异的现象：

训练曲线突然跳水——前一秒 reward 还在涨，下一秒骤降，然后要花很久爬回来。

这不是算法错了，而是**步子太大迈过了头**。

先把这一章的主线钉住：

problem -> starting point -> invention -> verification -> example

一句话压住核心：

PPO 要解决的问题不是"怎么学"，而是"别一次改太猛"。

1. Problem：为什么 Actor-Critic 还不够？

第七章里 Actor 的更新形式大致是：

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[A_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]$$

方向没错，但工程上会撞墙。

1.1 策略一变，数据分布跟着变

对比 supervised learning：

- 数据集固定 -> 模型更新不影响训练分布

RL 里完全不同：

- 策略一改 -> agent 行为变 -> 采样数据变 -> 状态分布变

结果：策略如果一步改太大，系统会跳到完全不同的行为区域。

1.2 "一步跨过头"的直接后果

某次 advantage 特别大时，Actor 可能：

- 把某个动作概率从 0.3 拉到 0.9
- 把另一个动作概率从 0.5 压到 0.1

短期看像是"快速进步"，实际可能是：

- 那批样本恰好 noisy（偶然因素）
- 策略分布瞬间偏得很离谱
- 下一轮采样质量崩掉

典型现象：前几轮 reward 往上走 -> 某次 update 后骤降 -> 花很多轮爬回来。

核心矛盾：Actor-Critic 解决了"信号太吵"，但没解决"步子太大迈不过河"。

2. Starting Point：可控更新 vs 更大胆更新

从第一性原理问，PPO 要解决的核心不是"如何改得更快"，而是：

如何让策略朝对的方向改，但不要一下改太离谱？

直觉很简单：

- Advantage 告诉我们"这类动作该强化还是削弱"
- 但它没告诉我们"一次该改多猛"

所以 PPO 不是在推翻 Policy Gradient，而是在它外面加一层更新控制机制：

方向 -> advantage 给出（不变）
步幅 -> 必须限制在安全范围内（新增）

3. Invention：PPO 是怎么被"发明"出来的？

一旦接受"更新方向对，但步子不能太大"，下一步自然会问：

怎么判断这次策略更新是不是走太猛了？

3.1 先比较"新策略"和"旧策略"对同一动作的态度变化

最自然的比较量是这个概率比值：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

它表示：

- 分子：新策略对这个动作的概率
- 分母：旧策略对这个动作的概率

所以：

- $r_t > 1$ ：新策略更偏向这个动作
- $r_t < 1$ ：新策略更不偏向这个动作
- $r_t = 1$ ：这个动作的态度没变

这个量非常关键，因为它直接把"更新是否太猛"转成了可测的相对变化。

3.2 如果 advantage 为正，我们希望概率上升；但不能无限上升

假设某动作的 $A_t > 0$ ，说明：

- 这个动作比平均好
- 应该更常做

那理想趋势是：

- r_t 增大一点

但如果它从 1.0 一下冲到 3.0 ，就意味着：

- 新策略把这个动作概率放大了 3 倍
- 这通常已经过猛了

同理，如果 $A_t < 0$ ，我们希望它概率下降；但也不能一脚踩死。

所以 PPO 的核心发明就是：

允许更新朝正确方向移动，但一旦移动幅度超出安全区，就把收益截住。

3.3 Clipped Objective：给目标函数装一个"限位器"

PPO 最经典的目标函数是：

```


$$L^{\text{CLIP}}(\theta) = \mathbb{E} \Big[ \min \Big( r_t(\theta), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \Big) \Big]$$


```

别被式子吓住，拆成人话就是：

- 正常情况下，让新策略按 advantage 的方向改
- 但如果概率比值 r_t 超过允许范围 $[1-\epsilon, 1+\epsilon]$
- 那就不要再继续奖励这种过大的改动

这里的 **clip** 像一个机械限位器：

- 太小了，截回来
- 太大了，也截回来

于是更新逻辑变成：

可以变好
但不要突然变得"过度自信"

3.4 为什么要取 **min**？

这是 PPO 里最精髓的一刀。

因为我们不是想"找到更大的目标值"，而是想：

当新策略试图因为 sample noise 而过度获利时，主动保守一点。

所以对同一个样本：

- 一个是原始目标 $r_t A_t$
- 一个是被 clip 过的保守目标
- 取两者更小的那个

这等于在说：

你可以进步，但别拿一次幸运样本把自己改疯了。

3.5 PPO 仍然站在 Actor-Critic 框架上

PPO 并没有抛弃第七章的框架。

它通常仍然包含：

- **Actor**：输出策略
- **Critic**：估计 value

- **Advantage**：作为策略更新权重

只不过它在 Actor 的目标函数上又补了一层安全护栏。

所以结构上可以理解成：

Actor-Critic 解决"怎么直接学策略 + 稳定给反馈"
PPO 解决"即使反馈稳定了，更新也别跨太大步"

4. Verification：为什么 PPO 是 inevitable outcome？

这一节不是验证"PPO 是什么"，而是验证**为什么必须这样设计**。

4.1 如果不用 clip 会怎样？

去掉 clip 的机制，退回到普通 policy gradient：

- noisy sample $\rightarrow$ advantage 估计偏大 $\rightarrow$ 策略一步改太猛 $\rightarrow$ reward 骤降

这就是第七章里看到的现象：**训练曲线突然跳水**。

所以"限制更新幅度"是必须的。但具体怎么限制？

4.2 如果用其他约束机制会怎样？

Option A：梯度裁剪 (gradient clipping)

- 问题：只限制梯度大小，不控制策略分布变化
- 结果：即使梯度小，策略分布也可能大幅偏移

Option B：KL 散度惩罚 (TRPO 的做法)

- 在目标函数里加 $KL(\pi_{old}, \pi_{new})$ 惩罚项
- 问题：需要二阶优化、计算开销大
- TRPO 本质上和 PPO 想解决同一个问题，但实现更复杂

Option C：直接限制参数更新步长 (step size)

- 问题：参数空间的变化不等于策略分布的变化
- 网络结构不同，同样的参数变化幅度效果完全不同

4.3 为什么 clip + min 是 inevitable？

一旦接受"必须限制新旧策略差异"这个前提，自然走向：

1. 用概率比值 r_t 衡量差异 $\rightarrow$ 直接可测、跟策略分布挂钩
2. 用 clip 截住过大变化 $\rightarrow$ 简单、不需要二阶优化
3. 用 min 取保守目标 $\rightarrow$ 防止 sample noise 导致过度自信

这三个选择不是"偶然发明"，而是：

- `r_t` 是衡量策略变化的最自然方式
- `clip` 是最简单的约束机制
- `min` 是最安全的保守策略

所以 PPO 不是"某个聪明人想出来的 trick", 而是在"限制更新幅度"这个需求下, 最简单有效的解法。

4.4 边界在哪里？

PPO 也不是完美：

- `epsilon` 太大 -> 约束失效
- `epsilon` 太小 -> 学习太慢
- `advantage` 估计不准 -> 方向本身就歪了, 限制再安全也没用

验证结论： PPO 通过概率比值 + clip, 把"大步更新的风险"变成了"小步改进的稳定性"。

5. Example：机器人走路时 PPO 如何避免"突然学崩"？

5.1 没有 clip 时会发生什么（具体场景）

假设双足机器人在某次 rollout 里：

- 偶然用一个特殊的发力节奏迈了一步
- 没摔倒, 还往前冲了 0.5 米
- `advantage` = +2.3 (比平均高很多)

普通 policy gradient 的反应：

这批样本太好了 -> 赶紧把这个动作概率拉高
旧策略： $P(\text{特殊发力}) = 0.15$
新策略： $P(\text{特殊发力}) = 0.78$

后果（下一轮 rollout）：

- 机器人开始频繁用这个发力模式
- 发现只在那种特定地形下有效
- 在其他场景直接摔倒
- `reward` 从 +12 骤降到 -3

5.2 PPO 怎么处理同样的情况？

同样的样本, `advantage` = +2.3：

PPO 的 clip 机制 (假设 `epsilon`=0.2)：
旧策略： $\pi_{\text{old}}(\text{特殊发力}) = 0.15$
新策略尝试： $\pi_{\text{new}}(\text{特殊发力}) = 0.78$
概率比值 $r_t = 0.78 / 0.15 = 5.2$

```
clip(r_t, 0.8, 1.2) -> 1.2 (被截住!)
```

结果：

- 新策略最多只能把概率拉到 $0.15 * 1.2 = 0.18$
- 而不是直接跳到 0.78

5.3 这个例子的可视理解

想象一条训练曲线：

Episode:	1	10	20	30	40	50	60	70
Reward (无 clip):	-5	+8	+12	+15	+3	-2	+5	+9
Reward (PPO):		-5	+6	+8	+9	+10	+11	+12

关键差异：

- 无 clip：第 40 步骤降（策略一步改太猛）
- PPO：稳步上升，没有跳水

这就是为什么连续控制任务里 PPO 特别好用——它让"站稳了再往前走"成为默认行为。

6. 这一章最后压成一张脑图

Problem:
Actor-Critic 虽然能学，但策略一步更新过大时容易突然崩掉

Starting Point:
我们需要的不是更猛更新，而是可控更新

Invention:
比较新旧策略对同一动作的概率比值 r_t
再用 clip 把过大的策略变化截住

Verification:
保留 advantage 指出的正确方向
同时限制激进更新，显著提升训练稳定性

Example:
机器人偶然找到一个好动作时，PPO 会适度强化，而不是一下把策略带偏

本章速记卡片

一句话主线

- Actor-Critic 解决了"怎么学策略 + 怎么给稳定反馈"
- PPO 继续解决"更新方向对了，但步子不能太大"
- 它用新旧策略概率比值衡量变化幅度
- 再用 `clip` 阻止激进更新

必背公式

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

$$L^{CLIP}(\theta) = \mathbb{E} \Big[\min \Big(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \Big) \Big]$$

必背术语

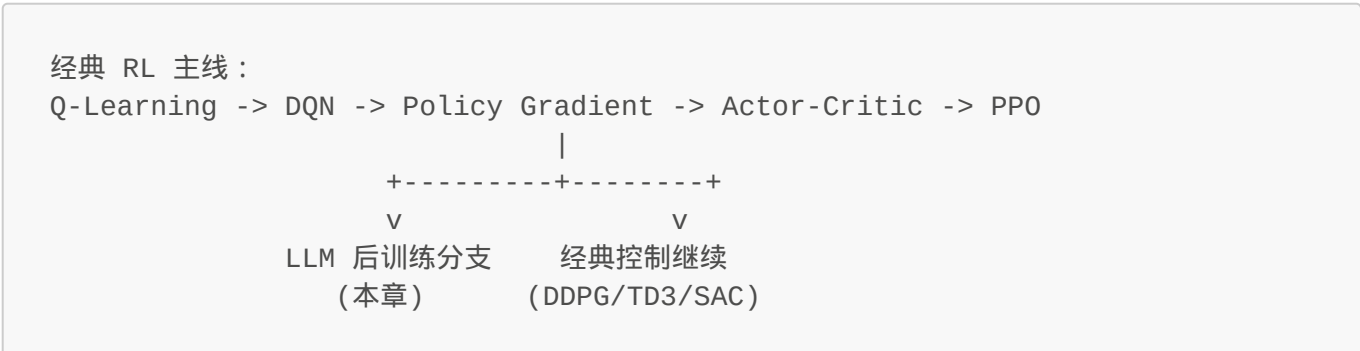
- Old Policy
- Probability Ratio
- Clip
- Surrogate Objective
- PPO

最短背诵版

1. PPO 还是在学策略
2. 但它会比较新旧策略差了多少
3. 如果差太大，就把更新收益截住
4. 目标是防止一步改太猛
5. 所以 PPO 的关键词是"稳定的小步改进"

第九章：LLM 分支入口——为什么 GRPO 应该在 PPO 之后讲？

你在这里



问题：GRPO 该放在哪里讲？

如果你现在学完了 PPO，脑子里可能有这几个疑问：

- "接下来是不是该讲 GRPO 了？"
- "GRPO 和 PPO 是什么关系？"
- "它像 DQN -> TD3 / SAC 那样是主线的下一步吗？"

先把位置说死：**GRPO 最合适放在 PPO 之后，但它不是经典控制主线的下一站，而是 LLM 后训练分支的特化算法。**

原因很简单：

- **PPO** 解决的是：策略更新别太猛
- **GRPO** 继承的正是这条思路
- 但它的服务场景是 **LLM 对齐 / reasoning post-training**，不是通用控制任务

所以它在教程里的位置应该是：

```
Policy Gradient -> Actor-Critic -> PPO
                        |
                        v
                LLM 分支：RLHF / GRPO
```

一句话先压住：**GRPO 不该插在 PPO 前面，也不该硬塞进经典机器人控制主线。**

1. Problem：为什么不能把 GRPO 和 DQN / PPO 并排当成同一级主线算法？

1. Problem：为什么不能把 GRPO 和 DQN / PPO 并排当成同一级主线算法？

因为它们解决的问题层级不一样。

1.1 经典 RL 主线关心的是"如何学会行动"

前面这些章节主要在解决：

- 怎么定义 value
- 怎么从采样中更新策略
- 怎么降低方差
- 怎么稳定策略更新
- 怎么处理连续动作

这些问题是 **通用 RL** 问题。

不管你是在：

- 走迷宫
- 控机器人
- 玩 Atari

都会遇到。

1.2 GRPO 关心的是"LLM 场景下，怎么更便宜地做相对策略优化"

GRPO 的语境明显更窄：

- 一个 prompt 往往会采样多条回答
- 奖励很多时候是相对比较出来的，而不是环境一步一步给的 dense reward
- 训练对象是 language model，不是传统 control policy
- 我们常常更关心生成质量排序、group-relative 信号、以及减少额外 value model 成本

所以从"问题被什么逼出来"这个角度看，GRPO 并不是对前面所有 RL 任务都自然适用的下一站，而是：

在 PPO 已经成立之后，LLM 训练场景又提出了新要求，于是长出来的专用分支。

2. Starting Point：如果 PPO 已经能做策略优化，LLM 场景还缺什么？

从第一性原理问：

LLM 后训练里的反馈，和机器人控制里的反馈，到底有什么不同？

差别非常大。

2.1 LLM 常常不是"每一步都有环境奖励"

在机器人里，你可以有：

- 每走一步的 reward
- 摔倒的惩罚
- 能耗惩罚

但在 LLM 里，很多时候更像这样：

- 给一个 prompt
- 采样几条完整回答
- 用 reward model 或规则给整条回答打分
- 再比较这些回答谁更好

也就是说，反馈更像：

- **序列级 / 回答级**
- **相对排序式**
- 而不是传统 control 里的逐步环境回报

2.2 如果直接照搬 PPO，value / critic 这一套在 LLM 里成本很高

PPO 在很多实现里会搭配 value function / critic：

- 用来估 advantage
- 用来降低方差

但放到大语言模型后训练里，代价会很重：

- 训练更复杂
- 需要额外 value head 或 value model
- 稳定性和工程成本都上去

于是 LLM 场景会自然提出一个新问题：

能不能保留 PPO 这种"限制策略别一步改太猛"的优点，同时又更贴合"多答案比较"这种反馈形式，并尽量减少 critic 负担？

这就是 GRPO 的出发点。

3. Invention：GRPO 是怎么从 PPO 的语境里长出来的？

先说本质，不先堆细节：

GRPO 可以看成是：把 PPO 的"保守策略更新"思想，和 LLM 场景里的"组内相对奖励"结合起来。

3.1 从"单条样本值多少钱"转向"同组里谁相对更好"

在 LLM 场景里，一个很自然的训练单元是：

- 同一个 prompt
- 采样出一组 responses
- 对这组 responses 打分或比较

这时候最有信息量的问题常常不是：

这条回答的绝对价值是多少？

而是：

在同一个 prompt 的这组回答里，哪条更好？好多少？

这就天然引出"group-relative"信号。

3.2 Advantage 的来源不再主要靠 critic，而更像组内相对基线

PPO / Actor-Critic 里，Advantage 常常来自：

- value estimate
- GAE
- TD-style baseline

但在 GRPO 的语境里，更自然的是：

- 同一个 prompt 下采多条样本
- 比较它们的 reward
- 用组内均值、相对排名、标准化分数之类的方式形成相对优势信号

也就是说，它把"这条样本比 baseline 好多少"这个问题，更多地交给：

- **组内相对比较**

而不是：

- 单独训练一个 critic 去估每个 token / 序列的 value

3.3 PPO 的"别走太猛"思想仍然保留

这点很关键。

如果没有 PPO 那条主线，你很难理解 GRPO 的核心节制逻辑。

GRPO 并不是说：

- 我们不要保守更新了

而是说：

- **仍然需要限制新旧策略差异，避免一步把模型带偏**
- 只不过 advantage / reward 这部分，更偏向组内相对化构造

所以它和 PPO 的关系更像：

```
PPO: 保守策略更新 + advantage
GRPO: 保守策略更新 + group-relative advantage
```

这就是为什么它一定要建立在你已经理解 PPO 之后再讲。

4. Verification：为什么说"PPO 之后讲 GRPO"比"把 GRPO 提前"更合理？

4.1 不先讲 PPO，你很难解释 GRPO 到底继承了什么

如果学生还没理解：

- 为什么 policy update 会过猛
- 为什么要限制新旧策略差异
- surrogate objective 在干什么

那你一上来讲 GRPO，很容易变成：

- 记一个名字
- 背一个实现

- 知道它"好像是 RLHF / DeepSeek 在用的"

但不知道它到底是从什么矛盾里长出来的。

4.2 不先切到 LLM 场景，你也很难解释它为什么不是通用主线算法

如果不先告诉读者：

- 经典控制任务和 LLM 后训练的反馈结构不一样

那 GRPO 就会看起来像：

- "PPO 的升级版"
- 或者"下一个更高级的标准算法"

这其实会误导。

更准确的理解是：

GRPO 不是对 PPO 的普遍替代，而是 PPO 思想在 LLM 组相对奖励场景下的一种特化实现。

4.3 所以最自然的教学顺序是什么？

最顺的顺序是：

```
Policy Gradient
-> Actor-Critic
-> PPO
-> RLHF / LLM post-training 背景
-> GRPO
```

这样每一步都在回答：

- 前一步留下了什么问题
- 新场景又提出了什么限制
- 新方法为什么非长出来不可

这条因果线就会非常顺。

5. Example：把 PPO 和 GRPO 放在 LLM 场景里对比看

假设现在有一个数学推理 prompt：

题目：求一个方程的解

模型针对同一个 prompt 采样出 4 个回答：

- 回答 A：过程完整，答案对

- 回答 B：答案对，但过程乱
- 回答 C：过程像样，但算错
- 回答 D：胡说八道

5.1 如果按经典 PPO 视角看

你会想：

- 每条回答有一个 reward
- 再结合 value / advantage 去做保守策略更新

这当然能做，但在 LLM 场景里会显得比较重。

5.2 如果按 GRPO 视角看

你会更自然地想：

- 这 4 条是同组 samples
- 我更关心它们的相对好坏
- A 明显优于 D，B 略优于 C
- 所以更新时应该让模型更偏向 A/B 这类回答风格，远离 C/D

也就是说，信号的构造方式会更像：

同 prompt 下，多样本组内比较
-> 得到相对优势
-> 再做保守策略更新

这就是它为什么特别适合放在 LLM post-training 那一支里。

6. 教程结构上，最推荐怎么放？

如果你的教程目标是"先打通通用 RL 主线，再进入 LLM RL"，那我推荐这样排：

第 1-8 章：经典 RL 主线

1. RL 基本元素
2. Policy
3. Value / Bellman
4. Q-Learning
5. DQN
6. Policy Gradient
7. Actor-Critic
8. PPO

第 9 章：从经典 RL 到 LLM post-training

- 为什么 LLM 的反馈结构不同
- RLHF 在解决什么问题
- 为什么 PPO 会先被拿来 LLM 对齐

第 10 章：GRPO

- 为什么在 LLM 场景下，需要 group-relative 信号
- 它和 PPO 的继承关系是什么
- 它解决了什么工程 / 稳定性 / 成本问题

如果你暂时不想单开一章讲 RLHF，也可以把它收成：

第九章：LLM 强化学习导论（含 PPO -> GRPO 过渡）

但无论如何，**不要把 GRPO 放到 PPO 前面，也不要放到 DDPG / TD3 / SAC 那条连续控制支线中间。**

7. 这一章最后压成一张脑图

Problem:

GRPO 不是通用 RL 主线算法，直接塞进经典控制路径会让结构混乱

Starting Point:

LLM post-training 的反馈更像组内相对比较，而不是传统环境逐步 reward

Invention:

保留 PPO 的保守更新思想

把 advantage 更多建立在 group-relative reward / baseline 上

Verification:

只有先懂 PPO，再切到 LLM 场景，GRPO 的位置和意义才会自然

Example:

同一个 prompt 下采多条回答，按组内相对质量更新策略

本章速记卡片

一句话主线

- GRPO 最适合放在 PPO 后面讲
- 但它属于 LLM post-training 分支，不是经典控制主线必经站
- 它继承了 PPO 的保守更新思想
- 同时把优势信号更多建立在组内相对比较上

必背判断

- PPO：通用的保守策略优化主线方法
- GRPO：PPO 思想在 LLM 组相对奖励场景下的特化分支
- 不要 放在 PPO 前面
- 不要 硬塞进经典连续控制支线

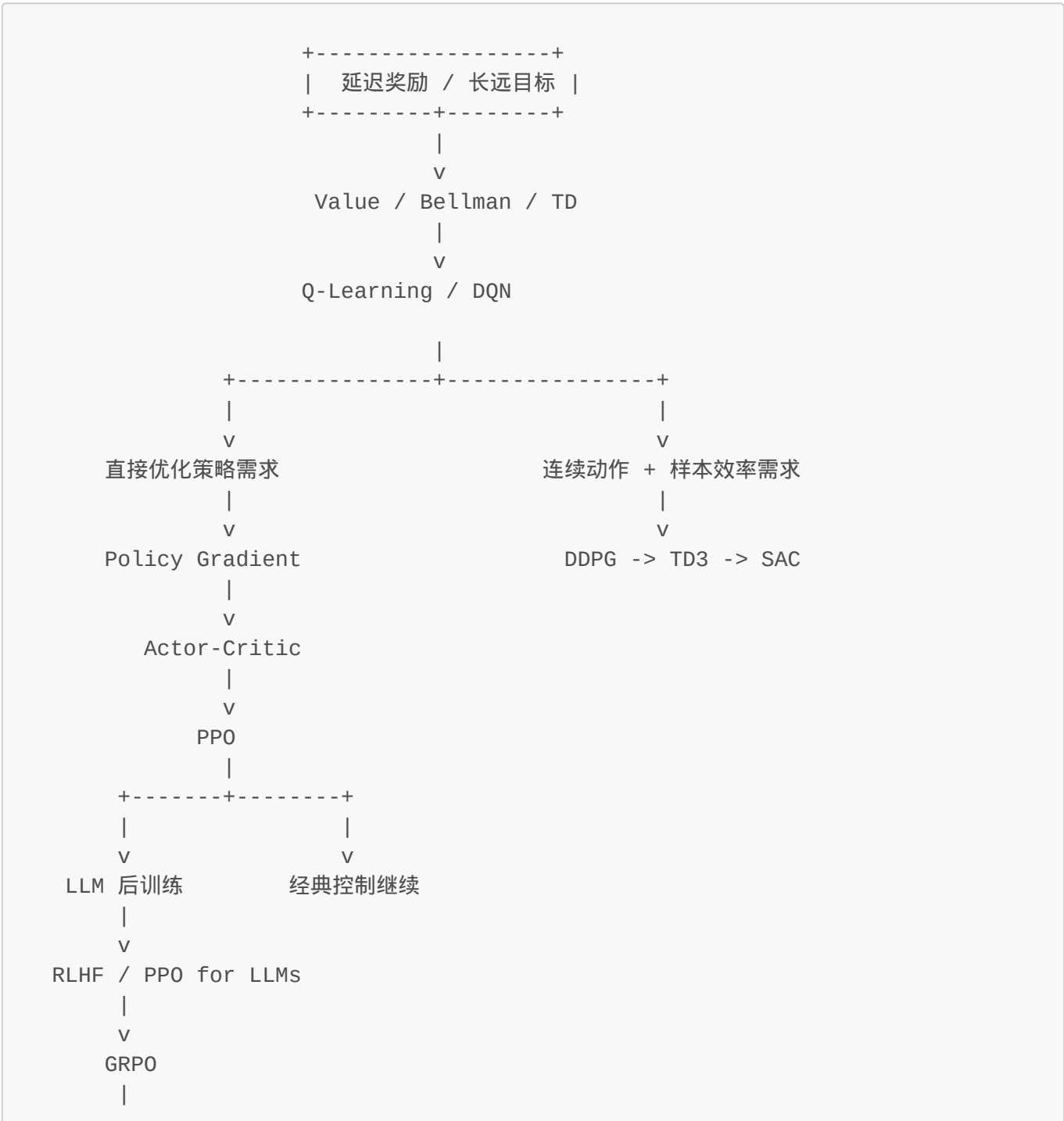
最短背诵版

- 1. 先学 PPO
- 2. 再切到 LLM post-training
- 3. 然后讲 GRPO
- 4. 因为 GRPO 依赖 PPO 的保守更新思想
- 5. 但它的真正语境是语言模型组相对奖励

第十章：预训练目标与人类偏好错位——RLHF 的必然性

位置导航：LLM 分支 -> RLHF 起点

全局导航图



V
Outcome -> Process -> Verifiable Reward

你在这里：LLM 分支 -> RLHF 起点

序：预训练教会模型“像互联网说话”，但没教会它“按人类偏好回答”

从经典 RL 切到 LLM，不要一上来就背 RLHF 缩写。先抓住真正把这条分支逼出来的问题。

前面几章里，我们一直在处理这样的主线：

策略怎么表示？
价值怎么评估？
更新怎么稳定？

到了语言模型，这些问题还在，但任务表面变了：

- 输入不再是机器人状态，而是 prompt
- 动作不再是转向或扭矩，而是下一个 token
- 策略不再是控制器，而是 language model 的条件分布

可真正把新分支逼出来的，不是“模型结构变了”，而是：

预训练目标和人真正想要的回答目标，并不完全一致。

先把这一章的主线钉住：

problem -> starting point -> invention -> verification -> example

一句话先压住：

RLHF 的本质，不是让 LLM 临时学点奖励技巧，而是解决“最大化下一个 token 似然”与“输出人真正偏好的回答”之间的目标错位。

先约定一下这一支里的术语，避免后面名字打架：

- **policy** = 策略 = language model 的条件分布
- **reward model / preference model**：本教程默认把它们近似看成同一类东西，都是“把人类偏好压缩成可计算分数的模型”；后文优先统一写 **reward model**
- **critic / value function**：都在回答“当前状态或轨迹值多少”，如果强调函数对象就写 **value function**，如果强调在 Actor-Critic 里的角色就写 **Critic**

1. Problem：为什么一个会续写文本的模型，还不等于一个“对齐的人类助手”？

先看预训练到底在做什么。

语言模型预训练的核心目标，大致是：

$$\max_{\theta} \mathbb{E}[\log p_{\theta}(x_t | x_{<t})]$$

也就是：

- 给定前文
- 预测下一个 token
- 在海量语料上把这个条件分布拟合好

这个目标非常强大，因为它让模型学会了：

- 语法
- 常识模式
- 世界知识的统计压缩
- 推理痕迹的模仿能力

但它天然不是“人类助手目标”。

1.1 预训练学到的是“像数据分布那样说话”，不是“像用户想要的那样回答”

互联网文本里同时包含：

- 正确答案
- 废话
- 冲突观点
- 有害内容
- 阴阳怪气
- 模棱两可的表达

如果模型只是忠实拟合语料分布，那它学到的是：

在统计上像互联网，而不是在目标上像一个可靠助手。

1.2 很多真正重要的标准，不容易写成 next-token loss

用户真正关心的，常常是这些：

- 有帮助吗？
- 真实吗？
- 安全吗？
- 啰嗦吗？
- 推理过程稳不稳？
- 有没有真的回答问题，而不是只写得像？

这些标准的共同问题是：

- 它们通常是 **回答级别** 的属性
- 很难分解成每个 token 的局部监督

也就是说，经典 cross-entropy 目标缺少一个关键能力：

它不擅长表达“整条回答在人类看来到底好不好”。

1.3 于是出现目标错位

到这里，矛盾已经很清楚了：

预训练目标：更像训练语料中的下一 token 分布
真实助手目标：给出更有帮助、更安全、更符合偏好的完整回答

所以问题不是“预训练没用”，而是：

预训练解决了语言建模问题，但没完全解决偏好对齐问题。

2. Starting Point：如果人类偏好很难直接写成标签，那我们至少可以比较“哪个回答更好”

从第一性原理问：

如果“好回答”难以逐 token 精确定义，那人类还能提供什么可靠信号？

最自然的答案不是绝对分数，而是比较。

2.1 人类很难给出完美标尺，但通常能比较两个回答谁更好

比如同一个 prompt 下，给出两个回答：

- 回答 A：准确、简洁、有结论
- 回答 B：绕圈、模糊、漏了重点

很多时候，人类不容易说：

- A 的价值是 8.7 分
- B 的价值是 4.2 分

但很容易说：

- **A 比 B 更好**

这个观察非常关键，因为它把监督形式从：

- “给每个 token 打标签”

改成了：

- “对完整回答做相对偏好比较”

2.2 这让我们有机会构造一个“偏好模型”

如果我们收集很多人类比较：

- 对同一 prompt 的多个回答排序

- 或至少标出 winner / loser

那就能训练一个模型去近似回答：

给定 prompt 和回答，这个回答在人类看来有多值得偏好？

这就是 reward model / preference model 的出发点。

2.3 但有了偏好分数，还缺最后一步：怎么让策略真正朝偏好方向更新？

到这里我们只是多了一个打分器。

可真正想要的是：

- 让 language model 本身更倾向输出高偏好回答
- 而不是只在训练后拿个评分器旁观

于是前面几章学过的 RL 主线就重新接回来了：

有了“回答级偏好信号”

-> 就可以把 LLM 看成策略

-> 再用 policy optimization 去推动它更偏向高奖励回答

这就是 RLHF 的起点。

3. Invention：RLHF 是怎么从“偏好比较”自然长出来的？

RLHF 可以拆成三层看，不然很容易变成黑箱缩写。

3.1 第一步：先让模型学会“按指令回答”

通常会先做 **SFT** (Supervised Fine-Tuning)：

- 用高质量 instruction-response 数据
- 让模型先进入“助手模式”

这一步不是最终对齐，但它很重要，因为它给策略一个合理初始化：

- 不至于从纯预训练分布直接开始乱飘

可以把它理解成：

预训练：学会语言 and 知识压缩

SFT：学会进入“被提问 -> 给回答”的助手接口

3.2 第二步：收集人类偏好，训练 reward model

然后对同一个 prompt 采样多个回答，让人类比较：

- 哪个更有帮助
- 哪个更准确
- 哪个更安全

再训练一个 reward model，去拟合这种偏好排序。

于是原本模糊的“人类偏好”被压缩成一个可计算信号：

```
prompt + answer -> reward score
```

3.3 第三步：把 LLM 当成策略，用 RL 方法推动它追高奖励

这一步才是 **RLHF** 里的 **RL**。

此时：

- **状态** 可以理解为 prompt + 已生成前缀
- **动作** 是下一个 token
- **策略** 是 `p_theta(token | context)`
- **奖励** 来自 reward model 或偏好信号

于是 language model 也可以被放进 RL 语言里：

```
LLM = 一个按上下文输出 token 分布的策略
```

然后就能问：

如何更新这个策略，让它更倾向生成高 reward 的完整回答？

这时候，**Policy Gradient -> Actor-Critic -> PPO** 那条主线就重新变得有用。

3.4 为什么实践里经常是 PPO 先接上来？

因为 **PPO** 解决了一个非常关键的问题：

既然 reward model 可能 noisy，策略更新就不能太猛。

如果没有约束，模型可能为了刷 reward model：

- 语言风格变形
- 出现 reward hacking
- 分布快速漂移
- 失去原本的可读性和泛化

所以 PPO 在 LLM 场景里受欢迎，不是偶然，而是因为它刚好提供了：

- advantage-style 更新方向
- 对新旧策略差异的保守控制

这就是为什么很多人会先讲：

- RLHF -> PPO for LLMs

再往后才自然长出 GRPO 这类更贴近组相对奖励的变体。

4. Verification：怎么确认“RLHF 是必要的一层”，而不只是训练花活？

4.1 它有没有解决预训练目标错位？

有，至少方向上是对的。

预训练只会说：

- 什么文本在统计上更像数据

RLHF 则额外引入：

- 什么回答在人类偏好上更好

也就是说，它把“像数据”之外的标准显式带进了优化目标。

4.2 它是不是比纯监督微调更能表达回答级偏好？

通常是。

因为很多偏好不是唯一标准答案，而是：

- 更清楚
- 更稳妥
- 更少废话
- 更符合帮助性

这些更适合通过：

- 排序
- 比较
- 回答级 reward

来表达，而不是只靠 token-level imitation。

4.3 它有没有代价？

当然有。

RLHF 不是魔法，它会引入新的问题：

- reward model 会有偏差
- 可能被 exploit
- 训练更复杂
- 计算成本更高
- alignment 不等于真实性，模型可能更会“像对齐”，不一定更真懂

所以验证结论应该说得准确一点：

RLHF 不是完美终点，但它是把“人类偏好”显式接进训练目标的一条自然路径。

4.4 为什么这一步对理解后面的 GRPO 很关键？

因为如果你不先理解：

- LLM 的奖励很多时候是回答级的
- 人类信号常常来自比较而非 dense reward
- PPO 在这里承担的是“保守优化”角色

那后面讲 **GRPO** 时，你就会不知道它到底在继承什么、又在替代什么。

所以顺序必须是：

```
预训练 / SFT
-> 偏好比较与 reward model
-> RLHF / PPO for LLMs
-> GRPO
```

5. Example：同一个问题，为什么“像语料”不等于“像好助手”？

假设用户问：

如何安全地开始学习强化学习？

模型可能生成两种回答。

回答 A

- 结构清楚
- 先讲学习路径
- 提醒先做 toy environment
- 不乱堆术语
- 给出明确的下一步实践建议

回答 B

- 词很多
- 堆一堆 buzzwords
- 夹杂不必要的行话
- 没有学习顺序
- 看起来“像懂”，但并不真正帮用户前进

从纯 next-token 分布看：

- B 可能也很像训练语料里常见的技术写作风格

但从用户偏好看：

- A 明显更好

这就说明：

```
likelihood 高
!=
helpfulness 高
```

而 RLHF 试图做的，就是把这种“回答级别的好坏差异”变成训练信号。

6. 这一章最后压成一张脑图

```
Problem:
预训练优化的是 next-token likelihood
但用户真正关心的是回答级别的帮助性、真实性、安全性和偏好匹配

Starting Point:
人类很难逐 token 打标准答案
但通常能比较两个完整回答谁更好

Invention:
先做 SFT 进入助手模式
再训练 reward model 拟合人类偏好
最后把 LLM 当策略，用 RL 方法推动它偏向高奖励回答

Verification:
RLHF 没有消灭所有问题
但它把“人类偏好”从旁观标准变成了训练目标的一部分

Example:
一个“像语料”的回答，不一定是一个真正“像好助手”的回答
```

章节回顾卡

一句话主线

- 预训练教会模型续写，不等于教会模型按人类偏好回答
- 人类偏好更适合用回答级比较来表达
- RLHF 的核心是把这些偏好转成 reward，再用 RL 推动策略更新
- PPO 常被接到这里，是因为它能保守地优化策略

必背术语

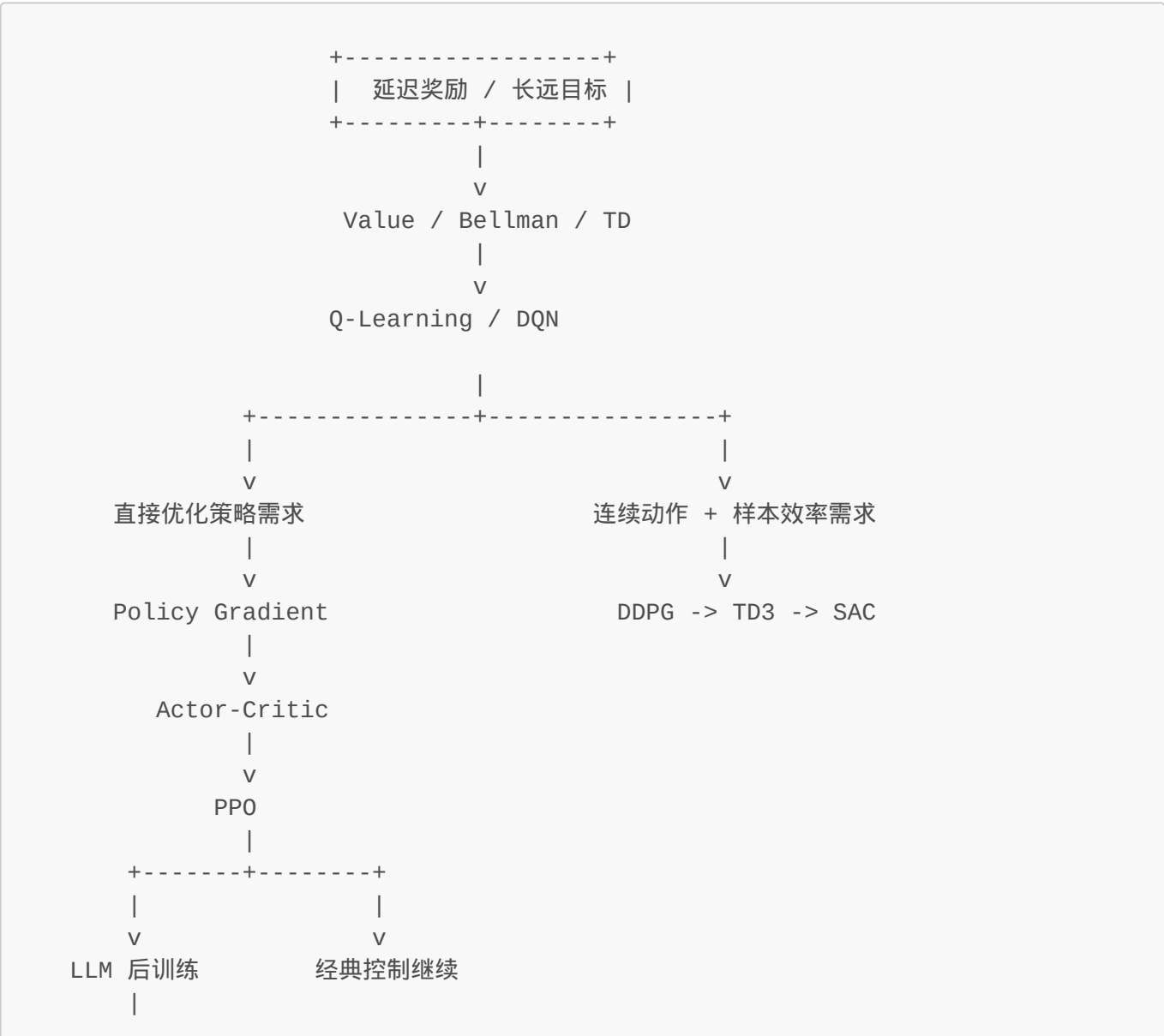
- Pretraining
- SFT
- Preference Data
- Reward Model
- RLHF
- PPO for LLMs

最短背诵版

1. 预训练只保证“像数据”
2. 但助手需要“像人类偏好”
3. 人类更容易比较回答，而不是给逐 token 标签
4. 所以先学偏好，再用 RL 推策略
5. 这就走到了 RLHF

第十一章：PPO for LLMs - 当动作变成 token，PPO 还在优化什么？

全局导航图



```

      V
RLHF / PPO for LLMs
      |
      V
    GRPO
      |
      V
Outcome -> Process -> Verifiable Reward

```

你在这里：LLM 分支 -> PPO for LLMs

序：PPO 进了语言模型，不是换个名字，而是换了"状态、动作、奖励"的语义

到了 LLM 分支，很多人会有一个错觉：

PPO for LLMs 不就是把 PPO 原封不动搬过来吗？

不对。核心思想没变，但语义全变了。

在经典控制里：

- 状态可能是机器人姿态
- 动作可能是关节扭矩
- 奖励可能来自环境反馈

在 LLM 里：

- 状态是 **prompt + 已生成前缀**
- 动作是"下一个 token 选什么"
- 一整段回答生成完以后，才更容易拿到回答级 reward

先把这一章的主线钉住：

```
problem -> starting point -> invention -> verification -> example
```

一句话先压住：

PPO for LLMs 的本质，是把语言模型当作 token-level policy，在 reward model 的指导下做保守更新，同时用 KL 约束防止模型偏离原本可用语言分布太远。

1. Problem：为什么 SFT 之后还会继续逼出 PPO for LLMs？

SFT 已经让模型进入了"像助手一样回答"的接口，但它仍然有三个明显边界。

1.1 SFT 学的是 imitation，不是 preference optimization

在 SFT 里，模型主要学的是：

- 给定 prompt

- 模仿高质量 answer 的 token 序列

这当然有用，但它本质还是：

- **模仿已有示范**

而不是：

- **主动优化回答级偏好目标**

1.2 很多偏好不是唯一标准答案

比如：

- 更简洁
- 更稳妥
- 更 helpful
- 更不容易胡说

这些往往不是"只有一个 gold answer"，而是：

- 多个回答都可以
- 但质量高低不同

这种信号更像 reward，而不像标准 supervised label。

1.3 如果直接只追 reward，又会把模型带偏

一旦有了 reward model，你会自然想：

- 那就让模型猛追高 reward 不就行了？

问题是 reward model 本身并不完美：

- 可能 noisy
- 可能有 shortcut
- 可能被 exploit

如果策略更新太猛，模型就会：

- 套路化迎合 reward model
- 语言分布变形
- 丢掉原本预训练 + SFT 积累的通用能力

所以问题可以压成一句：

我们既想让 LLM 朝高偏好回答移动，又不能让它为了刷 reward 而一步走偏。

2. Starting Point：把 LLM 重新翻译成 RL 语言

想让 PPO 接上来，第一步不是写公式，而是完成语义映射。

2.1 状态是什么？

在第 t 个生成位置，状态可以理解成：

```
s_t = prompt + 已生成 token 前缀
```

也就是说，当前上下文本身就是 state。

2.2 动作是什么？

动作是：

```
a_t = 选择下一个 token
```

这和机器人控制最大的不同在于：

- 动作空间极大（整个词表）
- 一次回答是很多步 token action 串起来的轨迹

2.3 策略是什么？

策略就是语言模型的条件分布：

$$\pi_{\theta}(a_t | s_t) = p_{\theta}(\text{token}_t \mid \text{prompt}, \text{token}_{\{<t\}})$$

这一步一旦看清，前面的 RL 主线就能重新接上：

LLM 不是例外
它只是一个动作空间极大、轨迹很长、奖励更偏回答级的策略模型

2.4 那奖励从哪来？

在 LLM 场景里，奖励通常不是每个 token 都有环境反馈，而更像：

- 整个回答生成完以后
- 由 reward model 给分
- 再可能结合规则奖励 / 安全惩罚 / 格式奖励

所以它更像序列级 reward，再反向分配到 token 轨迹上。

3. Invention：为什么 PPO 的"保守更新+KL 约束"是这个问题的唯一合理解？

3.1 Axioms 层次：这个问题的不可绕过性是什么？

先把最底层的 axioms 钉死：

Axiom 1 (SFT 的本质)： SFT 只能优化 token-level likelihood，无法直接优化回答级偏好
Axiom 2 (Reward Model 的不完美)： RM 有 noise、可能被 exploit、可能和真实人类偏好有 gap
Axiom 3 (语言分布的脆弱性)： LLM 的语言能力是预训练 + SFT 累积的结果，剧烈更新会破坏通用能力

如果只接受这三个 axioms，那么问题就变成：

如何在 RM 指导下优化偏好，同时不破坏模型原有的语言能力？

这不是"要不要加 KL"的问题，而是**KL 约束是必然的解**——因为如果允许策略自由跑向高 reward，最优策略就是"找到 RM 的捷径并疯狂刷分"。

3.2 Starting Point：我们手上有什么工具？

- **PPO** 本身有 clipped surrogate objective —— 这已经是一种保守更新
- **Reference model** π_{ref} 可以冻结在 SFT checkpoint，作为"语言能力锚点"
- **KL penalty** 可以作为距离度量： $D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$

3.3 Invention：为什么 PPO + KL 是 inevitable outcome？

Step 1: 为什么要用 PPO 而不是 PG？

- PG 一步更新太猛，容易 overshoot
- PPO 的 clip 机制天然限制单步更新幅度 → **这是保守性的第一层**

Step 2: 为什么还需要 KL penalty？

- PPO 只限制"当前 update step 不要太大"
- 但不限制"最终策略可以离 π_{ref} 多远"
- KL penalty 是**对整体分布距离的约束**——这是第二层护栏

压缩机制的本质：

$$KL(\pi_{\theta} \parallel \pi_{\text{ref}}) = E_{\{x \sim \pi_{\theta}\}}[\log \pi_{\theta}(x) - \log \pi_{\text{ref}}(x)]$$

如果 KL 很小 → π_{θ} 和 π_{ref} 在大部分 token 上决策一致
 → 模型不会为了刷 reward 而发明怪异语言模式

Step 3: 为什么是"参考分布"而不是别的东西？

- SFT checkpoint 代表了"人类示范 + 通用语言能力"的平衡点
- RM 只代表偏好信号，可能 noisy / biased
- KL constraint 的本质是：**让 RM 指导优化方向，但不让它完全接管策略**

3.4 Verification：能否独立重推导这个设计？

试着遮住答案，自己推一遍：

1. **问题是什么？** — SFT 只能 imitation，RM 能指导偏好但可能 noisy

2. 如果只用 RM + PG 会发生什么？ — 模型会 exploit RM，语言分布崩塌
3. 需要什么机制来防跑偏？ — 保守更新 + 距离约束
4. PPO 的 clip 够不够？ — 只限制单步，不限制整体分布
5. 还需要什么？ — KL penalty 对参考分布的距离度量
6. 参考分布选哪里？ — SFT checkpoint，因为它有语言能力的积累

如果你能自己推到"KL constraint is necessary"这个结论，那才真正理解了 PPO for LLMs。

3.5 Example：同一个 prompt 下发生了什么？

假设 prompt 是：

请解释 Bellman 方程，并给一个最小例子。

SFT 阶段：

- 模型学的是"模仿高质量回答的 token 序列"
- 优化目标：maximize likelihood of gold answers

PPO for LLMs 阶段：

- 模型采样出多条回答
- RM 评分：结构清晰的 > 混乱堆术语的
- PPO update：让高分回答的 token 轨迹更容易出现
- KL constraint：但不能为了刷分而发明怪异措辞

对比：

SFT: "这条 gold answer 的每个 token，我都要学会"

PPO for LLMs: "这类完整回答整体更符合偏好 -> 让生成这类回答的概率提升"
+ "但不要离 SFT checkpoint 太远 -> KL constraint"

3.6 一句话压缩 (axioms version)

Problem: SFT 只能 imitation, RM 可能 noisy, 直接追 reward 会破坏语言能力
Axioms: (1) token-level likelihood $\neq$ preference optimization, (2) RM imperfect, (3) language distribution fragile
Inevitable Outcome: PPO's conservative update + KL penalty against reference distribution
Compression Mechanism: KL distance as a measure of "how far from SFT capabilities"

4. Verification：独立重推导——遮住答案，你能重建这个设计吗？

4.1 从 axioms 出发能推导出什么？

已知：

- Axiom 1: SFT 只能优化 token-level likelihood → 无法直接优化回答级偏好
- Axiom 2: RM 不完美 (noisy, exploitable)
- Axiom 3: LLM 的语言能力是脆弱的，剧烈更新会破坏

问题：如何让模型朝高偏好方向移动，同时不破坏语言能力？

4.2 Step-by-step 推导（试着遮住答案自己推）

Q1: SFT 够不够？

- **不够。** 因为 SFT 只能 imitation，不能直接优化"回答级偏好"

Q2: 那直接用 RM + Policy Gradient 行不行？

- **不行。** 因为 RM 不完美，模型会 exploit RM → 怪异措辞、刷分、语言崩塌

Q3: 需要什么机制来防跑偏？

- **保守更新** — PPO 的 clip 限制单步幅度（第一层护栏）
- **分布距离约束** — KL penalty 限制整体偏离程度（第二层护栏）

Q4: KL constraint 为什么是"对参考分布的距离"而不是别的？

- 因为 SFT checkpoint 代表了"人类示范 + 语言能力"的平衡点
- RM 只指导方向，但不能让它完全接管策略
- **KL distance = "离原始语言能力有多远"**

Q5: PPO for LLMs 的本质是什么？

Reward Model 说：这个方向更好
Reference Model 说：别跑太远
PPO 负责：在两者之间找到保守的更新路径

4.3 Verification Test：如果你能回答这三个问题，就真懂了

1. **如果去掉 KL penalty 会发生什么？** — 模型会 exploit RM，语言分布崩塌
2. **为什么 PPO 的 clip 还不够？** — 只限制单步 update，不限制最终策略距离
3. **Reference model 选哪里最合适？为什么？** — SFT checkpoint，因为它有语言能力积累

4.4 它是不是最终答案？

不是。

PPO for LLMs 仍然有很多成本：

- rollout 很贵 (采样完整回答)

- reward model 训练很贵
- value / critic head 也可能难训
- KL 系数和 reward scale 很敏感

这也是为什么后面会继续逼出更贴近 LLM 场景的变体，比如 **GRPO**。

关键 insight：PPO for LLMs 是"必要的妥协"——在 RM 指导和语言能力保护之间找平衡点。但这个设计本身也会暴露新的问题（成本、工程复杂度），从而继续推动新算法诞生 🌟

5. Example：同一个 prompt，SFT vs PPO for LLMs 到底在优化什么？

假设 prompt 是：

请解释 Bellman 方程，并给一个最小例子。

SFT 阶段发生了什么？

输入：(prompt, gold_answer) pairs

目标：maximize likelihood of gold answers

模型学到的是："这些 token 序列应该这样排列"

本质：Imitation learning — 模仿已有示范，但不直接优化"回答质量偏好"

PPO for LLMs 阶段发生了什么？

1. 采样阶段：

- 同一个 prompt，用当前 policy π_θ 采样出多条回答（比如 4-8 条）
- 每条回答是一个 token trajectory

2. RM 评分阶段：

- 完整回答生成完后，reward model 给分
- 回答 A：结构清楚 $\rightarrow R(A) = 0.85$
- 回答 B：混乱堆术语 $\rightarrow R(B) = 0.32$

3. PPO update 阶段：

Advantage signal: $A = R(A) - \text{baseline}$ (来自 critic / value head)

Update rule:

maximize $E[\min(r_\theta(a|s) * A, \text{clip}(r_\theta(a|s), 1-\epsilon, 1+\epsilon) * A)] - \beta * KL(\pi_\theta || \pi_{\text{ref}})$

Where $r_\theta = \pi_\theta / \pi_{\text{old}}$ (importance sampling ratio)

4. KL constraint 的作用：

- 如果某个 token 决策在 SFT checkpoint 里概率很低，但 RM 给了高分
- KL penalty 会惩罚这种"为了刷分而发明怪异模式"的行为
- **本质：RM 指导方向，但不能让它完全接管策略**

对比表格（压缩版）

维度	SFT	PPO for LLMs
优化目标	token-level likelihood	response-level preference + KL constraint
信号来源	gold answer (supervised)	reward model (preference signal)
更新风格	imitation	conservative update against reference
风险	过拟合示范数据	exploit RM / language distribution collapse
防跑偏机制	-	PPO clip + KL penalty

关键差异（一句话）

SFT: "这些 token 序列应该这样排列" (imitation)
PPO for LLMs: "这类完整回答整体更符合偏好 -> 让生成这类回答的概率提升，但不要离 SFT checkpoint 太远"

6. 这一章最后压成一张因果地图 (axioms -> inevitable outcome)

Problem:

SFT 只能模仿示范，RM 能指导偏好但可能 noisy，直接追 reward 会破坏语言能力

Axioms (不可绕过的底层事实):

1. token-level likelihood $\neq$ response-level preference optimization
2. RM is imperfect (noisy, exploitable)
3. LLM's language distribution is fragile to aggressive updates

Forced Problems (如果只接受 axioms，什么矛盾会出现?):

- 只用 SFT $\rightarrow$ 无法直接优化回答级偏好
- 只用 RM + PG $\rightarrow$ exploit RM，语言崩塌
- 需要什么? $\rightarrow$ 在 RM 指导和能力保护之间找平衡点

Inevitable Outcome:

PPO's conservative update (第一层护栏) + KL penalty against reference distribution (第二层护栏)

Compression Mechanism:

$KL(\pi_\theta || \pi_{ref})$ = measure of "how far from SFT capabilities"
 $\rightarrow$ RM 指导优化方向，但不让它完全接管策略

Verification (遮住答案能重建吗?):

Q1: 去掉 KL penalty 会怎样? $\rightarrow$ exploit RM, language collapse

Q2: PPO clip 为什么不够? → 只限制单步 update, 不限制整体分布距离

Q3: Reference model 选哪里? → SFT checkpoint (有语言能力积累)

Example:

SFT: "这些 token 序列应该这样排列" (imitation)

PPO for LLMs: "这类完整回答整体更符合偏好 -> 提升概率, 但 KL constraint 别跑偏"

本章速记卡片 (axioms version)

一句话主线

- **LLM** = token-level policy, complete response = trajectory
- **RM** gives preference signal at response level
- **PPO + KL** is the *inevitable solution* to: optimize preferences without breaking language capabilities

Axioms (必背底层事实)

1. **SFT limitation**: token-level likelihood $\neq$ preference optimization
2. **RM limitation**: noisy, exploitable, may not align with true human preference
3. **LLM fragility**: language distribution is cumulative (pretraining + SFT), aggressive updates break it

Inevitable Outcome (为什么必须这么设计?)

- PPO's clip → 保守更新的第一层护栏 (限制单步幅度)
- KL penalty → 保守更新的第二层护栏 (限制整体分布距离)
- Reference model = SFT checkpoint → "语言能力锚点"

Verification Test (遮住答案能重建吗?)

1. **去掉 KL penalty?** — exploit RM, language collapse
2. **PPO clip 为什么不够?** — only limits single step, not overall distribution distance
3. **Reference model 选哪里? 为什么?** — SFT checkpoint, because it has accumulated language capabilities

最短背诵版 (压缩成因果链)

SFT → 只能 imitation ← problem

RM + PG → exploit RM ← new problem

需要什么? → 在 RM 指导和能力保护之间找平衡点

PPO clip → 限制单步 update (第一层)

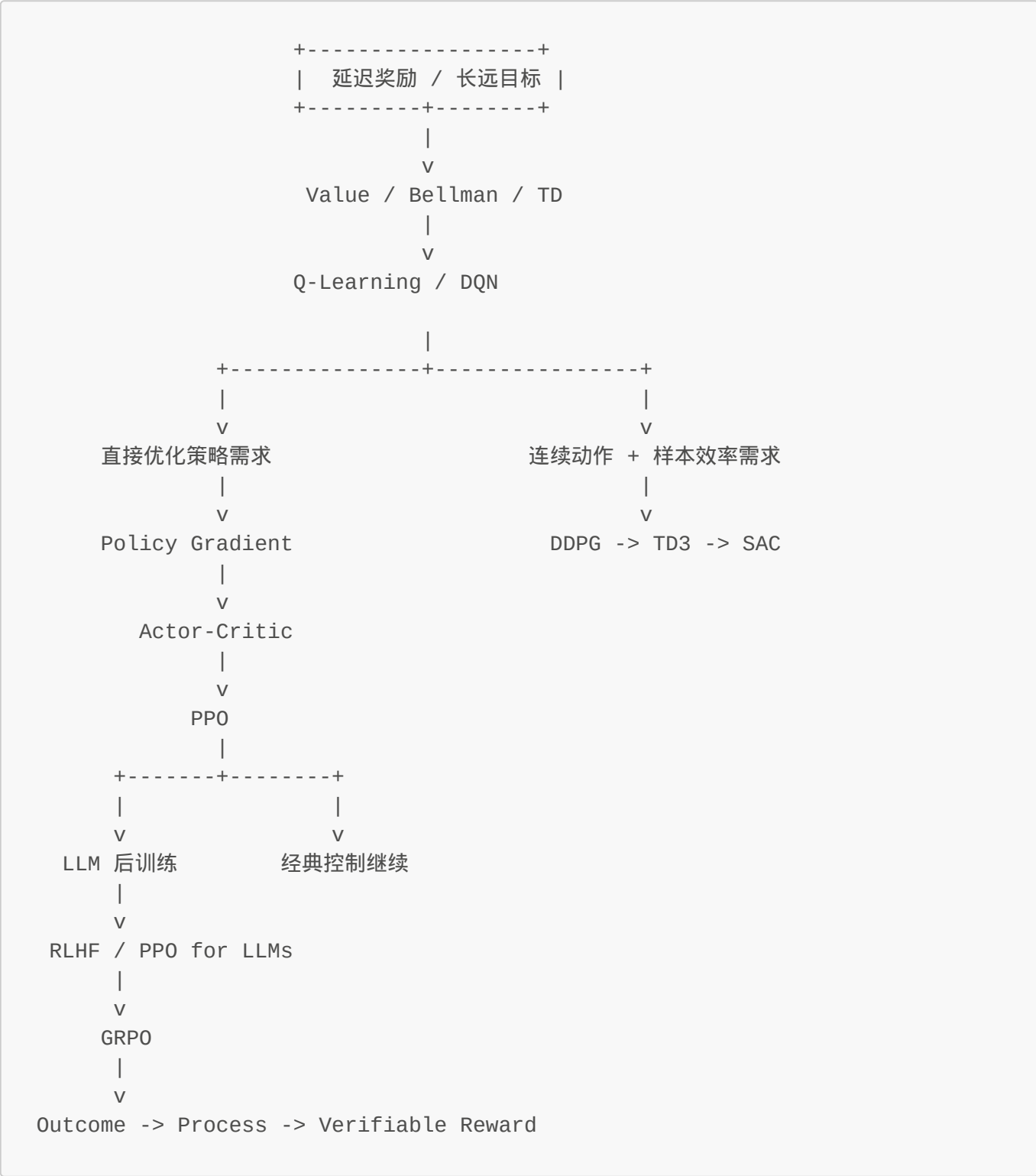
KL penalty → 限制分布距离 (第二层)

Inevitable: PPO for LLMs = conservative update against reference distribution

🔑 **关键 insight:** PPO for LLMs 不是"把 PPO 搬到 LLM", 而是在 **axioms 约束下的 inevitable outcome** ——RM 指导优化方向, Reference model 保护语言能力, PPO 负责保守更新。这个设计本身也会暴露新问题 (成本、工程复杂度), 继续推动 GRPO 等变体诞生。

第十二章：GRPO - 在 LLM 场景里，为什么"组内相对比较"会自然替代一部分 critic 角色？

全局导航图



序：PPO for LLMs 之后，为什么还会继续逼出 GRPO？

第十一章已经把一件事讲清楚：

LLM 可以被看成 policy，PPO 可以接进来做保守优化。

但新问题随之出现：

- rollout 贵
- critic / value 训练复杂
- 回答质量很多时候本来就是相对比较出来的

这就会逼出一个更贴近 LLM 反馈结构的问题：

既然同一个 prompt 下我们本来就会采多条回答并做比较，那能不能直接利用"组内相对优劣"来构造策略更新信号？

一句话压住本质：

GRPO = PPO 的保守更新框架 + 组内相对优势信号

1. Problem：为什么 PPO for LLMs 之后还会继续逼出 GRPO？

1.1 LLM 的反馈天然就是"问题比答案"

最自然的数据形态本来就是：

- 同一个 prompt
- 采样多条 responses
- 比较谁更好

完全依赖 critic 去估绝对 value，会显得绕。

1.2 critic 在大模型后训练里很贵

配一套完整的价值网络：

- value head
- GAE
- 额外稳定训练逻辑

工程复杂度明显上升。

LLM 场景会自然追问：

能不能更多利用"同组比较"本身来构造 baseline，而不是重度依赖单独 critic？

1.3 相对信号往往已经足够

对一个 prompt，4 条回答里谁最好、谁最差、谁比谁略好——这些相对信息往往已经足够驱动策略更新。

GRPO 面对的问题不是"PPO 错了"，而是：

PPO 在 LLM 里还能不能更贴合真实反馈形态、同时更省掉一部分 value 建模负担？

2. Starting Point：如果同组里能直接比较，那 baseline 就不一定非要来自 critic

从第一性原理看，策略更新真正需要的是：

这个样本，相对 baseline 来说，是更值得强化，还是更该削弱？

在 Actor-Critic / PPO 里，这个 baseline 常常来自：

- value function
- advantage estimate
- TD / GAE 等结构

但在 LLM 同 prompt 多采样的场景里，一个更自然的 baseline 是：

- 同组其他回答的平均水平
- 同组中的相对排名
- 组内标准化后的 reward

所以这个出发点很关键：

只要能构造"相对平均更好还是更差"的信号
baseline 不一定非要由 critic 单独学习出来

这就是 **group-relative** 这三个字真正有力量的地方。

3. Invention：GRPO 到底改了 PPO 的哪一层？

3.1 训练单元从"单条 response"变成"同 prompt 下的一组 responses"

GRPO 里更自然的样本组织方式是：

- 固定一个 prompt
- 采样 **k** 条回答
- 对这 **k** 条回答打分
- 在组内做相对比较

于是策略更新不再只看：

- 这一条回答绝对得了多少分

而更看：

- 它在这一组里相对更好多少

3.2 relative advantage 被组内比较逼出来

如果一组 reward 是：

```
[0.9, 0.7, 0.2, -0.1]
```

那最自然的问题不是：

- 第一个样本值多少钱

而是：

- 第一个样本比这一组平均高多少
- 第四个样本比平均低多少

于是你就会得到某种组内相对优势信号：

- 高于组均值 -> 强化
- 低于组均值 -> 削弱

这实际上是在用组内结构充当 baseline。

3.3 PPO 的保守更新思想仍然保留

这一点别丢。

GRPO 不是说：

- 有了组内比较，就可以随便猛更了

恰恰相反，它仍然继承：

- 新旧策略不能一下差太远
- 更新要保守
- 防止模型因为局部噪声突然跑偏

所以最简压缩是：

```
GRPO = PPO 的保守更新框架 + 组内相对优势信号
```

3.4 为什么它特别适合 reasoning / multi-sample 场景？

因为很多推理任务本来就适合：

- 同题多采样
- 比较答案对错、过程质量、格式满足度

这时组内相对信息密度很高，能直接拿来驱动优化。

4. Verification：为什么这种改造在 LLM 场景里是自然的？

4.1 更贴合真实数据形态

LLM 后训练里，人类或 reward pipeline 本来就在做：

- 多回答比较
- 排序
- 胜负判断

GRPO 只是把这种数据结构直接变成优化信号。

4.2 减轻一部分 critic 负担

baseline 的一部分功能，现在由组内平均、相对排名、组内标准化奖励来承担。这能让方法更贴近"比较式反馈"这个现实。

4.3 GRPO 与 PPO 的关系

GRPO 建立在 PPO 的保守更新思想之上，只是把 advantage 构造改得更 LLM-native。

GRPO 不是推翻 PPO，而是 PPO 在 LLM 多样本相对奖励场景下的一种自然特化。

5. Example：同一个数学题，多条推理答案如何驱动 GRPO？

prompt：

解一个二次方程，并写出推导过程。

模型采样 4 条回答：

- A：答案对，过程清楚
- B：答案对，但过程乱
- C：过程看似合理，但中间算错
- D：答案和过程都不对

如果组内 reward 大致是：

A: 0.95
B: 0.75
C: 0.10
D: -0.20

那 GRPO 的直觉就是：

- A/B 这类回答高于组平均 -> 强化
- C/D 低于组平均 -> 削弱
- 同时仍然限制新策略别一步走太猛

也就是说，它直接利用了"问题比较"这件事本身。

6. 这一章最后压成一张脑图

Problem:

PPO for LLMs 依然可能依赖较重的 critic/value 结构, 而 LLM 反馈天然更像同组比较

Starting Point:

策略更新真正需要的是相对 baseline 信号, 而 baseline 不一定非要来自独立 critic

Invention:

同 prompt 多采样 -> 组内比较 -> 构造 relative advantage
再配合 PPO 风格的保守更新

Verification:

它更贴合 LLM 后训练的比较式反馈结构
同时保留了"别走太猛"的核心思想

Example:

同一道数学题下, 多条回答按组内相对质量驱动策略更新

本章速记卡片

一句话主线

- GRPO 建立在 PPO 上
- 它不再那么依赖单独 critic 来给 baseline
- 而是更多利用同 prompt 多样本的组内相对奖励
- 所以它特别适合 LLM reasoning post-training

必背术语

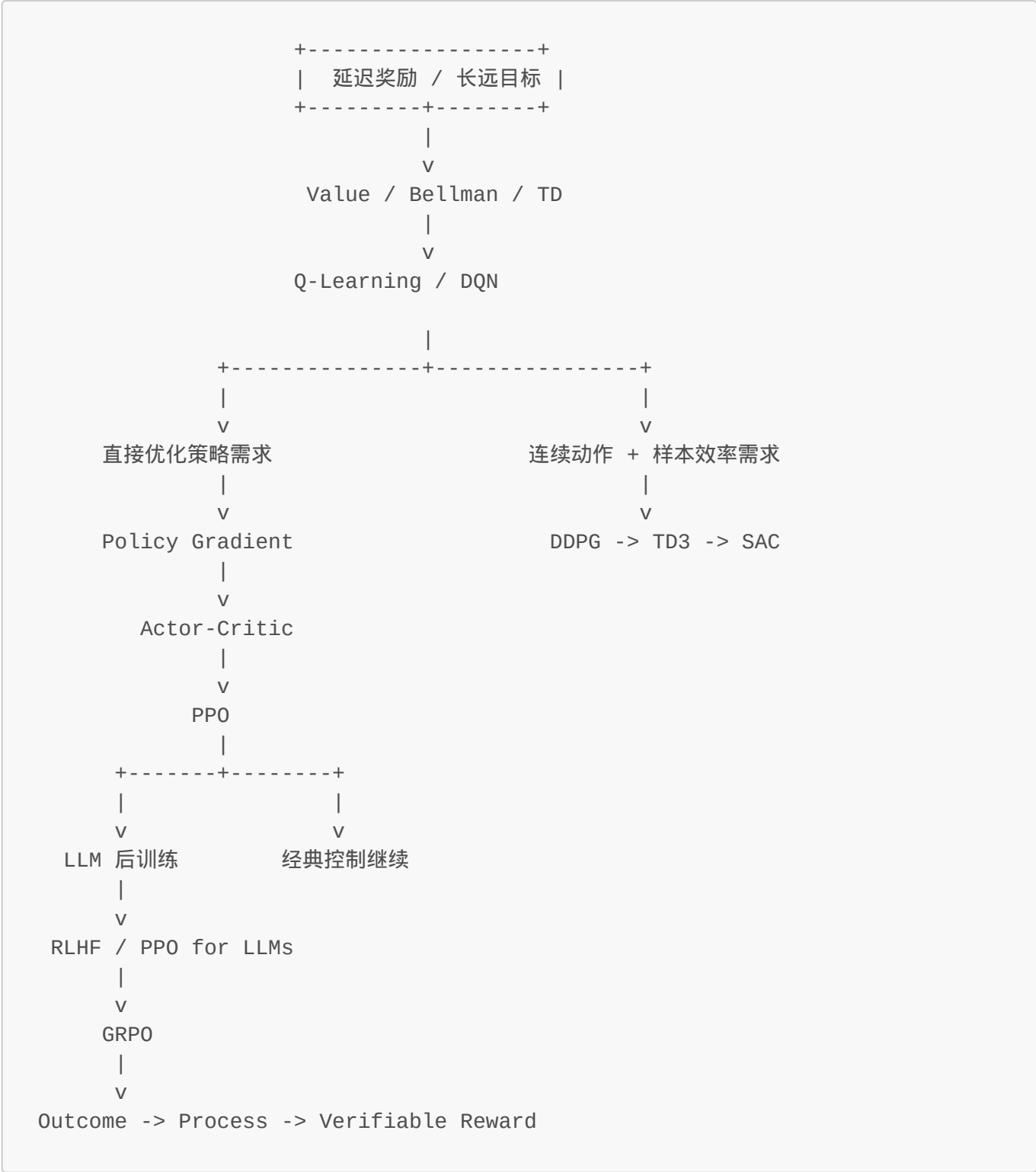
- Group
- Relative Reward
- Relative Advantage
- Baseline
- GRPO

最短背诵版

1. 同一个 prompt 采多条回答
2. 组内比较谁更好
3. 高于组平均的强化, 低于组平均的削弱
4. 同时保留 PPO 的保守更新
5. 这就是 GRPO 的核心直觉

第十三章：DDPG - 为什么连续动作会逼出“确定性 Actor-Critic”？

全局导航图



你在这里：连续控制分支 -> DDPG

序：既然 PPO 能做连续动作，为什么还要 DDPG？

到这里，连续控制分支正式接上了。

一个自然问题：

Policy Gradient / Actor-Critic / PPO 都能处理连续动作，那 DDPG 是干嘛的？

答案只有一个压力点：

样本贵。

一句话先压住：

DDPG 的本质，是把 DQN 的 off-policy 数据复用能力，和 Actor-Critic 的连续动作表达能力拼在一起。

1. Problem：PPO/随机策略路线在连续控制里哪里不够？

1.1 连续动作空间没法像 DQN 那样直接 $\arg\max$

这正是前面 Policy Gradient 出现的原因之一。

1.2 但纯随机策略方法样本效率经常不够高

在机器人控制里，数据贵：

- 采样慢
- 真实环境有代价
- 仿真也不便宜

纯靠 on-policy 新轨迹，成本太高。

1.3 我们其实很想保留 DQN 那种 replay buffer + off-policy 学习能力

因为 DQN 的一个巨大优点是：

- 老数据也能反复学
- 样本可以高复用

所以连续控制的矛盾变成：

我既要能输出连续动作
又想像 DQN 那样高效复用经验

这就自然逼出 DDPG。

2. Starting Point：能不能让 Actor 直接输出连续动作，而 Critic 继续学 Q？

从第一性原理看，最自然的想法是：

- 既然 $\arg\max_a Q(s, a)$ 在连续空间里难做
- 那不如训练一个 Actor，直接给出“当前最该采取的动作”

于是会出现这样的分工：

Actor

直接输出动作：

```
a = \mu_\theta(s)
```

这里 $\mu_\theta(s)$ 是确定性策略。

Critic

学习动作价值：

```
Q_w(s, a)
```

它回答：

- 在状态 s 下，采取动作 a 值多少钱

于是问题就被改写成：

不用在连续动作空间上做暴力 $\arg\max$
而是让 Actor 直接学会近似 $\arg\max$ 的动作输出

3. Invention：DDPG 是怎么拼出来的？

3.1 把 DQN 的 Bellman 训练方式保留给 Critic

Critic 仍然可以像 Q-learning 那样做 Bellman 回归：

```
y = r + \gamma Q_{\text{target}}(s', \mu_{\text{target}}(s'))
```

然后让：

```
L_{\text{critic}} = (Q(s, a) - y)^2
```

这一步说明 DDPG 没有丢掉 value-based 的核心资产：

- bootstrapping
- target network
- replay buffer
- off-policy 学习

3.2 让 Actor 直接朝着让 Q 变大的方向更新

既然 Critic 已经会评估动作值，那 Actor 最自然的目标就是：

输出一个能让 Critic 评分更高的动作。

所以 Actor 会按 $Q(s, \mu(s))$ 的梯度方向更新。

直觉上就是：

- Critic 说这个动作值更高
- 那 Actor 就把输出往这个方向推

3.3 为什么叫 deterministic ?

因为它不是输出动作分布再采样，而是直接输出一个确定动作：

```
a = \mu_\theta(s)
```

探索通常靠额外加噪声完成，比如：

- Gaussian noise
- OU noise

也就是说：

- 训练时：动作 = Actor 输出 + 探索噪声
- 部署时：动作 = Actor 直接输出

3.4 一句话压缩

```
DDPG = DQN 的 off-policy / replay / target 思想  
+ Actor-Critic 的连续动作表达  
+ 确定性策略输出
```

4. Verification：DDPG 真的解决了连续控制里的关键矛盾吗？

4.1 它能不能处理连续动作？

能。

因为 Actor 直接输出连续值，不再需要离散化动作。

4.2 它有没有保留高样本效率？

有一部分。

因为它是 off-policy：

- 旧经验可以重复利用
- replay buffer 很重要

这通常比纯 on-policy 方法更省样本。

4.3 它有没有新的问题？

有，而且不小：

- Critic 容易过估计
- Actor 可能被错误 Critic 带偏
- 训练对超参数很敏感
- 探索质量也常常不稳定

所以 DDPG 是一个桥，但不是终点。

下一章要解决什么？ 就是这些稳定性问题 → **TD3**。

5. Example：机械臂控制连续扭矩

状态：

- 机械臂关节角
- 角速度
- 末端与目标位置关系

动作：

- 每个关节的连续扭矩

如果动作离散化：

- 很粗糙
- 组合爆炸

DDPG 的做法是：

- Actor 直接输出一组连续扭矩
- Critic 评价这组扭矩值不值钱
- replay buffer 反复复用过往经验

这就很适合“动作连续且样本贵”的任务。

6. 这一章最后压成一张脑图

Problem:

连续动作没法直接 $\arg\max$ ，但 on-policy 随机策略又可能太费样本

Starting Point:

让 Actor 直接输出动作，让 Critic 继续学 Q

Invention:

用 deterministic policy 输出连续动作

保留 replay buffer / target network / off-policy Q-learning 思想

Verification:
它兼顾了连续动作表达和较高样本复用
但稳定性仍然不够，容易过估计

Example:
机械臂连续扭矩控制

本章速记卡片

一句话主线

- 连续动作逼出了 Actor
- 样本效率需求又逼回了 DQN 的 off-policy 思想
- **DDPG** 就是这两条线的拼接
- 它能做连续控制，但稳定性还有问题

必背术语

- **Deterministic Policy**
- **Replay Buffer**
- **Target Network**
- **Off-Policy**
- **DDPG**

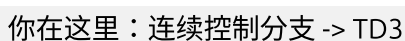
最短背诵版

1. Actor 直接输出连续动作
2. Critic 评估这个动作值多少钱
3. 老经验可以反复学
4. 所以样本效率较高
5. 但训练容易不稳定

第十四章：TD3 - 为什么 DDPG 会被"双 Critic + 延迟更新"继续修正？

全局导航图





DDPG 解决了连续动作空间的问题，但落地时暴露出一个致命缺陷：

Critic 高估某些动作的价值 → Actor 追着假高分跑 → 错误被自我强化

当 DDPG 的过估计和耦合问题积累到一定程度，系统会被迫引入更保守的 Critic 机制和延迟更新来稳住训练。

1.1 单个 Critic 的误差会被 Actor 放大

- Actor 会主动朝那个动作靠
- 结果越来越多数据围绕这个假高分区域产生

- 错误被自我强化

1.2 Bellman + function approximation 很容易产生 overestimation

只要：

- 目标值里有 $\max$ / 贪心倾向
- 函数逼近有噪声

就容易把价值高估。

在连续控制里，虽然不是显式 $\max_a Q(s, a)$ ，但 Actor 实际上在学"让 Q 尽量大"的动作，所以问题并没有消失。

1.3 Actor 和 Critic 如果同步猛更，会彼此追着跑

Critic 还没评稳，Actor 就跟着动；Actor 一动，Critic 学习分布又变。

所以系统不稳的根源可以压成：

Q 容易高估
+ Actor 会追高估值
+ 两个网络更新耦合太紧

2. Starting Point：如果估值容易虚高，那就主动让 Critic 更保守一点

从第一性原理看，我们真正想要的不是"更乐观的值函数"，而是：

宁愿稍微保守一点，也别让 Actor 被假高分带偏。

于是自然会想到两个修正方向：

- 不要只信一个 Critic
- 不要让 Actor 每一步都追着还没稳定的 Critic 跑

这就是 TD3 的起点。

3. Invention：TD3 的三个关键补丁是怎么长出来的？

3.1 Twin Critics：两个 Critic 取更小值

TD3 训练两个 Critic：

$Q_1(s, a)$, $Q_2(s, a)$

构造 target 时取更保守的那个：

$$y = r + \gamma \min(Q1_{\text{target}}(s', a'), Q2_{\text{target}}(s', a'))$$

这一步的直觉非常清楚：

- 如果一个 Critic 偶然虚高
- 另一个没那么高
- 取最小值可以压住过估计

3.2 Delayed Policy Update : Actor 更新更慢

TD3 不让 Actor 每一步都更新，而是：

- Critic 多更新几次
- Actor 少更新一次

这相当于在说：

先把评分系统校准得更稳一点，再让决策系统跟着动。

3.3 Target Policy Smoothing : 给 target action 也加一点平滑噪声

这是第三个补丁。

在计算目标值时，对 target action 加一点小噪声，可以避免 Critic 把非常尖锐、脆弱的动作峰值当成真最优。

直觉上就是：

- 真正好的策略应该在一个小邻域内都还行
- 不是只对一个极窄动作点突然爆高分

3.4 一句话压缩

TD3 = 双 Critic 压过估计
+ 延迟 Actor 更新降耦合
+ target action 平滑防尖峰骗分

4. Verification : 这些补丁为什么真的有效？

4.1 双 Critic 为什么能减轻过估计？

因为高估通常是噪声往上偏造成的。取两个值里更小的那个，会让 target 更保守。

4.2 延迟更新为什么有用？

因为 Actor 不再每一步都追着不稳定的 Critic 跑。

这样可以降低：

- 错误快速放大
- 两个网络相互追逐导致震荡

4.3 平滑 target action 为什么重要？

因为它在防一种很危险的情况：

- Critic 在某个极窄动作点给出异常高值
- Actor 学着钻这个尖峰漏洞

平滑以后，只有周围也不错的动作区域才更可能被认为真优。

4.4 TD3 是不是终点？

也不是。

它依然主要是：

- deterministic policy
- 依赖外加噪声探索

而探索本身仍然可能不够自然。

这会继续逼出 SAC。

5. Example：一个具体的方向盘控制场景

场景设定：

状态 s = [车速，横向偏移，航向角误差]
动作 a = 方向盘转角 ($-30^\circ \sim +30^\circ$)
目标：沿车道中心线行驶

DDPG 会怎么崩？

假设在某个弯道场景下：

1. Critic 偶然给 $a=25^\circ$ 这个极端角度打了一个虚高分数（可能是噪声）
2. Actor 学着往 25° 方向猛转方向盘
3. 结果车辆失控，但系统已经学会了"极端转角=高分"的错误映射

TD3 怎么稳住？

1. **双 Critic**：Q1 说 25° 值得 80 分，Q2 说只值 40 分 → 取 40 分
2. **延迟更新**：Actor 等 Critic 多跑几轮校准后再动
3. **平滑 target**：给 25° 加噪声后平均打分，发现周围动作都差 → 这个尖峰是假的

结果：训练曲线更稳，最终策略不会追逐虚假高分。

6. 这一章最后压成一张脑图

Problem:

DDPG 的单 Critic + 同步更新 → 过估计被 Actor 快速放大

Starting Point:

宁愿更保守，也别让错误价值被系统自我强化

Invention:

双 Critic 取较小值 + 延迟 Actor 更新 + target action smoothing

Verification:

训练曲线更稳，收敛率提升，对超参数鲁棒性增强

Example:

方向盘控制中，避免追逐虚假的极端角度尖峰

本章速记卡片

一句话主线

- DDPG 的核心问题是过估计和耦合过紧
- TD3 用三个补丁一起修它
- 目标不是更激进，而是更保守、更稳
- 所以 TD3 常被看作 DDPG 的稳定增强版

必背术语

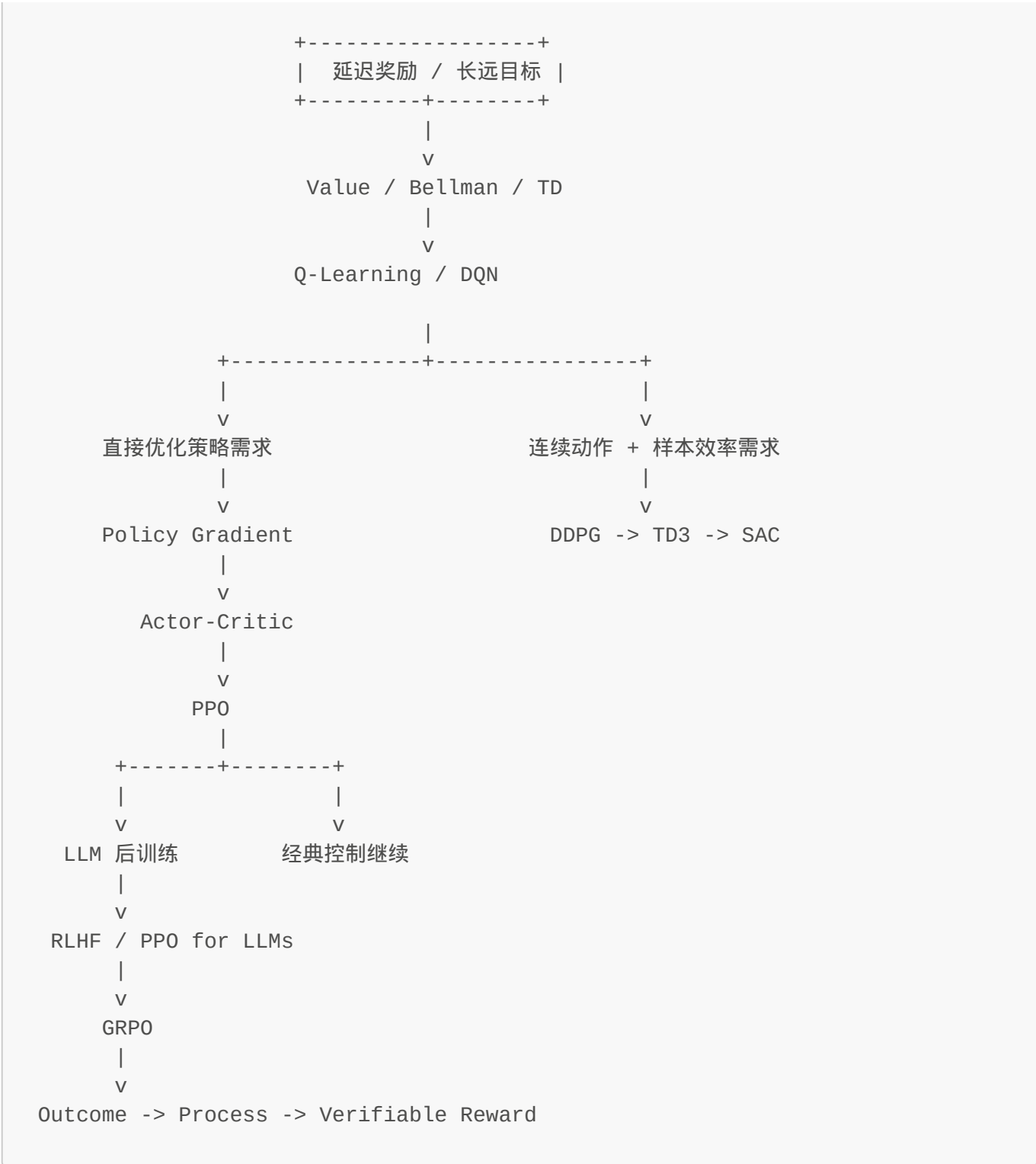
- Twin Critics
- Delayed Policy Update
- Target Policy Smoothing
- Overestimation
- TD3

最短背诵版

1. 两个 Critic 比一个更稳
2. Actor 不要每步都更新
3. target action 也要做平滑
4. 核心目的是压过估计
5. TD3 比 DDPG 更稳定

第十五章：SAC - 为什么最大熵原则会逼出"既学回报，也保留随机性"的算法？

全局导航图



你在这里：连续控制分支 -> SAC

序：如果探索不该只是"外加噪声"，那随机性就该进目标函数本身

到了 TD3，你会发现它虽然更稳了，但还有一个深层限制：

探索主要还是靠外加噪声，而不是策略本身天然愿意保持合理随机性。

这会逼出一个更根本的问题：

能不能把"探索"从训练技巧，升级成优化目标的一部分？

一句话先压住：

SAC 的本质，是不只最大化 reward，还同时鼓励策略保持较高 entropy，让 agent 在学高回报行为时仍然保留有价值的随机性。

1. Problem：为什么 DDPG / TD3 之后还会继续逼出 SAC？

1.1 deterministic policy 的探索太依赖手工噪声

在 DDPG / TD3 里，常见做法是：

- Actor 给出一个确定动作
- 再手动加噪声做探索

这当然能工作，但它有个问题：

- 探索和目标函数本身是分离的

也就是说，模型本身并没有学到：

- 什么时候应该更确定
- 什么时候应该保留随机性

1.2 真实任务里，随机性有时不是缺陷，而是资产

有些环境需要：

- 多模态动作选择
- 避免过早陷入局部最优
- 在不确定状态下保留尝试空间

如果策略太早塌缩成单点动作，可能会：

- 探索不足
- 容易卡住

1.3 所以"探索"不该只是外挂

这就把问题逼得很明确：

如果随机性本身有价值，那它就应该直接出现在优化目标里。

2. Starting Point：既然随机性有价值，就把 entropy 也当成被奖励的对象

从第一性原理看，reward 最大化目标只表达了一件事：

- 追求高回报

但它没有表达：

- 保持策略分布的丰富性

于是最自然的扩展就是：

不仅要高 reward
还要高 entropy

熵 **entropy** 在这里可以粗略理解成：

- 策略分布有多分散
- 行为有多不那么死板

所以 SAC 的出发点就是：

把"探索价值"写进目标函数，而不是只靠训练时手工加噪声。

3. Invention：从 axioms 出发，SAC 为什么必须这样存在？

3.0 第一步：找到不可约的 axioms（基础事实）

Axiom 1: reward maximization → 追求高回报是目标
Axiom 2: exploration is necessary → 不探索就无法发现更好的策略
Axiom 3: noise $\neq$ intrinsic randomness → 外挂噪声不是策略本身的随机性

3.1 第二步：如果只有 axioms，会引出什么矛盾？

从 Axiom 1 + Axiom 2 出发：

- reward maximization 会自然导向确定性策略（选最大 Q 值的动作）
- exploration necessary 需要保持随机性
- **矛盾！** 最大化 reward 和保持探索性是内在冲突的

DDPG / TD3 的做法：

确定性 policy → 外挂噪声 → 强行探索

但这不是从目标函数本身解决，而是训练技巧层面的补丁。

3.2 第三步：唯一合理的解决方案路径是什么？

如果"探索有价值"这个事实成立（Axiom 2），那有且只有一种方式真正内生它：

Option A: 外挂噪声 → ✗ 不是目标函数的一部分
Option B: entropy 进目标函数 → ✔ 这才是内生化的唯一路径

为什么必须是 Option B？因为：

- 优化器只能优化"被明确表达的目标"
- 如果探索有价值，它就必须出现在 objective 里
- 所以 **maximize reward + entropy** 是唯一合理的推导结果

3.3 第四步：如何让它可扩展（compression mechanisms）？

纯加 entropy 项会导致数值不稳定。SAC 引入了：

$$L_{\text{SAC}} = \underset{\substack{\uparrow \\ \text{reward 目标}}}{E[\text{reward}]} + \alpha * \underset{\substack{\uparrow \\ \text{entropy 正则化}}}{H(\pi)}$$

α : temperature parameter $\rightarrow$ 控制随机性 vs 回报的权衡

这就是 SAC 最终形式的推导链条。

3.4 policy 重新变成 stochastic

和 DDPG / TD3 的确定性策略不同：

DDPG/TD3: $\pi(s) \rightarrow a$ (确定动作)
 SAC: $\pi(a|s) \sim N(\mu, \sigma^2)$ (概率分布, 从中采样)

这意味着：

- 探索是 policy 自身的属性
- entropy 可以被显式计算和优化
- 模型学到"什么时候该随机、什么时候该确定"

3.5 Critic 的 Bellman target 也要调整

因为 objective 变了，Bellman 方程也要对应调整：

$$\begin{aligned} \text{传统 Q-learning: } Q(s, a) &= E[r + \gamma Q(s', a')] \\ \text{SAC Q-learning: } Q(s, a) &= E[r + \gamma(Q(s', a') - \alpha \log \pi(a'|s'))] \\ &\quad \uparrow \qquad \qquad \qquad \uparrow \\ &\quad \text{entropy 修正项} \end{aligned}$$

这说明 SAC 不是"局部修补"，而是从目标函数到 Bellman target 的完整一致性重构。

3.6 一句话压缩推导链

Axiom: reward + exploration are both necessary
Contradiction: deterministic policy kills exploration
Solution: entropy must enter objective (only one way)
Result: SAC = off-policy Actor-Critic with stochastic $\pi + \alpha H(\pi)$

3.7 检验理解：能否独立重推？

试着隐藏上面的推导，自己重建：

1. DDPG/TD3 的确定性策略有什么根本问题？（探索是外挂）
2. 如果探索有价值，它必须出现在哪里？（目标函数里）
3. 如何表达"随机性价值"？（entropy）
4. 所以 objective 是什么？（reward + α *entropy）
5. policy 要改成什么形式？（stochastic）

如果能独立推出这 5 步，说明理解了 SAC 的必然性。

4. Verification：为什么最大熵这条路确实解决了前面的深层问题？

4.1 它有没有让探索变成目标内生的一部分？

有。

这正是 SAC 和 DDPG / TD3 的最大哲学差别：

- 前者：探索是目标的一部分
- 后者：探索更多是外挂噪声

4.2 它能不能缓解过早收缩到差策略？

通常能。

因为高熵目标会惩罚过快塌缩，让策略在 early stage 保留更多尝试空间。

4.3 它是不是就一定比 TD3 好？

也不是绝对。

不同任务里：

- 有时 TD3 更简单直接
- 有时 SAC 更稳、更强

但从思想层面看，SAC 解决的是一个更深的问题：

如何让探索不再只是训练技巧，而成为优化目标的一部分。

5. Example：移动机器人导航

状态：

- 位置
- 速度
- 雷达 / 传感器信息
- 目标相对方向

动作：

- 连续线速度
- 连续角速度

如果策略太早确定成一种走法：

- 可能会陷入局部路线
- 遇到新障碍适应差

SAC 会倾向于：

- 在学习高 reward 路径的同时
- 维持一定策略随机性
- 不那么快塌成唯一动作模板

这对复杂环境探索很有价值。

6. 这一章最后压成一张脑图

Problem:

DDPG / TD3 的探索主要靠外加噪声，随机性没有进入目标函数

Starting Point:

如果随机性本身有价值，那 entropy 就应该被显式奖励

Invention:

最大化 reward + entropy

使用 stochastic policy，并保留 off-policy Actor-Critic 结构

Verification:

探索从外挂技巧变成目标内生部分

能缓解策略过早塌缩

Example:

移动机器人导航时，保持合理随机性有助于找到更稳健路径

本章速记卡片

一句话主线

- SAC 不是只学高 reward

- 它还显式鼓励高 entropy
- 所以探索不再只是外加噪声
- 这让它在很多连续控制任务里又稳又强

必背术语

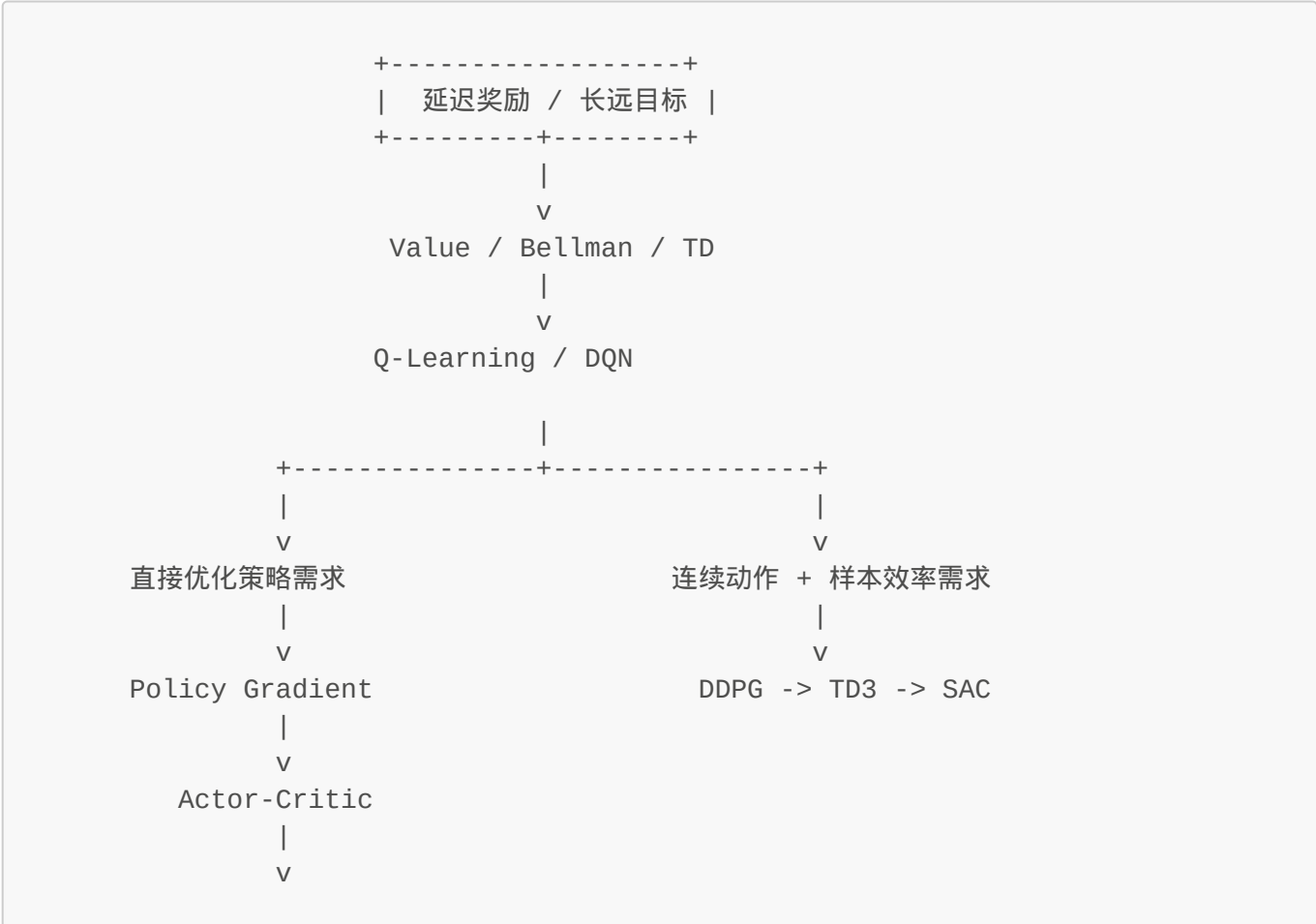
- Entropy
- Temperature alpha
- Stochastic Policy
- Maximum Entropy RL
- SAC

最短背诵版

1. 不只要高 reward
2. 还要保留随机性
3. 所以 entropy 直接进目标函数
4. policy 重新变成 stochastic
5. 这就是 SAC 的核心

第十六章：Outcome Reward - 为什么"只看最终答案对不对"既强大又不够？

全局导航图





你在这里：LLM reasoning reward 分支 -> Outcome Reward

序：当任务可以自动判分时，reward 突然变得更可靠了

到了 LLM reasoning 这条线，一个非常重要的变化是：

有些任务的正确性，不需要人类主观比较，系统自己就能验。

比如：

- 数学题答案对不对
- 代码单测过不过
- SQL 查询结果对不对
- 形式化证明是否通过检查器

这会带来一个巨大的训练信号升级：

- reward 不再只是"人类偏好像不像"
- 而是可以直接来自任务结果是否正确

一句话先压住：

Outcome Reward 的本质，是把"最终结果是否正确"直接变成奖励信号。它很强，因为可验证；但它也不够，因为它不告诉你中间过程到底哪一步好或坏。

1. Problem：为什么 RLHF / 偏好模型之后，还会继续逼出 outcome reward？

1.1 偏好信号很有用，但它仍然可能主观、稀疏、昂贵

人类比较回答谁更好，当然重要，但它有天然成本：

- 标注贵
- 主观性强
- 一致性有限
- 对复杂任务不一定判得稳

1.2 某些任务其实有"硬答案"

在数学、代码、工具调用等任务里，很多时候你根本不需要人类说：

- 这个回答看起来更好

你可以直接问：

- 答案对吗？
- 测试过了吗？
- 执行结果匹配吗？

这意味着：

有一类任务的 reward 可以由 verifier 自动产生。

1.3 这类信号通常比主观偏好更硬

因为它不是"像不像好回答"，而是：

- 真通过 / 假通过
- 真正确 / 真错误

这会让训练信号在某些场景里变得非常有力。

所以问题可以压成一句：

既然有些任务可以自动验证最终结果，为什么不直接用结果正确性做 reward？

2. Starting Point：如果结果能验证，那就别只让模型"看起来会"，要让它"真的做对"

从第一性原理看，训练目标应该尽量贴近最终任务。

如果最终任务要的是：

- 算对
- 证明对
- 代码跑通

那最自然的目标就不该只是：

- 看起来像个好解释

而是：

- **最终结果真的正确**

于是 reward 可以写得非常直接：

```
结果对 -> 高 reward
结果错 -> 低 reward
```

这一步特别重要，因为它让训练目标和真实任务成功标准几乎重合。

3. Invention：Outcome Reward 是怎么工作的？

3.1 先生成完整回答或完整解题轨迹

模型先对一个任务生成：

- 一条完整答案
- 或一条完整推理 + 最终结论

3.2 再把最终结果交给 verifier

验证器可能是：

- 数学答案匹配器
- 代码测试器
- 执行环境
- 规则检查器

它输出的信号可能是：

- 1 / 0
- 通过 / 不通过
- 或更细一点的结果分数

3.3 把这个结果级信号作为 reward

于是训练目标变成：

- 更倾向生成那些最终能被验证通过的回答

在 reasoning 训练里，这特别强，因为它直接在优化：

不是"像会推理"，而是"最后真把题做对"。

4. Verification：为什么 outcome reward 很强，但仍然不够？（以及你怎么验证自己懂了）

4.1 它为什么强？

因为它可验证、客观、可规模化。

相比人类偏好：

- 更便宜
- 更一致
- 更不容易被主观风格干扰

思考检查点：

如果你要把 outcome reward 向别人解释，能不能只说"结果对就奖，错就不奖"？如果不够，缺了什么？

4.2 它为什么不够？

因为它通常只在最后给分。

这就意味着：

- 中间过程哪一步有帮助，它不一定知道
- 错了也只知道"最后错了"
- credit assignment 仍然很粗糙

也就是说，它解决了"reward 靠不靠谱"的问题，但没完全解决：

正确结果是怎么被一步步做出来的？

验证理解（遮住答案后思考）：

1. 如果 reward 只在最后给，那模型怎么知道中间哪步走错了？
2. 这和前面 Q-learning 的 credit assignment 问题有什么相似/不同？
3. 如果你要改进 outcome reward，从第一性原理出发会怎么做？

（答案：1. 不知道；2. 都稀疏，但 outcome 至少客观；3. 过程也要给分 -> process reward）

这就自然过渡到下一章：**Process Reward**。

本章理解检查：

- ☐ 能用自己的话说清 outcome reward 的"强"和"不够"分别是什么
- ☐ 能说清楚 verifier 在训练里扮演什么角色
- ☐ 明白为什么这章之后还要 process reward

5. Example：代码生成为什么特别适合 outcome reward？

prompt：

写一个函数，返回数组中的最大值。

模型生成代码后，系统直接跑单测：

- 测试全过 -> 高 reward
- 有 case 挂掉 -> 低 reward

这里 verifier 并不关心：

- 注释漂不漂亮
- 措辞像不像教程

它只关心：

- 代码功能是否正确

这就是 outcome reward 的典型力量。

6. 这一章最后压成一张脑图

Problem:

偏好信号有用，但在可验证任务里仍然太主观、太贵

Starting Point:

如果最终结果能自动验证，就应该直接优化结果正确性

Invention:

生成完整答案 -> verifier 检查最终结果 -> 结果对错作为 reward

Verification:

outcome reward 客观、强、可扩展

但 credit assignment 仍然偏粗

Example:

代码生成用单测是否通过来给 reward

本章速记卡片

一句话主线

- Outcome Reward 直接奖励最终结果是否正确
- 它比纯偏好信号更硬、更客观
- 特别适合数学、代码、工具调用等可验证任务
- 但它仍然不知道中间哪一步推理是好是坏

必背术语

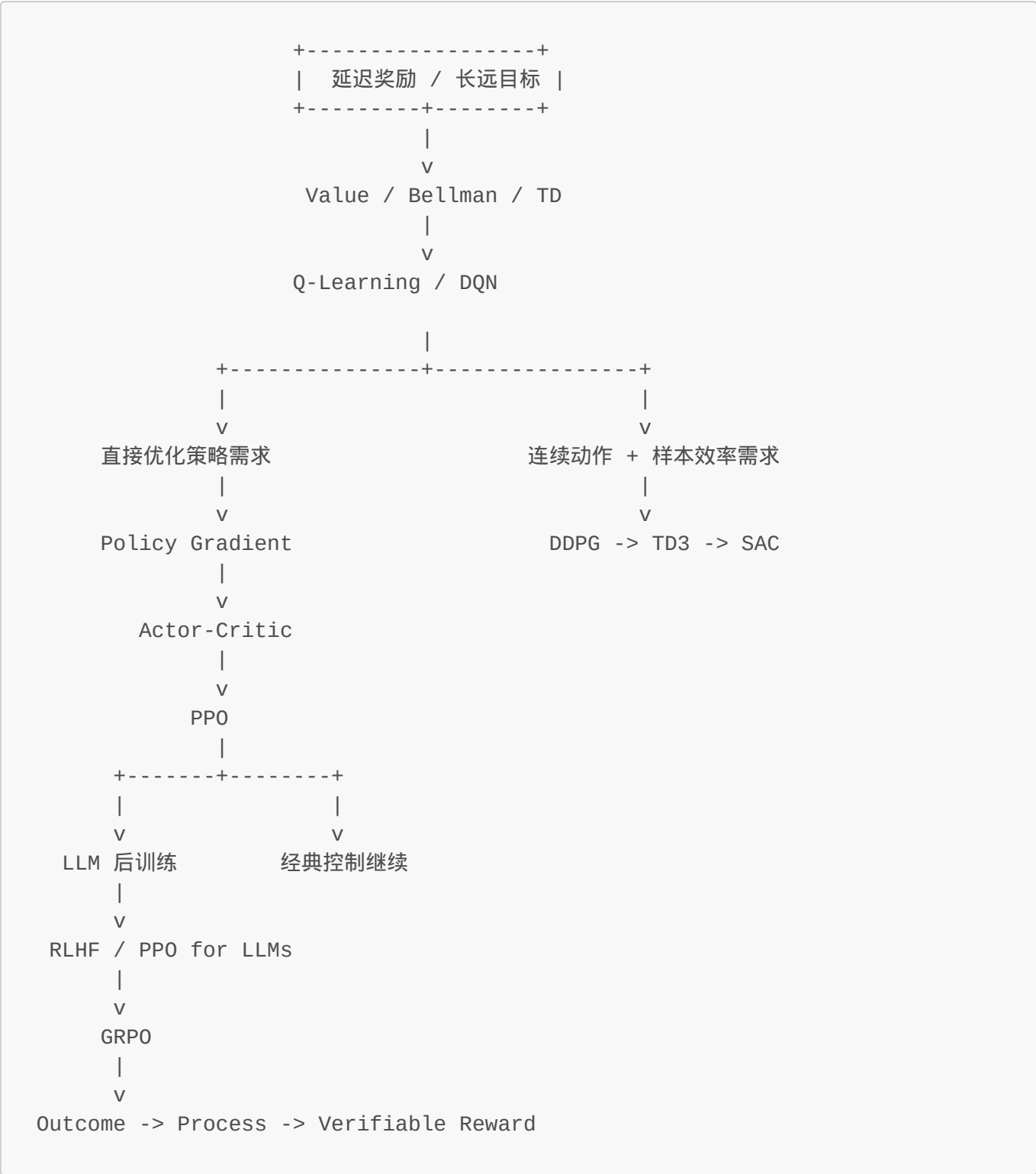
- Verifier
- Outcome Reward
- Final Answer
- Pass/Fail Signal
- Credit Assignment

最短背诵版

1. 最终结果能验
2. 就直接按结果给 reward
3. 这很强，因为客观
4. 但只看结尾，过程信息太少
5. 所以还会逼出 process reward

第十七章：Process Reward - 为什么只奖最终答案还不够？

全局导航图



你在这里：LLM reasoning reward 分支 -> Process Reward

序：如果推理质量决定泛化性，那训练信号就不能只在终点出现

outcome reward 有一个致命的盲区：它无法区分"真的会"和"碰巧对"。

在数学题里，模型可能中间乱推、最后蒙对。在代码生成里，逻辑极脆但恰好过测试。

如果你只看 outcome，这些投机行为都会拿到高 reward —— 而训练出来的模型会学会投机，而不是推理。

Process Reward 的必然性从这里开始：

如果最终能力来自过程质量，那训练信号就必须作用在过程上。

1. Problem：Outcome Reward 为什么还不够？

1.1 最终结果对，不等于推理过程好

在数学题里，模型可能：

- 中间乱推
- 最后碰巧写对

在代码里，模型可能：

- 逻辑极脆
- 恰好过了当前测试

如果你只看 outcome，那这些都可能拿到高 reward。

1.2 只看终点，credit assignment 太粗

如果最后错了，你不知道：

- 是第 2 步错了
- 还是第 8 步错了
- 还是前面都对，最后抄错了数字

所以 outcome reward 虽然硬，但信息太晚、太粗。

1.3 reasoning 能力的关键，常常在中间过程

如果我们真正想让模型学会：

- 拆问题
- 保持逻辑一致
- 逐步验证
- 少走歪路

那训练信号就不能只在最后一秒钟出现。

于是问题变成：

能不能让"中间推理步骤"本身也获得奖励或惩罚？

2. Starting Point：从第一性原理看，为什么 outcome reward 必然失效？

已知事实：

- 1. 最终答案由中间推理步骤生成（因果链）
- 2. Reward 梯度通过时间/序列反向传播到策略更新
- 3. Credit assignment 的粒度决定了哪些行为会被强化

推导矛盾：

- 如果只在终点给 reward → credit assignment 只能分配一个标量值给整个轨迹
- 但不同步骤对最终结果的贡献度不同（有些关键，有些冗余）
- 投机性行为（shortcut learning）和真正推理在 outcome 层面可能无法区分

强制性问题出现：

如果我要训练"可泛化的推理能力"而非"碰巧答对的技巧"，我必须让 reward signal 的粒度匹配推理过程的粒度。

这就是 process reward 被迫出来的原因——不是"更好"，而是 outcome reward 在 reasoning 任务上根本不够。

3. Invention：Process Reward 如何从必然性问题推导出机制？

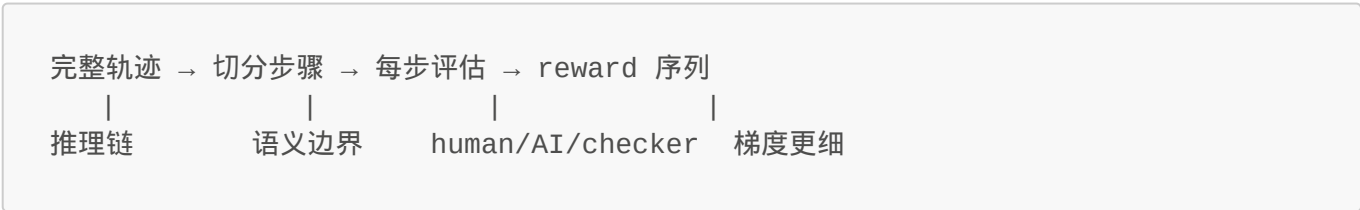
3.1 核心压缩机制：分布式细粒度信号

outcome reward 的瓶颈是：单点标量值无法编码整个推理轨迹的质量分布。

process reward 的解法是把 credit assignment 问题空间化：

- 时间维度上：把"最后一步分配奖励"变成"每一步都可能信号"
- 信息维度上：把"对/错二元结果"变成"步骤质量连续谱系"

3.2 实现路径（如何把推理拆成可监督的单元？）



评分来源可以是：

- 人类标注：专家标注每个中间步骤是否正确/合理
- 规则检查器：数学推导的格式验证、逻辑一致性检查
- 强模型 critique：用更强的 model 对弱模型的每一步做判断
- 可验证子目标：某些步骤本身有自动验证标准（如公式代入是否匹配）

3.3 策略更新的目标转变

outcome reward：优化 P(最后答案正确 | 输入)

process reward：优化 P(每步推理合理 | 前序状态)

关键差异：

- **更细的 credit assignment** → 错误定位精确到步骤级
- **更强的可泛化性** → 学的是"稳健推理结构"而非"特定题目套路"
- **抑制投机行为** → shortcut learning 在过程中就会被惩罚

Process reward 的本质压缩：**把"credit assignment 问题"从单点标量分配变成分布式序列信号。**

4. Verification：能否独立重新推导 process reward 的必要性？

验证测试：隐藏所有关于 process reward 的知识，只从基本原理出发。

4.1 重演必然性链条

1. **起点：**我想训练模型学会推理（而非碰巧答对）
2. **已知约束：**
 - Reward signal 决定哪些行为被强化
 - Credit assignment 的粒度决定学习的精度
3. **观察矛盾：**outcome reward 只能给整个轨迹一个标量值
4. **推论：**如果两个轨迹 outcome 相同但过程质量不同，outcome reward 无法区分它们
5. **强制结论：**要训练"高质量推理过程"，reward signal 必须在过程中出现

✓ 能够独立重演 → process reward 不是"更好的选择"，而是**必然选择**。

4.2 反例验证：不用 process reward 会怎样？

- 模型学会投机（shortcut learning）
- 在训练数据上表现好，但泛化到新题型时崩塌
- 推理过程不稳定、不可解释

这正是 outcome-only reward 的失败模式。

4.3 代价与边界

process reward 不是免费午餐：

- **标注成本高：**需要步骤级监督信号
- **评估器本身可能有噪声：**如果 step evaluator 错了，会误导模型
- **切分困难：**有些推理的"步骤边界"是模糊的

所以在高价值 reasoning 任务（数学、代码生成、复杂规划）里值得投入，但在简单任务上可能过度工程化。

5. Example：数学题里的 outcome vs process

题目：解方程 $x^2 - 5x + 6 = 0$

模型 A：

- 中间因式分解正确
- 推理清晰
- 最后答案正确

模型 B：

- 中间乱写
- 最后碰巧给出 $x=2, 3$

如果只看 outcome：

- A 和 B 都可能得高分

如果加上 process reward：

- A 会明显更高
- B 会被压下去

这就是为什么 process reward 更适合训练真正稳的 reasoning。

6. 这一章最后压成一张脑图

Problem:

outcome reward 无法区分"稳健推理"和"投机碰巧", 因为单点标量值丢失过程信息

Starting Point:

奖励信号的粒度必须匹配推理过程的粒度 (第一性原理)

Invention:

把 credit assignment 从"单点分配"压缩成"分布式序列信号"

Verification:

能独立重演必然性链条: 要训练可泛化推理 $\rightarrow$ reward 必须在过程中出现

Example:

数学题里, 过程正确的解法 vs 碰巧蒙对答案, process reward 能区分它们

本章速记卡片

一句话主线

- outcome reward 只能给整个轨迹一个标量值
- process reward 让每一步推理都能获得信号
- 它训练的是"可泛化的推理结构"而非"碰巧答对的技巧"
- 代价: 标注成本高、评估器可能有噪声、步骤切分困难

必背术语

- **Process Reward** - 过程级奖励信号
- **Step-Level Signal** - 步骤粒度的监督信号
- **Credit Assignment Granularity** - credit assignment 的粒度问题
- **Shortcut Learning** - 投机性行为 (被 process reward 抑制)
- **Process Supervision** - 过程监督范式

最短背诵版（5 行压缩）

1. outcome reward 无法区分"真会"和"碰巧对"
2. credit assignment 粒度必须匹配推理过程粒度
3. process reward = 分布式细粒度信号替代单点标量值
4. 训练可泛化推理结构，抑制 shortcut learning
5. 代价高但必要性强（不是更好，是 outcome-only 根本不够）

第十八章：Verifiable Reward - 为什么 reasoning 训练越来越依赖"可自动检查"的中间与最终信号？

📍 定位

Outcome Reward → Process Reward → **Verifiable Reward**

本章聚焦：如何把 reward 变成可验证、可自动化、可规模扩展的信号。

序：最理想的 reward，不只是强，还要便宜、稳定、可扩展

LLM reasoning post-training 正在朝一个方向收敛：

尽量把 reward 变成可验证、可自动化、可规模扩展的信号。

人类偏好、outcome reward、process reward——都在追求同一个目标：**给模型稳定、可信、可大规模供应的训练反馈。**

核心原则：

Verifiable Reward = 用可自动检查的事实约束代替主观印象

1. Problem：为什么 reasoning 训练越来越偏向 verifiable reward？

人工标注的瓶颈：

- 贵、慢、一致性差

复杂推理任务的需求：数学、编程、定理证明、工具使用——这些场景需要海量训练数据，人工覆盖不过来。

结论：reward 必须可自动判、可重复判、规则明确。这样才能：

- 更稳
- 更便宜
- 更可扩展

于是"verifiable"成了 reward 设计的核心标准。

2. Starting Point：任务能形式化，就把判分外包给 verifier

奖励的职责不是"优雅地描述任务"，而是：**稳定地区分更好和更差的行为。**

verifier 可以作用在：

- 最终答案
- 中间步骤
- 格式约束
- 工具调用结果
- 外部环境反馈

所以：

Verifiable reward = 一种 reward 设计原则，不是单一算法

3. Invention：Verifiable Reward 在实践里怎么落地？

3.1 最终结果检查

数学答案比对、单元测试、执行结果比对。

3.2 中间过程检查

子步骤公式是否成立、中间程序状态是否正确、推理链条是否满足局部规则。

3.3 结构与格式检查

JSON 是否合法、工具调用参数是否匹配 schema、是否遵守输出协议。

3.4 混合奖励（最常见）

实践里很少单一 reward，通常是：

- 偏好 reward + outcome reward + process reward + verifier signal

这层底座原则很简单：**凡是能自动判的地方，就尽量别只靠主观打分。**

4. Verification：这条路为什么重要？有边界吗？

好处：

- 信号更稳（规则检查比人类印象一致）
- 扩展性更强（verifier 能批量跑，数据规模大很多）

边界：不是所有任务都容易验证。开放式写作、审美表达、长篇创意——这些仍需偏好或人工判断。

结论：在能验证的任务上，verifiable reward 是主力；在难验证的任务上，它和偏好信号并存。

5. Example：现代 reasoning 训练管线的真实形态

代码智能体训练例子：

- **Layer 1**：输出格式是否合法（JSON/schema）
- **Layer 2**：跑单测看最终行为是否正确
- **Layer 3**：检查关键中间步骤是否满足约束
- **Layer 4**（可选）：偏好模型评估可读性和帮助性

reward 不再是单一分数，而是 layered system：格式正确性 + 过程正确性 + 结果正确性 + 人类偏好。这就是 verifiable reward 思路的现代形态。

6. 这一章最后压成一张脑图

Problem:

reasoning 训练需要大规模、稳定、可信的奖励信号，纯人工偏好不够便宜也不够稳

Starting Point:

只要任务能部分形式化，就应该尽量把判分交给 verifier

Invention:

把最终结果检查、过程检查、格式检查等自动信号都纳入 reward 设计

Verification:

这提高了稳定性和扩展性

但开放式任务仍需要偏好或人工信号补充

Example:

代码智能体训练同时结合格式、过程、结果与偏好四层奖励

本章速记卡片

一句话主线

- **Verifiable Reward** 不是单一算法，而是一种 reward 设计原则
- 能自动检查的地方，就尽量交给 verifier
- 它让 reasoning 训练更稳、更便宜、更可扩展
- 但开放式任务仍然需要偏好信号配合

必背术语

- **Verifier**
- **Verifiable Reward**
- **Outcome Signal**
- **Process Signal**
- **Hybrid Reward**

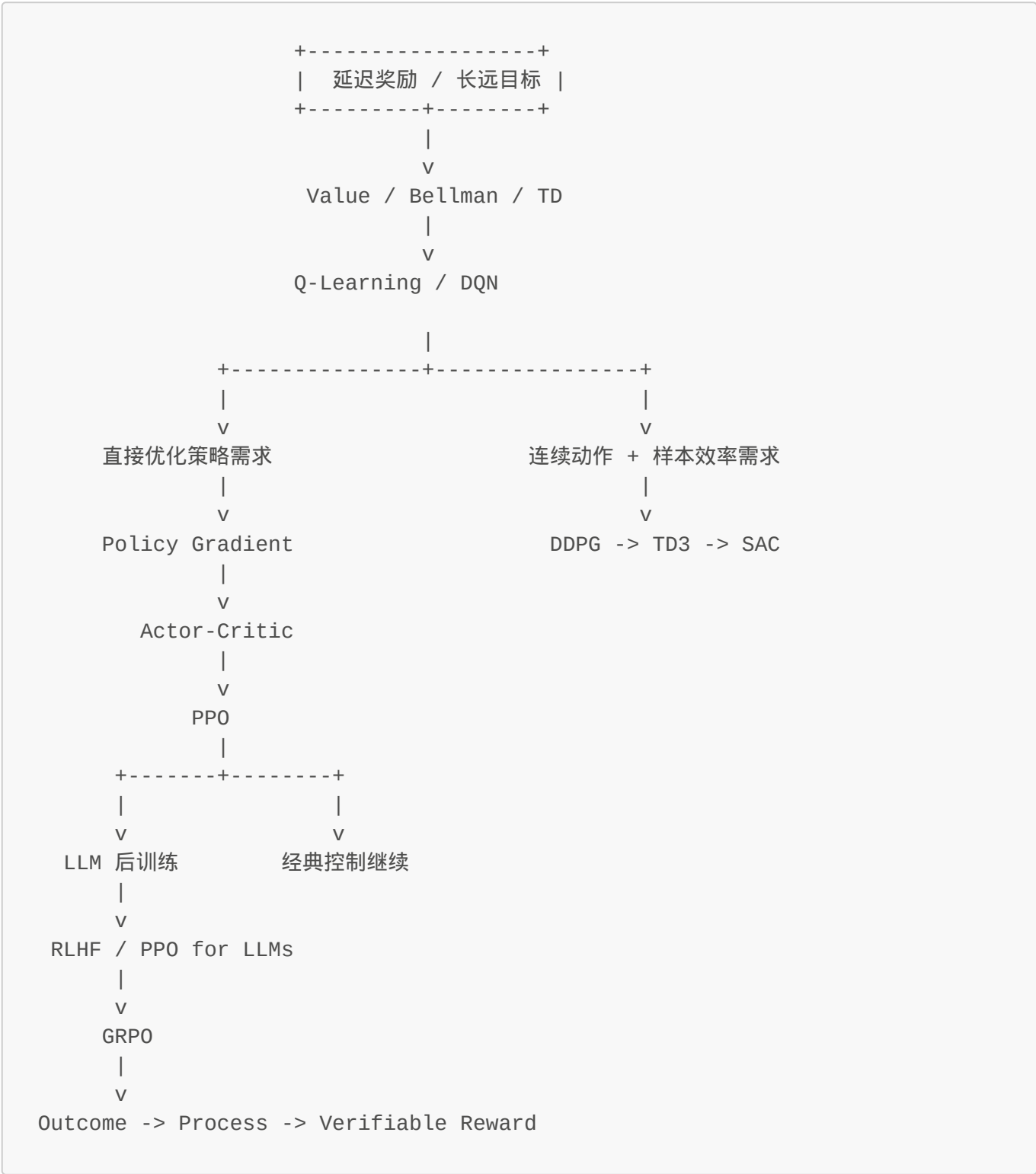
最短背诵版

1. 奖励最好能自动检查

- 2. 最终结果、过程、格式都可以验
- 3. 这样训练更稳定、更可扩展
- 4. 但不是所有任务都能完全验证
- 5. 所以 verifier 和偏好通常会并存

第十九章：总复习总览 - 把整本 RL 教程压成一张因果地图

全局导航图



序：别把 RL 学成算法名单，要把它学成“一个问题不断逼出下一个问题”的系统

如果学到这里，脑子里只剩：

- Bellman
- Q-Learning
- DQN
- PPO
- GRPO
- SAC

那还不算真正吃透。

真正该留下来的，不是“我记住了多少算法名字”，而是：

我能不能说清楚：每一个新方法，到底是被前一个方法的什么矛盾逼出来的？

这一章不再展开新算法，而是把整本书压成一张统一主线：

```
problem -> starting point -> invention -> verification -> example
```

一句话先压住：

整本 RL 教程的核心，不是从简单算法一路堆到复杂算法，而是围绕三个永恒矛盾不断演化：怎么评价未来、怎么优化行为、怎么让学习更稳定更高效。

1. 第一条主线：Value 路线到底在解决什么？

最早的问题是：

如果 reward 常常延迟出现，agent 怎么知道现在的动作值不值得做？

这逼出了：

- Value Function
- $Q(s, a)$
- Bellman Equation

这条线的核心发明不是某个公式本身，而是一个思想：

把长远回报压缩成局部递推关系。

于是后面自然长出：

- Bellman Update
- TD Error
- Q-Learning

再往后，当表格装不下状态空间时：

- **DQN** 被逼出来

所以这条线可以压成：

延迟奖励

- > 需要未来价值
- > Bellman 递推
- > Q-Learning 可学习化
- > DQN 神经网络化

1.5 全书术语约定：别让名字差异掩盖了本质

为了防止读到后面被不同社区的叫法搞晕，这里统一一下：

- **policy** = 策略 = 决定在状态下怎么行动的规则；在 LLM 里就是 `p_theta(token | context)`
- **value function** = 价值函数 = 对未来回报的估计；**Critic** 是它在 Actor-Critic 结构里的角色名
- **reward model / preference model**：在这本教程里默认近似等价，统一偏向写 **reward model**
- **outcome reward / process reward / verifiable reward** 不是互斥算法名，而是三种不同粒度的奖励设计思路

如果你能穿透这些命名差异，看见背后的功能角色，那说明你真的没有被术语牵着跑。

2. 第二条主线：Policy 路线到底在解决什么？

Value 路线虽然强，但很快遇到另一个矛盾：

如果最终要优化的是行为策略，为什么总要绕道先学 value ？

特别是在连续动作场景里，这个问题更尖锐，因为：

- **argmax_a Q(s, a)** 变难
- 动作分布本身也变重要

这逼出了：

- **Policy Gradient**

它的关键思想是：

直接把策略本身写成优化对象。

但紧接着又遇到新问题：

- 直接用 **G_t** 更新，方差太大

于是逼出：

- **Actor-Critic**

它把问题改写成：

- Actor 负责行动
- Critic 负责评价
- Advantage 成为二者桥梁

再往后，更新虽然方向对了，却可能走太猛：

- 于是 PPO 出现

所以这条线可以压成：

最终目标是策略

- > 直接优化 policy
- > 方差太大
- > 加 Critic 稳定反馈
- > 更新过猛
- > PPO 加安全护栏

3. 第三条主线：连续控制路线到底在解决什么？

到了连续控制，问题又换了一层：

我既想处理连续动作，又想保留 off-policy 的高样本效率。

这逼出了：

- DDPG

它的本质是：

- Actor 直接输出连续动作
- Critic 继续学 Q
- replay buffer / target network 保留下来

但 DDPG 很快暴露出：

- 过估计
- Actor 造假高分
- 系统耦合过紧

于是逼出：

- TD3

它用：

- 双 Critic
- 延迟更新
- target smoothing

继续修正。

再往后，大家意识到一个更深的问题：

探索不该只是外挂噪声，而应该进入目标函数本身。

于是逼出：

- SAC

它的关键思想是：

- 不只追 reward
- 还追 entropy

所以连续控制这条线可以压成：

```
连续动作 + 样本效率需求
-> DDPG
-> 过估计与不稳定
-> TD3
-> 探索不该只是外挂
-> SAC
```

4. 第四条主线：LLM 后训练路线到底在解决什么？

当我们从经典 RL 切到 LLM，会遇到一个完全不同表面的任务，但底层矛盾其实很熟悉：

模型会生成文本，不等于模型会按人类偏好生成有用文本。

这逼出了：

- SFT
- Preference Data
- Reward Model
- RLHF

它的核心是：

把“人类偏好”显式接进训练目标。

接着又会遇到：

- reward model noisy
- 更新不能太猛

于是 PPO for LLMs 接上来。

再往后，LLM 场景会进一步提出：

既然同一个 prompt 下本来就会采多条回答并做比较，baseline 能不能更多来自组内相对信号？

于是 GRPO 被逼出来。

这条线可以压成：

```
会续写 != 会按偏好回答
-> SFT + preference modeling
-> RLHF
-> PPO for LLMs
-> group-relative feedback
-> GRPO
```

5. 第五条主线：Reasoning reward 路线到底在解决什么？

到了 reasoning 训练，问题又进一步细化：

“看起来会推理”不等于“最后真的做对”；“最后做对”也不等于“过程真的稳”。

先是发现：

- 某些任务最终结果可以自动验证

于是逼出：

- Outcome Reward

但只看最后结果，又太粗：

- 不知道中间哪一步好坏

于是逼出：

- Process Reward

再往后，整个领域会收敛到一个更一般的设计原则：

凡是能自动检查的地方，就尽量别只靠主观偏好。

于是 Verifiable Reward 成为一整类奖励设计哲学。

这条线可以压成：

```
偏好不够硬
-> outcome reward
-> 终点信号太粗
-> process reward
-> 尽量把奖励建立在可验证信号上
-> verifiable reward
```

6. 整本书真正的总地图：所有算法都在解哪三类矛盾？

如果把全书彻底压缩，其实一直在反复解决三类矛盾。

6.1 矛盾一：未来太远，怎么评价？

对应路线：

- Value
- Bellman
- TD
- Q-Learning
- Critic

关键词：

- 未来价值
- bootstrapping
- credit assignment

6.2 矛盾二：真正要优化的是行为，怎么直接动策略？

对应路线：

- Policy Gradient
- Actor-Critic
- PPO
- PPO for LLMs
- GRPO

关键词：

- policy optimization
- advantage
- conservative update

6.3 矛盾三：训练太不稳、太贵、太吵，怎么让它可用？

对应路线：

- Replay Buffer
- Target Network
- Twin Critics
- Entropy Regularization
- Verifier-based Reward

关键词：

- 稳定性
- 样本效率
- 可扩展性

所以整本书最短总结其实是：

RL = 评价未来 + 优化行为 + 稳定学习

所有算法都只是这三件事在不同任务表面下的具体实现。

7. 学完这本书后，你脑子里最该留下哪几句话？

1. Bellman 的本质

- 长远问题被压缩成局部递推

2. Q-Learning / DQN 的本质

- 学动作价值，再从价值里长出策略

3. Policy Gradient 的本质

- 直接优化策略，而不是绕道 value

4. Actor-Critic 的本质

- 行动模块和评价模块协作

5. PPO 的本质

- 更新方向对，但步子不能太大

6. DDPG / TD3 / SAC 的本质

- 连续动作场景下，围绕样本效率、稳定性、探索方式不断修正

7. RLHF / PPO for LLMs / GRPO 的本质

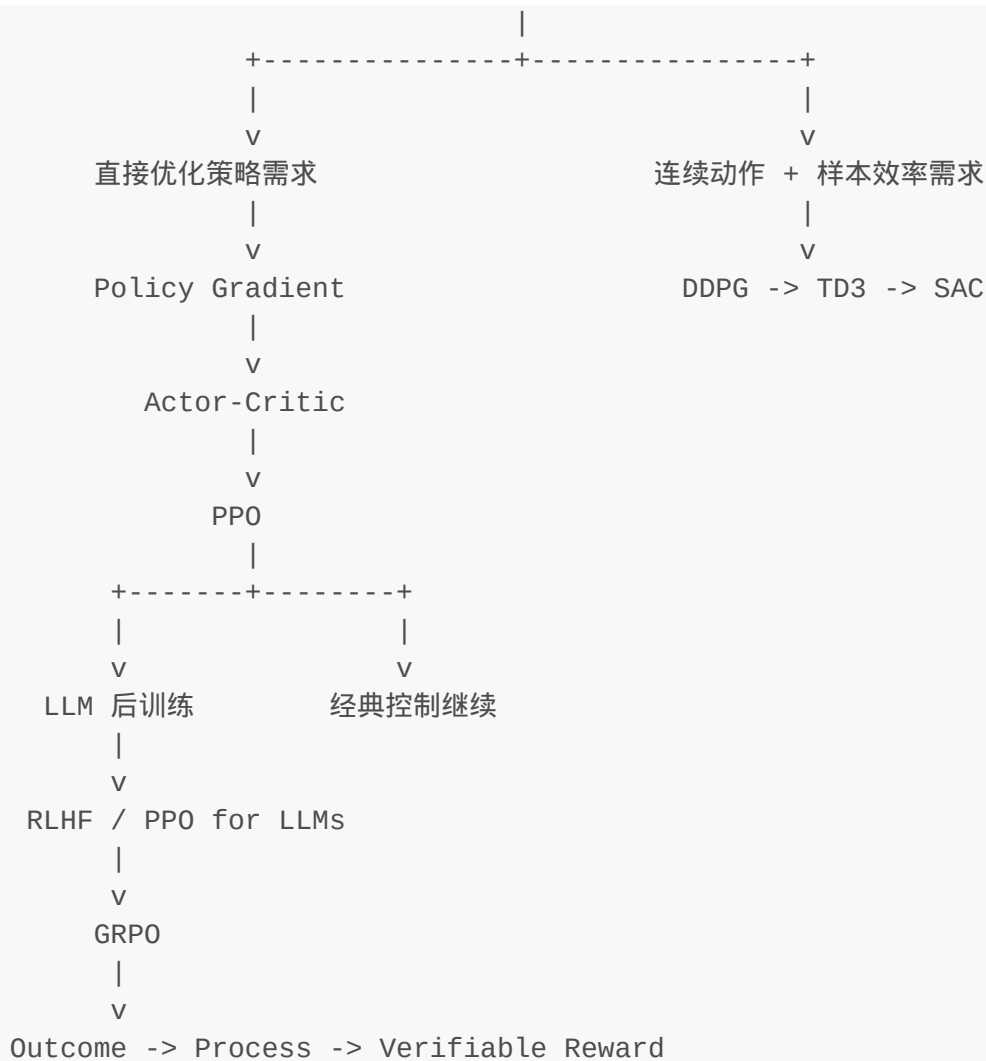
- 把 policy optimization 搬进 LLM 后训练，并让奖励结构更贴近偏好与组相对比较

8. Outcome / Process / Verifiable Reward 的本质

- 尽量让 reasoning 训练依赖更硬、更细、更可扩展的反馈信号

8. 如果把整本书压成一张 ASCII 图





9. 这一章最后压成一张脑图

Problem:

学完很多算法后，容易只剩名字，失去因果主线

Starting Point:

每个算法都不是孤立知识点，而是前一个矛盾逼出的下一个解法

Invention:

把全书重组为五条因果路线: value / policy / continuous control / LLM alignment / reasoning reward

Verification:

如果你能用“前一个方法留下了什么问题 -> 下一个方法为什么必然出现”来复述全书，说明你真的吃透了

Example:

整本书可以压成一张 ASCII 因果地图，而不是一串算法清单

一句话主线

- RL 不是算法堆砌，而是矛盾驱动的发明史
- 全书一直在反复解决三件事：评价未来、优化行为、稳定学习
- 经典控制、连续控制、LLM 后训练，本质上只是这些矛盾在不同表面的展开
- 真正学会的标志，是你能从问题重新推回算法，而不是只会背名字

必背总句

RL = 评价未来 + 优化行为 + 稳定学习

最短背诵版

1. Bellman 解决“未来怎么压缩”
2. Policy Gradient 解决“策略怎么直接学”
3. PPO 解决“更新怎么别太猛”
4. DDPG/TD3/SAC 解决“连续动作怎么高效又稳定”
5. RLHF/GRPO/Verifiable Reward 解决“LLM reasoning 怎么拿到更可靠反馈”

路线图

- ✓ 第一章：RL 是什么 (Agent / Environment / State / Action / Reward)
- ✓ 第二章：Policy π (确定性 vs 随机性)
- ✓ 第三章：Value Function $V(s)$ 和 $Q(s,a)$, Bellman 方程
- ✓ 第四章：Q-Learning - 自动学出 Q 表的算法
- ✓ 第五章：Deep Q-Network (DQN) - 用神经网络替代 Q 表
- ✓ 第六章：Policy Gradient - 直接优化策略
- ✓ 第七章：Actor-Critic - 结合 Policy 和 Value
- ✓ 第八章：PPO - 给策略更新套上“安全护栏”
- ✓ 第九章：LLM 分支入口 - 为什么 GRPO 应该放在 PPO 之后？
- ✓ 第十章：从经典 RL 到 LLM 对齐 - 为什么语言模型也会走到 RLHF？
- ✓ 第十一章：PPO for LLMs - 当动作变成 token, PPO 还在优化什么？
- ✓ 第十二章：GRPO - 在 LLM 场景里，为什么“组内相对比较”会自然替代一部分 critic 角色？
- ✓ 第十三章：DDPG - 为什么连续动作会逼出“确定性 Actor-Critic”？
- ✓ 第十四章：TD3 - 为什么 DDPG 会被“双 Critic + 延迟更新”继续修正？
- ✓ 第十五章：SAC - 为什么最大熵原则会逼出“既学回报，也保留随机性”的算法？
- ✓ 第十六章：Outcome Reward - 为什么“只看最终答案对不对”既强大又不够？
- ✓ 第十七章：Process Reward - 为什么只奖最终答案还不够？
- ✓ 第十八章：Verifiable Reward - 为什么 reasoning 训练越来越依赖“可自动检查”的中间与最终信号？
- ✓ 第十九章：总复习总览 - 把整本 RL 教程压成一张因果地图

当前结构：

- 主干：Bellman -> Q-Learning -> DQN -> Policy Gradient -> Actor-Critic -> PPO
- LLM 分支：RLHF -> PPO for LLMs -> GRPO -> outcome reward -> process reward

```
-> verifiable reward
- 连续控制分支：DDPG -> TD3 -> SAC
```

生成时间：2026-03-11 | 持续更新中 

学习路径建议

经典 RL 路径

1. 第1-4章 - 基础概念 → Q-Learning
2. 第5章 - DQN (神经网络版 Q-Learning)
3. 第6-8章 - Policy Gradient → Actor-Critic → PPO

连续动作控制路径

- 第13-15章 - DDPG → TD3 → SAC

LLM / Reasoning 路径

- 第9-12章 - LLM 后训练 → RLHF → GRPO
- 第16-18章 - Outcome → Process → Verifiable Reward

总复习

- 第19章 - 整本教程的知识地图

本教程采用"问题驱动式第一性理解"方式编写，每个概念都从"为什么要解决什么问题"出发，而非直接堆砌公式。